

The Tradelatest Book (Grok Review Pass)

Canonical Knowledge Book — full narrative. Part VII Research = Chapters 19-20 (programs results table + platform data map).

Generated 2026-08-07 from docs/book/
Grok review pass export — reading convenience only.
On conflict, repository code/config wins.

Table of Contents

The Tradelatest Book (Grok Review Pass)	1
Table of Contents	2
The Tradelatest Book - Table of Contents & Guide	13
The Tradelatest Book	13
How to read this	13
Grok review pass (2026-08-07)	13
Table of Contents	13
Part I - Foundations	13
Part II - Market Understanding	14
Part III - Market Semantics	14
Part IV - Decision Making	14
Part V - Execution	14
Part VI - Governance	15
Part VII - Research	15
Part VIII - Agent Intelligence	15
Part IX - Toward Full Repository Understanding	15
Appendix	16
Concept Index (Canonical Concept Rule)	16
How this book evolves	17
Chapter 00 - Quick Start: One Candle's Journey	18
Why this chapter exists	18
What problem it solves	18
What you need to already know	18
The idea	18
The system in one sentence	18
The happy path (one candle)	18
Priority order (not "profit first")	19
What is actually load-bearing today	19
How config becomes production	19
How to read the rest of the book	19
Known defects you should not ignore (even on a quick read)	20
Classification	20
Authoritative sources	20
Unresolved questions	20
Chapter 01 - Why Tradelatest Exists	21
Why this chapter exists	21
What problem it solves	21
What you need to already know	21
The idea	21
The one-paragraph goal	21
Profit is not the primary objective - read that literally	21
The economic target, when it does apply	22
Pattern interpreters are measured, never asserted	22
Classification	22
Authoritative sources	22
Unresolved questions	22
Chapter 02 - The Invariants and the Happy Flow	24
Why this chapter exists	24
What problem it solves	24
What you need to already know	24
The idea	24
The happy flow	24
The seven invariants	24
The deviation policy	25

The async "kitchen feeders"	25
Classification	25
Authoritative sources	26
Unresolved questions	26
Chapter 03 - How This Book Fits the Repository's Knowledge	27
Why this chapter exists	27
What problem it solves	27
What you need to already know	27
The idea	27
The repository already has record systems - this book is a new one, not a replacement	27
The memory hierarchy still governs when you're about to edit code	27
Topics vs. this book	28
The classification vocabulary this book uses	28
Classification	29
Authoritative sources	29
Unresolved questions	29
Chapter 04 - Architecture at a Glance	30
Why this chapter exists	30
What problem it solves	30
What you need to already know	30
The idea	30
The repository is not one clean package	30
The 36 src/ subpackages	31
Orientation map - packages without a dedicated chapter (Grok review pass)	31
The dependency shape, at the top level	32
Two companion generated docs worth knowing about	32
Classification	32
Authoritative sources	33
Unresolved questions	33
Chapter 05 - Data Ingestion and the No-Lookahead Discipline	34
Why this chapter exists	34
What problem it solves	34
What you need to already know	34
The idea	34
Where a candle comes from	34
The four-layer integrity stack	34
The honest caveat: L3 doesn't run everywhere	34
Classification	35
Authoritative sources	35
Unresolved questions	35
Chapter 06 - The Market Ontology: Canonical Semantic Authority	36
Why this chapter exists	36
What problem it solves	36
What you need to already know	36
The idea	36
What the ontology actually is	36
Why "authority" here means something specific	36
Downstream reach	37
Classification	37
Authoritative sources	37
Unresolved questions	37
Chapter 07 - The Feature Pipeline and the Canonical Vector	38
Why this chapter exists	38
What problem it solves	38
What you need to already know	38
The idea	38
What the pipeline does	38

A live doc-drift, caught and flagged (not silently fixed)	38
Who consumes the vector	39
Classification	39
Authoritative sources	39
Unresolved questions	39
Chapter 08 - The CRT State Machine: the Spine	40
Why this chapter exists	40
What problem it solves	40
What you need to already know	40
The idea	40
Two files named "CRT engine" - know which one you're reading	40
The nine-state lifecycle	40
CRT is a closed research surface - what that does and doesn't mean	41
Classification	41
Authoritative sources	41
Unresolved questions	41
Chapter 09 - Interpreters and the Pattern Contract	42
Why this chapter exists	42
What problem it solves	42
What you need to already know	42
The idea	42
The Interpreter Contract (Level 4)	42
The one interpreter that's actually been run through it	42
What this means for other interpreters	42
Classification	42
Authoritative sources	43
Unresolved questions	43
Chapter 10 - The Four Scoring Engines	44
Why this chapter exists	44
What problem it solves	44
What you need to already know	44
The idea	44
CRT - see Chapter 8	44
Gaussian - a channel that has largely collapsed to a constant	44
Zone Gate - a real geometric hard gate, honestly measured as not adding edge	45
RR - not actually a reward:risk engine	45
Classification	45
Authoritative sources	45
Unresolved questions	46
Chapter 11 - Fusion: Combining Independent Signals	47
Why this chapter exists	47
What problem it solves	47
What you need to already know	47
The idea	47
A layered scorer, not a weighted average	47
What Fusion is, honestly, right now	47
Classification	48
Authoritative sources	48
Unresolved questions	48
Chapter 12 - The Decision Engine: Semantic Approval Only	49
Why this chapter exists	49
What problem it solves	49
What you need to already know	49
The idea	49
What it decides	49
What it used to decide, and why that broke everything silently	49
How it was resolved - an ownership decision, not a threshold tweak	49

Classification	50
Authoritative sources	50
Unresolved questions	50
Chapter 13 - The Execution Planner: From Decision to Order Geometry	51
Why this chapter exists	51
What problem it solves	51
What you need to already know	51
The idea	51
A classifier, not a geometry engine	51
An open architectural caveat: which config the planner's geometry actually sees	51
A second open caveat: SL geometry may be using the wrong units	51
Proposed remediation (Grok review pass - options only; no code in this book)	52
ATR unit mismatch - proposed options	52
F-057 CRTConfig programmatic split-brain - proposed options	52
What this book still will not do	53
Classification	53
Authoritative sources	53
Unresolved questions	53
Chapter 14 - Ultron Risk Gate: the Final Capital Check	54
Why this chapter exists	54
What problem it solves	54
What you need to already know	54
The idea	54
What it checks	54
Where reward:risk economics actually live	54
The one module INOUT touches	54
Classification	54
Authoritative sources	55
Unresolved questions	55
Chapter 15 - Live Execution and INOUT	56
Why this chapter exists	56
What problem it solves	56
What you need to already know	56
The idea	56
The live tick entry point	56
src/execution/loop.py - real code, no caller yet	56
INOUT naming collision - RESOLVED (Grok review pass, 2026-08-07)	56
Classification	57
Authoritative sources	57
Unresolved questions	57
Chapter 16 - Config-First Doctrine and the Promotion Path	58
Why this chapter exists	58
What problem it solves	58
What you need to already know	58
The idea	58
Config-First: the target end-state	58
The hard rule: no silent defaults	58
The Authority Ladder - tunable is not the same as trusted	59
The promotion path itself	59
Classification	59
Authoritative sources	59
Unresolved questions	60
Chapter 17 - Truth Maintenance: Findings, Closure, and Authority	61
Why this chapter exists	61
What problem it solves	61
What you need to already know	61
The idea	61

The seven rules	61
Findings: the unit of validated belief	61
Epistemic Integrity - catching overclaims before they're registered	61
Closure & Authority: what "CLOSED" and "AUDITED" actually mean	62
Classification	62
Authoritative sources	62
Unresolved questions	63
Chapter 18 - A Field Guide to docs/governance/	64
Why this chapter exists	64
What problem it solves	64
What you need to already know	64
The idea	64
This is mostly an evidence ledger, not narrative documentation	64
The clusters	64
MSIP brief - Market State Interpretation Platform (Grok review pass)	65
How to actually use this directory	66
Classification	66
Authoritative sources	66
Unresolved questions	66
Chapter 19 - The Research Programs: Falsification as a Discipline	67
Why this chapter exists	67
What problem it solves	67
What you need to already know	67
The idea	67
Research results at a glance (data table)	67
The gate every program runs through	70
The ladder, program by program (narrative)	70
Program 9 in depth (Grok review pass - closes the coverage gap)	70
Why this is the right outcome, not a broken pipeline	71
Classification	71
Authoritative sources	72
Unresolved questions	72
Chapter 20 - The Research Platform: src/research/	73
Why this chapter exists	73
What problem it solves	73
What you need to already know	73
The idea	73
Research data map (where things live on disk)	73
The shape of src/research/	74
The Edge Research Platform (ERP)	74
An honest gap: the Research Measurement Contract exists on paper, not yet in code	74
Proposed path to close the gap (Grok review - roadmap, not an implement claim)	75
Reports that check the platform's own hygiene	75
Classification	75
Authoritative sources	76
Unresolved questions	76
Chapter 21 - The AI Automation Agent	77
Why this chapter exists	77
What problem it solves	77
What you need to already know	77
The idea	77
Five modes, each with its own tool surface	77
Deterministic planning, not generated planning	77
Implementation depth sketch (Grok review pass - still orientation, not a full rewrite)	77
Everything is logged, twice	78
Classification	78
Authoritative sources	78

Unresolved questions	78
Chapter 22 - The Control Plane	80
Why this chapter exists	80
7 - no database, no broker, no cloud dependency extends to "no network service surface" as a	80
What problem it solves	80
What you need to already know	80
The idea	80
A declarative catalogue, not an execution engine	80
The localhost dashboard	80
Supporting modules	80
Classification	81
Authoritative sources	81
Unresolved questions	81
Chapter 23 - Multi-LLM Coordination	82
Why this chapter exists	82
What problem it solves	82
What you need to already know	82
The idea	82
The pipeline, and who's never allowed to skip it	82
Code authority is exclusive; advice is shared	82
GATHER / ENHANCE / IMPLEMENT	82
The infrastructure underneath	83
Classification	83
Authoritative sources	83
Unresolved questions	83
Chapter 24 - Repository Encyclopedia	85
Why this chapter exists	85
What problem it solves	85
What you need to already know	85
The idea	85
Current book vs encyclopedia	85
1. Not covered by the book (~403 files)	86
2. They naturally split into four groups	86
A. Core but omitted (highest priority)	86
B. Sidecar / support modules	87
C. Dormant / archived	87
D. Operational scripts (largest numeric gap)	88
3. Why only ~35 files named?	88
4. Target: true Repository Encyclopedia	89
Recommended build order (priority)	89
Entry format (proposed template for future encyclopedia rows)	90
5. Relationship to existing record systems	90
Classification	91
Authoritative sources	91
Unresolved questions	91
Repository Encyclopedia	92
Status by phase	92
Machine twin (JSONL)	92
How to read an entry	93
Inventories used	93
Encyclopedia E1 - Spine Implementation (Group A)	94
Scope	94
1. src/core/ - Decision spine orchestration	95
Already in the architecture book (CITED)	95
Encyclopedia entries - omitted / supporting	95
src/core/__init__.py	95
src/core/types.py	95

src/core/feature_store.py	95
src/core/gate_intelligence.py	95
src/core/model_registry.py	96
src/core/collector.py	96
src/core/acceptance_controller.py	96
src/core/convergence_controller.py	96
src/core/dynamic_threshold.py	96
src/core/hierarchical_meta_fusion.py	96
src/core/regime_governor.py	96
src/core/signal_audit.py	97
src/core/signal_belief_tracker.py	97
src/core/governance_mode.py	97
src/core/backtest_port.py	97
src/core/ultron_risk_gate_wrapper.py	97
2. src/config_layer/ - Config, CRT, planner, models, LLM transport	97
Already in the architecture book (CITED)	97
Encyclopedia entries	98
src/config_layer/production_config.py	98
src/config_layer/config_builder.py	98
src/config_layer/market_router.py	98
src/config_layer/production_bundle.py	98
src/config_layer/stack_version.py	98
src/config_layer/model_paths.py	99
src/config_layer/model_resolver.py	99
src/config_layer/state_topology.py	99
src/config_layer/state_contract.py / state_contract_loader.py	99
src/config_layer/crt_config_provenance.py	99
src/config_layer/crt_sweep_taxonomy.py	99
src/config_layer/llm_inference_client.py	99
src/config_layer/llm_scorer.py / llm_narrative.py / insight_reporter.py	100
src/config_layer/rr/rr_dataset_builder.py	100
src/config_layer/rr/rr_pattern_miner.py	100
src/config_layer/rr/rr_fusion.py	100
src/config_layer/rr/__init__.py	100
src/config_layer/__init__.py	100
3. src/engines/ - Scoring engines + gates	100
Already CITED	101
Encyclopedia entries - support / optional	101
src/engines/__init__.py	101
src/engines/gaussian_engine.py	101
src/engines/live_engine.py	101
src/engines/zone_cluster_score.py	101
src/engines/trap_validator_engine.py	101
src/engines/llm_engine.py	101
src/engines/tradenet_meta_engine.py	101
4. src/runtime/ - Backtest, live hook, baselines, probes	102
Already CITED	102
Encyclopedia entries	102
src/runtime/__init__.py	102
src/runtime/baseline_capture.py	102
src/runtime/unified_replay_harness.py	102
src/runtime/backtest_bitnet.py	102
src/runtime/analyze_fusion_shadow.py	102
src/runtime/crt_baseline_trace.py	103
src/runtime/crt_fail_reason_counters.py	103
src/runtime/exit_model_band.py	103
5. src/execution/ - Multi-signal execution loop (partial)	103

Already CITED	103
Encyclopedia entries	103
src/execution/__init__.py	103
src/execution/alert_manager.py	103
src/execution/override_handler.py	103
src/execution/execution_intent_v1_0.py	104
6. src/live/ - Live broker / alert bridges	104
src/live/__init__.py	104
src/live/mt5_bridge.py	104
src/live/telegram_bridge.py	104
7. src/inout/ - Market data I/O (not the archived strategy rail)	104
src/inout/mt5_candle_fetcher.py	104
src/inout/alphavantage_candle_fetcher.py	104
src/inout/hummingbot_candle_fetcher.py	105
src/inout/perp_funding_fetcher.py	105
8. src/control_plane/ - Localhost command catalog + dashboard	105
Already CITED	105
Encyclopedia entries	105
src/control_plane/__init__.py	105
src/control_plane/cp_types.py	105
src/control_plane/jobs.py	105
src/control_plane/dashboard_api.py	105
src/control_plane/monitors.py	105
src/control_plane/report_api.py	106
src/control_plane/context_report.py	106
src/control_plane/code_context_extractor.py	106
src/control_plane/dot_graph_context.py	106
E1 coverage rollup	106
Next	107
Encyclopedia E1b - Feature Pipeline & Registry (deep map)	108
Scope	108
Layer map (semantic pipeline)	108
1. Math authority (immutable primitives + registry)	109
src/features/candle_math.py	109
src/features/derived_math.py	109
src/features/registry/primitive_registry.py	109
src/features/registry/composition_registry.py	110
src/features/registry/derived_registry.py	110
src/features/registry/_loader.py	110
src/features/registry/__init__.py	110
src/features/formula_registry.py	110
src/features/fm_resolve.py	110
src/features/feature_identity.py	110
2. Production vector emission & schema	110
src/features/feature_pipeline.py	110
src/features/feature_schema.py	111
src/features/schema_validator.py	111
src/features/gaussian_schema_contract.py	111
3. Online / CRT builders	111
src/features/feature_builder.py	111
src/features/crt_feature_builder.py	111
src/features/causal_structure.py	111
4. Session & broker clock	111
src/features/session_classifier.py	111
src/features/broker_clock.py	112
5. Semantic stack (states -> context -> shape -> CRT resolve -> model evidence)	112
src/features/feature_states.py	112

src/features/market_context.py	112
src/features/market_shape.py	112
src/features/crt_state_resolver.py	112
src/features/model_evidence.py	112
6. Dataset, validation, drift	112
src/features/dataset_builder.py	112
src/features/dataset_validator.py	112
src/features/feature_monitor.py	113
src/features/___init___py	113
Config keys (feature surface)	113
Tests & lint (pointers)	113
Danger flags	113
E1b exit criterion	114
Encyclopedia E2 - Research Utilities Index	115
Scope	115
How to use this index	115
1. Program -> code -> driver -> artifact (primary map)	115
2. Shared research spine (load-bearing root modules)	116
3. src/research/ package encyclopedia	118
3.1 adapters/ (4)	118
3.2 controls/ (3)	118
3.3 hypotheses/ (8) - Stage-2 economic consumers	118
3.4 candle_state/ (7) - multi-TF state & Program 9 Stage-1	118
3.5 measurement/ (4) - honest outcomes	119
3.6 clean_labels/ (3)	119
3.7 weekly_sweep/ (2)	119
3.8 zone_mapping/ (13)	119
3.9 episodes/ (13+)	120
3.10 ic002_entry_evolution/ (5)	120
3.11 ic003_shapes/ (3)	120
3.12 ic003b_sequence_geometry/ (8)	120
3.13 secondlow_v1/ (5)	120
3.14 path/ (2)	120
3.15 envelope_offline/ (3)	120
3.16 model_runners/ (23)	120
3.17 synthetic/ (10)	120
3.18 Root-only modules	120
4. scripts/research/ - driver clusters (126 files)	120
4.1 Qualify / M4 gate (16) - primary economic runners	121
4.2 Phase / Program 1 sub-drivers (4)	121
4.3 Regime / transition information (1+)	121
4.4 Program 9 M5 (2+)	121
4.5 Zone / ZoneGate research (14)	121
4.6 RR / clean labels (11)	121
4.7 BitNet / Gaussian / TradeNet diagnostics (8)	121
4.8 CRT parity / resolver (7)	121
4.9 IC / shapes (6)	121
4.10 Episodes / corpus / path (13)	121
4.11 XAU / metals / FX cost (8)	122
4.12 MSIP hypothesis runners (2)	122
4.13 Gate measurement (2)	122
4.14 Private / underscore helpers (4+)	122
4.15 Uncategorized residual (~28)	122
5. Configs, pre-regs, artifacts (not code but E2 surface)	122
6. Danger / authority flags	122
E2 exit criterion	123
Next	123

Encyclopedia E3 - Governance Tooling	124
Scope	124
How to use this index	124
1. Job map (primary navigation)	125
2. src/governance/ - library encyclopedia (19 files)	126
Promotion & validation (LIVE ops)	126
promotion_manager.py	126
shadow_promotion_gate.py	126
config_integrity.py	126
portfolio_validation.py	126
multi_strategy_validator.py / strategy_backtest.py	126
orchestrator.py	126
Registries & generated truth (LIVE meta)	127
findings_export.py	127
hypothesis_registry.py	127
framework_registry.py	127
script_registry.py / script_census.py / script_seed.py	127
module_census.py / module_attribution.py	127
Specialist / sidecar	127
3. scripts/governance/ - CLI encyclopedia (31 files)	127
3.1 Construction & promotion	127
3.2 Registry seeds & queries	128
3.3 Feature DAG / FM governance	128
3.4 Authority censuses & boundary scans	128
3.5 Decision packages / MSIP / RR freeze / zone remap	129
4. scripts/analysis/ - census & probe clusters (115 files)	129
5. scripts/maintenance/ - adjacent hygiene (9 files)	130
6. Evidence ledger home (docs, not scripts)	130
7. Danger / authority flags	130
E3 exit criterion	131
Next	131
Encyclopedia E4 - Sidecar / Support Modules (Group B)	132
Scope	132
How to use	133
1. src/bitnet/ - BitNet / zone intelligence stack (23)	134
2. src/utis/ - shared helpers (17)	135
3. src/training/ - offline training (8)	135
4. src/portfolio/ - multi-instrument allocation (6)	136
5. src/replay/ - institutional replay memory (6)	136
6. src/analytics/ - offline analytics (5)	136
7. src/expansion/ - controlled frequency scaling (6)	137
8. src/regime/ - regime surfaces (4)	137
9. src/msip/ - Market State Interpretation Platform shadow (6)	137
10. src/multi_llm/ - multi-LLM ops package (5)	138
11. src/retrieval/ - codebase RAG (9)	138
12. Small surfaces	138
src/events/ (2)	138
src/search/ (1)	139
src/monitoring/ (2)	139
Cross-cutting findings (why these stay sidecar)	139
Danger flags	139
E4 exit criterion	140
Next	140
Encyclopedia E5 - Dormant / Archived Surfaces (Group C)	141
Scope	141
How to use	142
1. src/strategies/ - S01-S10 multi-strategy scaffold (18)	142

2. src/scanner/ - multi-symbol scan rail (6)	143
3. src/journal/ - trade identity / provenance (6)	143
4. src/cognitive/ - cognitive bus (2)	143
5. src/feedback/ - AI feedback (2)	143
6. src/llm_research/ - offline LLM research pipeline (5)	144
7. src/data_ingestion/ - MIXED (6)	144
8. src/uat/ - multi-strategy UAT harness (4)	144
9. src/ui/ - remnant (1)	145
10. archive/** - ARCHIVED (do not import)	145
Reopen checklist (any Group C package)	145
Danger flags	146
E5 exit criterion	146
Next	146
Encyclopedia E6 - Remaining Operator Scripts (Group D residual)	147
Scope	147
How to use	148
1. scripts/data/ - market data I/O (22)	148
Fetch (thin wrappers over src/inout)	148
Build / convert / verify	148
2. scripts/training/ - model training CLIs (13)	149
3. Root scripts/*.py (10)	149
4. scripts/context/ - Portable Mind / multi-LLM (5)	150
5. scripts/control_plane/ (1)	150
6. scripts/backtest/ (2)	150
7. scripts/export/ (3)	150
8. scripts/evaluation/ (3)	151
9. scripts/groq_bridge/ (3)	151
10. scripts/misc/ (5)	151
11. Single-file operator CLIs	151
Cross-links (do not duplicate)	152
Danger flags (E6)	152
E6 exit criterion	152
Encyclopedia series status	152
Appendix A1 - Testing the System	154
Why this chapter exists	154
What you need to already know	154
The idea	154
Running it	154
Layout	154
Representative patterns	154
A doc-drift this appendix surfaces	154
Classification	154
Authoritative sources	155
Unresolved questions	155
Appendix A2 - Unresolved Questions (Rollup)	156
Why this chapter exists	156
What you need to already know	156
Resolved in the book (Grok review pass - no code changes)	156
Still open - book-scope gaps	156
Still open - real repository issues (code/doc; not fixed by this book pass)	157
Verification gaps (assumed from docs; not re-proven this pass)	157
Out of scope for this pass (explicit)	157

The Tradelatest Book - Table of Contents & Guide

The Tradelatest Book

What this is. A cover-to-cover reconstruction of this repository as a sequence of ideas, not a pile of files. Read it in order and you should understand *why* Tradelatest exists, *how* a candle becomes (or doesn't become) an order, and *where* every governance, research, and agent layer sits around that spine - without jumping between 900 documents to assemble the picture yourself.

What this is not. A replacement for the code, for `CLAUDE.md`, or for the existing reference docs (`docs/reference/`), memory docs (`docs/memory/`), or topic docs (`docs/topics/`). This book is a *map*. Every chapter ends with the file paths that are the real, authoritative source - when the book and the code disagree, **the code wins** (per `CLAUDE.md` §0's hierarchy, which this book inherits). See [Chapter 3](#) for exactly how the book relates to those other record systems.

Living document. New chapters are added and existing ones are deepened over time; chapter numbers never change once assigned, so links stay stable. See "How this book evolves" below.

How to read this

- **Just need the mechanics?** Start at [Chapter 00 - Quick Start](#) (one candle's journey, load-bearing surfaces, known defects) - then jump to deeper chapters as needed.
- **New to the repo (full narrative)?** Start at Chapter 1 and read straight through Part V (chapters 01-15). That is the complete, coherent story of one candle's journey from raw OHLCV to an approved (or rejected) order - the spine everything else in the repo hangs off.
- **Already know the spine, need one layer?** Jump straight to the Part that covers it (table below) - each chapter is self-contained given the "What you need to already know" prerequisites it lists.
- **Looking for a specific concept** (CRT, Fusion, Ultron, Promotion, ...)? Use the Concept Index at the bottom of this file - it points to the *one* chapter that owns that concept's explanation, per the book's Canonical Concept Rule.

Grok review pass (2026-08-07)

A structured review of the first PDF export drove narrative fixes **in this book only** (no trading system code changes): Quick Start; INOUT resolution; Program 9 coverage; ATR/F-057 proposed remediations; Measurement Contract path; MSIP brief; un-chaptered package map; A2 rollup update. Reading PDFs with `Grok` in the filename under `grok/` are regenerated from this tree.

Table of Contents

Part I - Foundations

Why the system exists, what must never break, and how to navigate everything else.

#	Chapter	Status
00	Quick Start - One Candle's Journey	Written (Grok review)
01	Why Tradelatest Exists	Written

02	The Invariants and the Happy Flow	Written
03	How This Book Fits the Repository's Knowledge	Written
04	Architecture at a Glance	Written

Part II - Market Understanding

Turning raw candles into a trustworthy, meaningful numeric surface.

#	Chapter	Status
05	Data Ingestion and the No-Lookahead Discipline	Written
06	The Market Ontology - Canonical Semantic Authority	Written
07	The Feature Pipeline and the Canonical Vector	Written

Part III - Market Semantics

Giving the numeric surface structure: states, transitions, and patterns.

#	Chapter	Status
08	The CRT State Machine - the Spine	Written
09	Interpreters and the Pattern Contract	Written

Part IV - Decision Making

Turning structure into a single execute/reject decision.

#	Chapter	Status
10	The Four Scoring Engines	Written
11	Fusion - Combining Independent Signals	Written
12	The Decision Engine - Semantic Approval Only	Written

Part V - Execution

Turning a decision into order geometry, and geometry into a capital-safe order.

#	Chapter	Status
13	The Execution Planner - From Decision to Order Geometry	Written
14	Ultron Risk Gate - the Final Capital Check	Written

15	Live Execution and INOUT	Written
----	--	---------

Part VI - Governance

How a config earns the right to run in production, and how the repo keeps its own truth honest.

#	Chapter	Status
16	Config-First Doctrine and the Promotion Path	Written
17	Truth Maintenance - Findings, Closure, and Authority	Written
18	A Field Guide to docs/governance/	Orientation-level

Part VII - Research

How the repo tries - and mostly fails, on purpose and by design - to find a tradeable edge.

Numeric program results live in Chapter 19's data table (not only narrative). Platform paths and artifact roots are in Chapter 20. PDF readers: use the bookmarks for "Chapter 19" / "Chapter 20", or open [grok/Tradelatest-Canonical-Knowledge-Book-Research-Grok.pdf](#).

#	Chapter	Status
19	The Research Programs - Falsification as a Discipline	Written (results table)
20	The Research Platform - src/research/	Written (data map)

Part VIII - Agent Intelligence

The layers that let an LLM (or a human) operate the system through natural language.

#	Chapter	Status
21	The AI Automation Agent	Orientation-level
22	The Control Plane	Orientation-level
23	Multi-LLM Coordination	Orientation-level

Part IX - Toward Full Repository Understanding

From architecture guide toward file-level coverage of the remaining surfaces.

#	Chapter	Status
24	Repository Encyclopedia	Written (charter + remaining-file map)
E1	Encyclopedia E1 - Spine Implementation	DONE (Group A file map)
E1b	Encyclopedia E1b - Features & Registry	DONE (28 feature modules)
JSONL	encyclopedia_rows.jsonl	GENERATED (~813 rows)

E2	Encyclopedia E2 - Research Utilities	DONE (program/driver/package index)
E3	Encyclopedia E3 - Governance Tooling	DONE (promote/registry/DAG/analysis index)
E4	Encyclopedia E4 - Sidecar Modules	DONE (Group B support packages)
E5	Encyclopedia E5 - Dormant / Archived	DONE (Group C + archive)
E6	Encyclopedia E6 - Remaining Scripts	DONE (operator residual ~70)

Appendix

#	Chapter	Status
A1	Testing the System	Written
A2	Unresolved Questions (rollup)	Written (Grok review update)

Concept Index (Canonical Concept Rule)

Each concept below has exactly one chapter that owns its explanation. Other chapters that touch it link back here instead of re-explaining it.

Concept	Canonical chapter
Quick Start / spine sketch	Ch.00
Market Ontology	Ch.06
Feature Pipeline / CANONICAL_FEATURES	Ch.07
CRT (state machine)	Ch.08
Gaussian / Zone Gate / RR engines	Ch.10
Fusion	Ch.11
Decision Engine	Ch.12
Execution Planner (ExecutionPlannerV1_2)	Ch.13
Ultron Risk Gate	Ch.14
Promotion / PromotionManager	Ch.16
Config Validation / ConfigValidator	Ch.16
Findings / Closure / Authority Ladder	Ch.17
Control Plane / CommandSpec	Ch.22
AI Automation Agent / PLAN_REGISTRY	Ch.21
Multi-LLM Protocol	Ch.23
Program 9 (M5 non-directional)	Ch.19

MSIP (shadow market state)	Ch.18
Repository Encyclopedia (remaining files A-D)	Ch.24

How this book evolves

Per the book's own charter: when the repository changes, **only the affected chapter is updated** - chapter numbers and links are permanent. If a new concept needs a canonical home, it gets a new chapter number appended at the end of its Part's range (never a renumber). Governance/Research/Agent Intelligence chapters (16-23) are marked "Orientation-level" - they are accurate but deliberately not exhaustive over their much larger source corpora (~250 files in `docs/governance/`, ~475 files across `src/research/` + `docs/research-readiness/`); deepening them is future book work, tracked in [A2](#).

Source of truth: this book is generated understanding, not authority. On any conflict between a chapter and the code/config it describes, the code/config wins - see `CLAUDE.md` §0 and §4.0 for the repo's own precedence rules, which this book does not override.

Chapter 00 - Quick Start: One Candle's Journey

Part I - Foundations

Status of this chapter: Written (Grok review pass, 2026-08-07)

Why this chapter exists

The rest of this book is deliberately careful about *how* knowledge is classified, which record system owns what, and what remains unfixed. That meta-layer is load-bearing for contributors who will edit the repo - but it can overwhelm a reader who only needs to understand **what the trading system does**. This chapter short-circuits the meta-layers and gives the core mechanics in one pass.

What problem it solves

"I just need to understand the spine" without reading Chapters 3, 17, and 18 first.

What you need to already know

Nothing. Jump back to [Chapter 1](#) when you want the *why*, or [Chapter 2](#) for the full invariant set.

The idea

The system in one sentence

Tradelatest takes **M15 OHLCV candles**, scores each bar through **four independent engines** (CRT, Gaussian, Zone Gate, RR), **fuses** those scores, **decides** accept/reject on semantic grounds, plans **entry geometry**, and finally allows or blocks the order through a **capital risk gate**. Everything is file-backed: no database, no message broker, no cloud dependency.

The happy path (one candle)

Step	What happens	Where it lives
1	Load candle (CSV / MT5 / live feed)	<code>src/inout/*_candle_fetcher.py</code> , loaders in runtime
2	Build a 39-dim feature vector (no lookahead)	<code>src/features/feature_pipeline.py</code>
3	CRT state machine advances (RANGE->...->EXECUTION)	<code>src/config_layer/crt_engine_v2.py</code>
4	Four engines score the bar independently	<code>src/engines/*</code> via <code>EngineRunner</code>
5	Fusion combines scores under a completeness gate	<code>src/core/fusion_engine.py</code>
6	Decision Engine: semantic approval only	<code>src/core/decision_engine.py</code>
7	Execution Planner: intent + entry; CRT owns SL/TP levels	<code>execution_planner.py</code> + <code>compute crt levels</code>

8	Ultron Risk Gate: final capital / RR / exposure check	src/core/ultron_risk_gate.py
9	Live path: HookedLiveEngine.process on each tick	src/runtime/live_engine_hook.py

Backtests stream candle-by-candle through the same spine (src/runtime/backtest_v2.py). Live always runs the fusion gate; research/backtest gate mode is config-driven (backtest.engine_gate_enabled).

Priority order (not "profit first")

The system's declared priorities are, in order: **replay correctness -> explainability -> telemetry -> AI as advisory only -> profit**. Profit is a consequence of a trustworthy process, not a license to skip gates. See [Chapter 1](#) for the full argument.

What is actually load-bearing today

Surface	Status (honest)
CRT state machine (mechanism)	CLOSED for OHLCV -> TRADE_OPENED boundary
Feature code surface (39-dim)	CLOSED for formula/PIT/write-site hygiene - not economic proof
Gaussian engine	AUDITED - live score collapses near-constant ~0.8825 (F-060)
Zone Gate	AUDITED - geometric HARD gate; non-pivotal on fusion decisions (F-036)
RR / rr_fusion	AUDITED - base RR flows; rr_fusion disabled (F-038/F-044)
BitNet / TradeNet	Built; inert or unwired on the active production config
Research Programs 1-8	CLOSED nulls under a fixed expectancy bar - discipline working
Active production config	configs/production/ACTIVE_VERSION -> currently v2_multi_2026_04 on this branch

How config becomes production

1. Tunables live in configs/production/*.json - no magic numbers in hot-path Python.
2. ConfigValidator.validate() runs hard + soft quality gates -> ValidationReport.
3. Only decision == "APPROVE" may promote; SHA-256 hash + append-only promotion_log.jsonl.
4. Rollback = restore archived config + update ACTIVE_VERSION.

Details: [Chapter 16](#).

How to read the rest of the book

If you want...	Read
Full why + invariants	Ch.01 -> Ch.02
Ontology, features, CRT depth	Ch.06 -> Ch.08

Engines, fusion, decision honesty	Ch.10 -> Ch.12
Geometry + risk + live path	Ch.13 -> Ch.15
Governance / findings / closure	Ch.16 -> Ch.18
Research programs + numeric results table	Ch.19 -> Ch.20 (Part VII; also dedicated PDF *Research-Grok.pdf)
In-repo agent + multi-LLM ops	Ch.21 -> Ch.23
Open defects & book gaps	A2

Rule on conflict: this book is a map. Code and active config win.

Known defects you should not ignore (even on a quick read)

1. **ATR unit mismatch** in `compute_crt_levels` - SL buffer may treat relative ATR as price units ([Ch.13](#) has proposed remediations; unfixed pending owner decision).
2. **CRTConfig split-brain (F-057)** - programmatic `BacktestRunner(cfg)` without `crt_config` can miss production JSON overrides.
3. **Doc-drift elsewhere in the repo** - e.g. some reference docs still say 38-dim; live code is 39-dim. This book tracks code.

Classification

Concept	Status
Candle -> order spine	Production
This Quick Start chapter	Orientation aid - not an authority surface

Authoritative sources

- `docs/architecture/signal-flow.md` - the 7-step walk with failure modes.
- `docs/architecture/goal.md` - purpose, invariants, priority order.
- `configs/production/ACTIVE_VERSION` - Tier-0 runtime truth.
- `src/core/engine_runner.py` - orchestrator every decision path passes through.

Unresolved questions

None for this chapter - open items live in [A2](#).

Previous: [Table of Contents](#) * **Next:** [Chapter 01 - Why Tradelatest Exists](#)

Related: [Chapter 02 - The Invariants and the Happy Flow](#)

Memory: `docs/memory/architecture-memory.md` before editing the spine.

Chapter 01 - Why Tradelatest Exists

Part I - Foundations

Status of this chapter: Written

Why this chapter exists

Before any code, config, or governance rule makes sense, one question needs an answer: what is this system actually *for*? Get that wrong and every later chapter reads as arbitrary engineering detail. Get it right and the rest of the book - four independent scoring engines, a fusion gate, a risk gate, a governance apparatus, a research program that mostly produces null results on purpose - snaps into place as one coherent design.

What problem it solves

Tradelatest reads M15 (15-minute) OHLCV candles for a handful of instruments, and for each candle asks: is this a valid, risk-bounded trading opportunity? If yes, it plans the trade's entry, stop, and target, and passes it through a final capital-safety check before anything is (in principle) executed. That is the entire functional surface of the system, stated in one sentence - but *how* it answers that question, and what it refuses to compromise on while answering it, is the real subject of this book.

What you need to already know

Nothing yet - this is the first chapter.

The idea

The one-paragraph goal

Tradelatest scores every M15 candle through four independent engines (CRT, Gaussian, Zone Gate, RR), fuses their scores under a weighted-completeness gate, plans entry/SL/TP via an execution planner, and approves the final position through a risk gate. Governance sits on top of all of this: no configuration reaches production without passing a validator and leaving an append-only audit trail. Everything is file-backed - there is no database, no message broker, no cloud dependency. This same one-paragraph summary appears in `CLAUDE.md §1` because it is the fixed point the rest of the repository orbits.

Profit is not the primary objective - read that literally

The single most important, least obvious fact about this repository: **profit is explicitly not the top-priority goal**. The declared priority order, from `docs/architecture/goal.md`, is:

1. **Replay correctness** - the same inputs must always produce the same outputs.
2. **Explainability** - a human (or an LLM) must be able to say *why* a decision happened.
3. **Telemetry continuity** - historical data must remain interpretable as the system evolves.
4. **Advisory-AI** - language models assist and explain; they never trigger trades unsupervised.
5. Only then: economic performance, and even that comes with a caveat - *structure validity is not the same thing as execution validity*. A candle can be a textbook-valid CRT structure and still be a bad trade once realistic costs and slippage are applied.

This ordering explains a huge amount of what looks unusual elsewhere in the codebase: a research program (Part VII) that closes hypothesis after hypothesis as *null* and treats that as a successful outcome; a governance layer (Part VI) that will block a config from production over an

un-auditable change even if backtests look good; and a decision spine that fails closed rather than guessing whenever something is ambiguous. The system is optimized to be **trustworthy first**, profitable second - because a system you cannot trust cannot be safely made profitable, but a system that lies convincingly about being profitable is actively dangerous.

The economic target, when it does apply

There is still a real, machine-readable economic objective - it's just deliberately subordinate and currently *advisory only*. It lives as the "Goal Layer" (id G001) inside the production config and states concrete targets: 20-40 trades/month (max 80), average reward:risk >= 2.0, win rate >= 0.35, max drawdown <= 10%, risk per trade 0.5%, expectancy >= 0.20R, M15 execution informed by H1 structure, reaction-only entries with human execution in the loop. Every backtest computes a `goal_report` against these targets, but nothing currently *enforces* them - there is a `goal.enforce` flag in the schema, and it stays off by design, because the system's honestly-measured current performance (see the Research Programs in Part VII) does not yet clear these targets. Turning that flag on prematurely would be exactly the kind of dishonesty the priority order above exists to prevent.

Pattern interpreters are measured, never asserted

One more design choice worth internalizing early: the system supports "interpreters" - modules that claim to recognize classical chart patterns (Point & Figure, Wyckoff, Market Profile, Order Flow). None of these are treated as intelligent by assumption. Each must satisfy an explicit Interpreter Contract and is *measured* against the Goal Layer via a forward-walk + qualification gate before it earns any influence - see [Chapter 9](#). This is the same epistemic discipline as the priority order above, applied at the level of an individual idea rather than the whole system.

Classification

Concept	Status
The one-paragraph goal (4 engines -> fusion -> decision -> execution -> risk gate)	Production
Goal Layer / G001 economic targets	Production artifact, advisory-only (measure, don't gate)
<code>goal.enforce</code> hard-gating	Built, dormant by design
Interpreter Contract-qualified pattern interpreters	Research

Authoritative sources

- `docs/architecture/goal.md` §1, §1.1 - the north-star statement and the Goal Layer targets, in full.
- `src/config_layer/goal_schema.py`, `src/config_layer/goal_validator.py` - the Goal Layer's code.
- `docs/topics/goal-layer.md` - the concept-level topic doc (code ↔ tests ↔ entry points).
- `CLAUDE.md` §1 - the same one-paragraph summary, kept in sync by convention.

Unresolved questions

None for this chapter - `goal.md` is a well-evidenced, internally consistent primary source.

Previous: [Chapter 00 - Quick Start](#) * **Next:** [Chapter 02 - The Invariants and the Happy Flow](#)
Related: [Chapter 19 - The Research Programs](#) (what "measured, never asserted" looks like in practice) * [Chapter 9 - Interpreters and the Pattern Contract](#)

Memory: none - this chapter summarizes a single primary doc rather than reverse-engineered code.

Chapter 02 - The Invariants and the Happy Flow

Part I - Foundations

Status of this chapter: Written

Why this chapter exists

Chapter 1 established *why* the system exists and what it refuses to compromise on in principle. This chapter makes that concrete: the exact shape of a candle's journey through the system (the "happy flow"), and the seven hard rules ("invariants") that every later chapter's design decisions trace back to. Every subsequent Part of this book is, in effect, a zoomed-in view of one segment of the diagram below.

What problem it solves

Without a fixed reference diagram, it's easy to lose track of what's on the synchronous decision path versus what merely *feeds* it asynchronously. This chapter draws that line once, clearly, so later chapters don't have to re-justify it.

What you need to already know

[Chapter 1](#) - the priority order (replay correctness first, profit last) is the reason these particular seven invariants were chosen.

The idea

The happy flow

One M15 candle closes. From there:

```
Candle -> [4 engines score independently] -> Fusion (all-four-or-reject) -> Decision (threshold ->
ACCEPT/REJECT) -> Execution Planner (entry/SL/TP/RR/TTL) -> Risk Gate (Ultron) -> order out, or a
logged rejection at any stage
```

In code, this is the **decision spine**:

```
EngineRunner -> FusionEngine.evaluate -> DecisionEngine -> ExecutionPlannerV1_2 ->
UltronRiskGate.evaluate
```

Three of the four engines score the candle as a single number each. The fourth, CRT, is different in kind: internally it's not a number generator, it's a **state machine** with nine legal states, walking a lifecycle:

```
RANGE -> SWEEP -> DISPLACEMENT -> EXPANSION -> RETEST -> EXECUTION -> RESOLUTION
```

plus two branches: RANGE ↔ SHADOW_PENDING -> SWEEP (an early-detection branch) and EXPANSION -> EXPIRED -> RANGE (a TTL-based soft-archive branch when a setup goes stale before retesting). CRT's *score* is what feeds Fusion; CRT's *state* is what makes it "the spine" - Part III is dedicated to it.

The seven invariants

These are the rules that must never break, regardless of how much throughput or profit a change promises ([docs/architecture/goal.md §3](#)):

1. **Deterministic replay.** Same inputs -> same outputs, always. This is invariant #1 for a reason: nothing else in the book - governance, research, agent advice - is trustworthy if replay isn't.
2. **All four engines or nothing.** `EXPECTED_ENGINES = {"crt", "gaussian", "zone_gate", "rr"}` in `EngineRunner`. If any engine is missing, the candle is rejected outright - there is no silent partial fusion.
3. **The LLM is advice, never a trigger.** Any LLM-backed gate returns a neutral score after repeated failures rather than blocking or hallucinating a decision. A code path that cannot tolerate that neutral fallback is considered broken by design.
4. **No config reaches production without governance.** Every promotion needs an approved `ValidationReport`, a SHA-256 config hash, and an append-only `promotion_log.jsonl` entry - see [Chapter 16](#).
5. **Telemetry is additive only.** Fields get renamed forward, never silently removed - old JSONL records must stay interpretable.
6. **CRT state moves only along legal transitions.** The nine-state graph above is the *entire* legal transition set - there is no free-form state graph.
7. **No database, no broker, no cloud dependency.** Everything is file-backed. A request to "add a table" or "use the ORM" is a convention break, not a valid design option.

Every change is additionally scored against five governance questions: is it still deterministic? Still comparable across runs? Auditable later? Can an LLM reason about it? Is execution authority still isolated? A change that fails any of these needs explicit sign-off, not silent acceptance.

The deviation policy

Not every deviation from "the ideal" is forbidden - the system distinguishes three tiers:

- ■ **Acceptable** - improves throughput, speed, or quality while keeping all seven invariants intact.
- ■■ **Needs a flag** - trades off an invariant for throughput (must be explicit, reversible, and visible).
- ■■ **Not acceptable** - breaks determinism, governance, the LLM-as-advice rule, telemetry continuity, or partial-fusion protection, *regardless* of how much throughput or speed it buys.

The async "kitchen feeders"

Four subsystems sit beside the synchronous spine, feeding it without ever calling into it directly - they communicate only through files, never direct function calls:

- **Governance** - `AutoTuner` -> `ConfigValidator` -> `PromotionManager` writes new production configs that the spine reads at its next load. See Part VI.
- **Training** - raw trade JSONL -> `DatasetValidator` -> `Trainer` -> `ModelRegistry` writes model artifacts that engines (mainly Gaussian and BitNet's zone gate) read. See [Chapter 10](#).
- **AI Agent** - either drives the spine through a backtest loop (Pipeline mode) or taps it read-only at the Fusion/Decision step (Copilot mode) - it can never mutate a decision in flight. See [Chapter 21](#).
- **INOUT** - a parallel live-execution strategy rail that joins the spine only at the very last step, `UltronRiskGate.evaluate()` - and is currently archived (`archive/inout_legacy/`). See [Chapter 15](#).

The full seven-step walk (data ingest -> feature pipeline -> scoring engines -> fusion & decision -> execution planner -> risk gate -> execution), with per-step module/entry-point/failure-mode detail and a Mermaid swim-lane diagram, lives in [docs/architecture/signal-flow.md](#) - this chapter's diagram is the compressed version; that file is the canonical one.

Classification

Concept	Status
The 7-step synchronous spine	Production
The 9-state CRT lifecycle	Production, CLOSED (see Ch.08)
The 7 invariants + 5 governance questions	Doctrine - enforced by review discipline, not (all) by code
Governance / Training / Agent / INOUT feeders	Production (Governance, Training, Agent) / Archived (INOUT)

Authoritative sources

- `docs/architecture/goal.md` §2-§4 - the happy flow, invariants, and deviation policy verbatim.
- `docs/architecture/signal-flow.md` - the authoritative 7-step walk with module/entry/failure-mode detail per step and the Mermaid swim-lane diagram (§4).
- `src/core/engine_runner.py` - `EXPECTED_ENGINES` and the completeness gate.
- `src/config_layer/crt_engine_v2.py`, `docs/architecture/event-taxonomy.md` §3 - the CRT transition graph, also re-exported at `state_topology.py:109`.

Unresolved questions

None - this chapter compresses two internally consistent, well-cross-referenced primary sources.

Previous: [Chapter 01 - Why Tradelatest Exists](#) * **Next:** [Chapter 03 - How This Book Fits the Repository's Knowledge](#)

Related: [Chapter 08 - The CRT State Machine](#) * [Chapter 16 - Config-First Doctrine and the Promotion Path](#)

Memory: `docs/memory/architecture-memory.md` (deep companion, load only when a full cross-package map is needed).

Chapter 03 - How This Book Fits the Repository's Knowledge

Part I - Foundations

Status of this chapter: Written

Why this chapter exists

This repository already has more documentation than most codebases have code: ~900 files under docs/, a 300+-file governance ledger, a memory hierarchy, a topics index, findings registries. A reader could reasonably ask "why does this need a *book* on top of all that?" This chapter answers that directly, and - because the book borrows vocabulary from those existing systems - defines the classification labels ("Production", "Experimental", "Research", ...) used in every chapter from here on, mapped honestly onto the repo's own status vocabulary rather than invented fresh.

What problem it solves

Without this chapter, a reader bounces between CLAUDE.md, docs/knowledge-map.md, docs/topics/readme.md, and docs/memory/README.md to figure out which of several overlapping "index" documents to trust for what. This chapter is that decision, made once.

What you need to already know

[Chapter 1](#) and [Chapter 2](#) - the substance this book organizes.

The idea

The repository already has record systems - this book is a new one, not a replacement

docs/knowledge-map.md names nine existing "record systems," each authoritative for a different question: the SESSION LOG (assistant_project.md, "what actually happened, dated"), Plans (docs/plans/, "what was designed"), Analysis (docs/analysis/, "point-in-time studies - evidence, not current truth"), Findings (docs/current-findings.md, "current validated conclusions"), Research families, Codebase structure maps, Topics, a Timeline (the date-join across the others), and Research substrates (frozen per-instrument datasets). None of these are *sequential* - each answers one kind of question well, but none walks a reader start-to-finish through the ideas.

This book is a tenth: the sequencing layer. It doesn't replace any of the above - it points into them, in order, the way a textbook points into a language's reference manual without reproducing it. Concretely: this book cites the SESSION LOG and findings for *what happened and what's proven*, docs/topics/ for *the grounded per-concept picture* (code ↔ tests ↔ entry points), and docs/memory/ for *the reading order into source* when you're about to edit a subsystem. If you're about to change code, docs/memory/ is still the more direct next stop - see below.

The memory hierarchy still governs when you're about to edit code

CLAUDE.md §0 fixes a hierarchy: CLAUDE.md -> the matching docs/memory/*-memory.md companion -> deep companions (docs/architecture/, docs/reference/) -> **source code (always wins)**. This book sits *alongside* that hierarchy, not above it: reading a book chapter is how you learn what a subsystem is and why it exists; loading the matching memory doc is still the step you take immediately before editing that subsystem's code. The six memory docs and the src/ trees they route to:

Memory doc	Routes to	Nearest book chapter
------------	-----------	----------------------

agent-memory.md	src/agent/	Ch.21
runtime-memory.md	src/runtime/	Ch.02 , Ch.05
feature-memory.md	src/features/	Ch.07
engine-memory.md	src/engines/	Ch.10
governance-memory.md	src/governance/ config_validator.py	Ch.16
architecture-memory.md	multi-package, full spine	Ch.02 , Ch.04

Topics vs. this book

docs/topics/ (26 files) and this book overlap in subject but not in shape. A topic doc is a single, continuously-updated concept dossier - code anchor, entry points, tests, status, one file, kept in sync with code on every touching turn (CLAUDE.md §6.4). A book chapter is a fixed point in a narrative sequence - it teaches the concept in the order a new reader needs to encounter it, and links to the topic doc for the always-current, code-synced detail. Where both exist for the same concept (e.g. CRT, Fusion+Decision, Execution Planning, Ultron, the Agent), **the topic doc is the fresher, code-verified source**; this book's chapter is the narrative on-ramp to it.

The classification vocabulary this book uses

Per the book's own brief, every concept is classified as one of: **Production, Experimental, Research, Legacy, Deprecated, Partial, Unknown**. The repository doesn't use these exact seven words - it has its own, more granular status vocabulary, built up across different subsystems. This book maps onto that vocabulary rather than inventing a parallel one:

Book label	Repo-native equivalent(s) it draws on
Production	Wired into the live decision spine; docs/topics/ status <code>living</code> ; Closure & Authority Index <code>CLOSED</code> (fully proven boundary)
Experimental	docs/governance/ closure status <code>AUDITED</code> (reviewed, not certified) or <code>COMPLETE</code> (a sub-step, not the whole domain); config-gated but not the active default
Research	Lives under src/research/, docs/research-readiness/; findings with confidence <code>Likely/Possible</code> ; Authority Ladder tier "information exists" or "economic usefulness exists" but not "authority earned" (§6.5)
Legacy	docs/topics/ status <code>DORMANT</code> - real code, off the live path, sidecar/inert/orphaned
Deprecated	Explicitly superseded per a finding's <code>SUPERSEDED/INVALIDATED_BY</code> marker (CLAUDE.md §6.2 rule 4 - the row is kept, never deleted)
Partial	Built but unwired (e.g. TradeNet v2, F-005) or docs/topics/ status <code>stub</code>
Unknown	No finding, topic, or closure artifact covers it - flagged in each chapter's Unresolved Questions, never guessed

A chapter's "Classification" section always uses this table's left column, so the label means the same thing everywhere in the book even though the underlying evidence format differs by subsystem.

Classification

Not applicable - this chapter defines classification rather than applying it.

Authoritative sources

- `docs/knowledge-map.md` - the nine existing record systems and how they connect (read in full for this chapter).
- `docs/memory/README.md` - the memory hierarchy and task->doc routing table.
- `docs/topics/readme.md` - the topic index, its status vocabulary (`living/stub/DORMANT`), and its explicit relationship to the idea-governance "Bricks" framework (kept parallel, not merged, by a 2026-06-01 decision).
- `CLAUDE.md` §0 (Architecture Memory Policy), §2 (Companion Documentation table).
- `docs/governance/closure_authority_index.json` and its `CLOSED/AUDITED/COMPLETE/OPEN` vocabulary (see [Chapter 17](#) for the full explanation of that index).

Unresolved questions

None - this is a navigational chapter over existing, verified index documents.

Previous: [Chapter 02 - The Invariants and the Happy Flow](#) * **Next:** [Chapter 04 - Architecture at a Glance](#)

Related: [Chapter 17 - Truth Maintenance](#) (the closure/authority vocabulary in full)

Memory: `docs/memory/README.md` (this chapter's primary subject).

Chapter 04 - Architecture at a Glance

Part I - Foundations

Status of this chapter: Written

Why this chapter exists

Chapters 1-3 gave the *why* and the *how to navigate*. Before descending into Part II's per-concept chapters, a reader needs one honest picture of the *shape* of the codebase: how big each piece is, what depends on what, and where the boundaries are. This chapter is that picture, built from the repo's own auto-generated architecture artifacts rather than re-derived by hand (so it stays verifiable against `docs/architecture/code-map.generated.md` and `module-roles.generated.md` whenever they're regenerated).

What problem it solves

`src/` alone has 1,400+ files across 36 subpackages. Without a map, "where does this change land?" is a search problem every time. This chapter turns it into a lookup.

What you need to already know

[Chapter 2](#) - the decision spine this map is organized around.

The idea

The repository is not one clean package

At the repo root, alongside the real source tree, sit years of working-repo residue: PDFs, zip backups, ad-hoc probe scripts, planning artifacts. The load-bearing top-level entries are:

Directory	Purpose	Approx. size
<code>src/</code>	The application itself	1,400+ files, 36 subpackages
<code>scripts/</code>	CLI/operational entry points (thin wrappers over <code>src/</code>)	500+ files, 18 subdirs
<code>docs/</code>	The knowledge corpus this book sits inside	~900 files
<code>configs/</code>	Production/research/experimental config surface	~65 files
<code>tests/</code>	Pytest suite, mirrors <code>src/</code> structure	~1,200 files
<code>multi_llm/</code>	Multi-LLM coordination protocol (see Ch.23)	~33 files
<code>data/</code> , <code>models/</code> , <code>results/</code>	Market data, trained artifacts, run outputs (mostly generated)	large, non-canonical
<code>CLAUDE.md</code>	The root AI-agent operating manual	-
<code>assistant_project.md</code>	The append-only session log - the spine of <i>history</i> (see Ch.03)	-

The 36 `src/` subpackages

Grouped roughly by the Part of this book that covers them (a subpackage not yet covered by a written chapter is marked so honestly rather than guessed at):

Subpackage	Covers	Book chapter
<code>features/</code>	Feature schema/pipeline/formulas	Ch.07
<code>config_layer/</code>	CRT engine, config schema/validation/goal schema, execution planner	Ch.06 , Ch.08 , Ch.13
<code>engines/</code>	The four scoring engines	Ch.10
<code>interpreters/</code>	Pattern interpreter contract (P&F, Wyckoff, ...)	Ch.09
<code>core/</code>	EngineRunner, FusionEngine, DecisionEngine, UltronRiskGate, core protocols	Ch.11 , Ch.12 , Ch.14
<code>runtime/</code>	Backtest engine (<code>backtest_v2.py</code>), live engine hook	Ch.05
<code>inout/</code> , <code>live/</code> , <code>execution/</code>	Live path: <code>inout/</code> = data fetchers; archived strategy rail under <code>archive/inout_legacy/</code>	Ch.15
<code>governance/</code>	Promotion, validators, registries, audit trail	Ch.16
<code>agent/</code>	AI automation agent (modes, tools, plan compiler)	Ch.21
<code>control_plane/</code>	Command registry / orchestration	Ch.22
<code>multi_llm/</code>	Multi-LLM coordination runtime	Ch.23
<code>research/</code>	Research programs (416 files - the largest subpackage by far)	Ch.19 , Ch.20
<code>bitnet/</code>	Low-bit zone-gate model	Ch.10
<code>training/</code>	Model training pipelines	Ch.16 (feeds the Training kitchen feeder)
<code>expansion/</code> , <code>journal/</code> , <code>replay/</code> , <code>portfolio/</code> , <code>regime/</code> , <code>retrieval/</code> , <code>strategies/</code> , <code>msip/</code> , <code>scanner/</code> , <code>feedback/</code> , <code>cognitive/</code> , <code>monitoring/</code> , <code>data_ingestion/</code> , <code>uat/</code> , <code>llm_research/</code> , <code>analytics/</code> , <code>events/</code> , <code>validation_access/</code> , <code>search/</code> , <code>logs/</code> , <code>ui/</code> , <code>utils/</code>	Sidecar / offline / supporting - see orientation table below	Not each given a full chapter; MSIP brief in Ch.18

Orientation map - packages without a dedicated chapter (Grok review pass)

These packages are **real code**, but they are either off the live decision spine by design (DORMANT / sidecar), research-only, or supporting infrastructure. This table is an orientation aid, not a claim that each deserves a full chapter tomorrow.

Package	One-line role	Typical posture
msip/	Shadow Market State Vector + emitters	Research/shadow - brief in Ch.18
replay/	Deterministic replay / harness support	Supporting runtime research
portfolio/	Portfolio allocation / validation helpers	Built; spine still single-candle (see findings F-013 class)
regime/	Regime labeling / observers	Research interpreters + diagnostics
strategies/	S01-S10 strategy modules	Mostly wrapper/scaffold; CRT wrapper is the familiar one
scanner/	Opportunity / setup scanning	Sidecar; not the live single-candle spine
expansion/	Bounded parameter expansion / search	Research/ops tooling
cognitive/, feedback/, journal/	Sidecar memory / feedback / journal	Explicitly DORMANT or non-spine consumers (F-012 class)
monitoring/, analytics/, events/	Telemetry and analysis	Supporting; not decision authority
retrieval/, search/, llm_research/	Retrieval / LLM research helpers	Experimental / research
data_ingestion/, uat/, validation_access/	Ingest + UAT + validation access	Ops / test surfaces
logs/, ui/, utils/	Logging helpers, UI remnants, shared utils	Supporting; some UI under archive

The dependency shape, at the top level

docs/architecture/code-map.generated.md's L0 package-dependency graph (auto-regenerated by scripts/analysis/gen_code_map.py) records edges like: agent -> config_layer, agent -> control_plane, agent -> core, agent -> governance, agent -> runtime (the agent depends on almost everything it can act on, never the reverse); and bitnet -> engines, bitnet -> features, bitnet -> utils (BitNet is a downstream consumer of the engine/feature layer, not upstream of it - this matters when reading [Chapter 10](#)'s account of the Zone Gate engine). The full 35-package graph is larger than is useful to reproduce here; treat this chapter's excerpt as an orientation and the generated file as the source of truth, since it's regenerated directly from imports rather than hand-maintained.

Two companion generated docs worth knowing about

- docs/architecture/module-roles.generated.md - a one-line role per src/ module, sourced from each module's own docstring. Useful as a fast "what does this file do" lookup when a chapter's "Authoritative sources" list points you at an unfamiliar path.
- docs/reference/architecture.md - a more traditional onboarding doc (tech stack with versions, directory structure, data-flow diagrams for the backtest/live/governance/agent paths, design patterns, external integrations, .env/config overview, execution entry points). This book does not reproduce that doc's content - treat it as the reference companion to this chapter's narrative framing.

Classification

Concept	Status
src/ 36-package structure	Production (the whole tree ships)
research/ (416 files)	Mixed - see Ch.19/Ch.20
The 9 subpackages marked DORMANT above	Legacy
Generated architecture docs (code-map, module-roles)	Production tooling, regenerate-don't-hand-edit

Authoritative sources

- `docs/reference/architecture.md` - tech stack, directory tree, data-flow diagrams, design patterns, config overview, entry points (headers verified this session; full content not reproduced here by design - see [Ch.03](#)'s anti-duplication policy).
- `docs/architecture/code-map.generated.md` - the authoritative import graph (regenerate via `python scripts/analysis/gen_code_map.py`).
- `docs/architecture/module-roles.generated.md` - per-module one-line roles.
- `docs/topics/readme.md` - the DORMANT subsystem table.

Unresolved questions

- The 22 subpackages in the "not yet individually chaptered" row above are real, verified to exist, but this pass didn't have room to give each its own chapter. Tracked in [A2](#).
- `docs/reference/architecture.md`'s full body (design patterns, external integrations, entry points) was header-verified but not read in full this session - a future pass should confirm this chapter's summary against the full text.

Previous: [Chapter 03 - How This Book Fits](#) * **Next:** [Chapter 05 - Data Ingestion and the No-Lookahead Discipline](#)

Related: [Chapter 02 - The Invariants and the Happy Flow](#)

Memory: `docs/memory/architecture-memory.md` (deep companion for full cross-package detail).

Chapter 05 - Data Ingestion and the No-Lookahead Discipline

Part II - Market Understanding

Status of this chapter: Written

Why this chapter exists

Everything downstream - features, engines, fusion, decisions - is only as trustworthy as the candle stream feeding it. Invariant #1 from [Chapter 2](#), deterministic replay, is only meaningful if the data itself never leaks future information into the past. This chapter covers Step 1 of the spine: how a candle enters the system, and what stops it from cheating.

What problem it solves

A backtest that can see the future is not a backtest - it's a curve-fit generator. This chapter explains the layered defense against that failure mode, and is honest about where that defense is strong versus merely adequate.

What you need to already know

[Chapter 2](#)'s seven-step spine - this chapter is Step 1 of it.

The idea

Where a candle comes from

In backtest, `runtime/backtest_v2.py`'s `CandleLoader.stream()` reads historical OHLCV and yields candles one at a time, in order. In live mode, `inout/scanner.py` plays the equivalent role, reading from a broker/data feed. Either way, the unit that leaves this step and enters [the feature pipeline](#) is a single `Candle`.

The four-layer integrity stack

No-lookahead isn't one gate - it's the *conjunction* of four layers, and that framing matters: a gap in any one layer doesn't necessarily mean data leaked, but it does mean the safety margin is thinner than the others.

- **L1 - schema.** Every candle is validated against the expected OHLCV shape before it's used.
- **L2 - temporal.** Candles must arrive in strictly increasing chronological order; out-of-order or duplicate timestamps are rejected. This backstop is inline and always-on, everywhere `stream()` is consumed.
- **L3 - dataset-level pre-flight.** A heavier, whole-dataset integrity check (`dataset_integrity.validate_dataset`) that runs *before* a backtest starts, catching structural problems (gaps, session-boundary anomalies) that a candle-by-candle check can't see.
- **RT - real-time generator causal ordering.** In the live path, the generator itself is constructed to only ever emit a candle once it has actually closed.

The honest caveat: L3 doesn't run everywhere

A repo-wide census found that the L3 pre-flight check runs at only **two call sites**, both inside `backtest_v2` (`_preflight_dataset` and `validate_universe`). Every other consumer of `CandleLoader.stream()` - the entire `src/research/` pipeline, analytics, governance, replay, `config_validator`, and the broader `scripts/` research/training/analysis fleet - streams candles with only the always-on L1/L2 inline backstop, not L3. This was captured as a documentation

correction: an earlier claim that L3 ran at "every backtest entry point" was itself an overclaim, now corrected. The scope of what this means is narrow and important to get right: **L1/L2 alone did still enforce schema and chronological ordering** on every one of those research runs - this is a *single-layer fragility* finding (L3's extra dataset-wide safety margin is concentrated in two call sites), not a claim that leakage occurred. It does not reopen or invalidate any research finding built on those paths.

Classification

Concept	Status
L1 (schema) / L2 (temporal) inline backstop	Production, always-on everywhere <code>stream()</code> is used
L3 dataset pre-flight	Production, but scope-limited to two <code>backtest_v2</code> call sites
RT causal ordering (live generator)	Production

Authoritative sources

- docs/architecture/signal-flow.md §1, Step 1 - the four-layer table with module/failure-mode detail.
- src/runtime/backtest_v2.py - `CandleLoader.stream()`, `_preflight_dataset`, `validate_universe`.
- src/inout/scanner.py - the live-mode equivalent.
- docs/current-findings.md - F-039 (the L3-scope correction, certain confidence, docs/research-only authority).

Unresolved questions

None beyond what F-039 itself already scopes precisely - see the finding for exact call-site detail.

Previous: [Chapter 04 - Architecture at a Glance](#) * **Next:** [Chapter 06 - The Market Ontology](#)

Related: [Chapter 07 - The Feature Pipeline](#)

Memory: docs/memory/runtime-memory.md.

Chapter 06 - The Market Ontology: Canonical Semantic Authority

Part II - Market Understanding

Status of this chapter: Written

Why this chapter exists

Before a candle can be scored, every quantity computed from it - a wick length, a displacement strength, a zone boundary - needs one, and only one, agreed-upon meaning across the whole codebase. This chapter covers the artifact that owns that meaning: `configs/formulas/market_ontology.yaml`. Understanding it first is what makes [Chapter 7](#)'s feature vector, and every engine chapter after it, legible rather than a wall of unexplained formulas.

What problem it solves

Historically, this repository suffered from *definitional drift* - the same-sounding quantity (`wick_size`, `body_ratio`) computed two different ways in two different files, silently. The ontology exists to make that structurally impossible going forward: one canonical node per concept, with every consumer required to bind to it rather than re-derive it locally.

What you need to already know

[Chapter 5](#) - the ontology's `base_inputs` are exactly the OHLCV fields that survive ingestion.

The idea

What the ontology actually is

`market_ontology.yaml` (version 1.4 as of this writing, ~2,600 lines) is the repository's declared "**permanent semantic authority**." It defines every market concept as a typed node: primitives (raw geometric building blocks), `feature_compositions`, `derived_metrics`, `rolling_indicators`, `temporal_context`, `structural_states`, `indicator_identities`, `execution_behaviours`, plus invariants and - importantly - `canonical_unknowns`, an explicit place to register a discovered-but-not-yet-understood market behavior rather than let it live only as a code comment or a TODO.

Every node carries a fixed field set: `id`, canonical name, aliases, a `knowledge_status` on a refinement ladder, its formula, its dependencies, and who produces/consumes it. The refinement ladder itself is worth internalizing, because it's the vocabulary this book borrows for describing how mature any given piece of market knowledge is:

```
UNKNOWN -> OBSERVED -> CHARACTERIZED -> MATHEMATICALLY_DEFINED -> FORMULA_DERIVED -> VALIDATED ->
PRODUCTION_CERTIFIED -> STABLE
```

A node only reaches `PRODUCTION_CERTIFIED` by demonstrating measured economic value - see the Authority Ladder in [Chapter 17](#). Reaching a mathematically clean formula is necessary but never sufficient for that top tier.

Why "authority" here means something specific

The ontology's authority is "literal supersession" for *meaning* - when a semantic question arises, this file is where the answer originates, and code, config, research, and even historical findings are expected to synchronize to it, not the other way around. But that authority is bounded by two physical constraints, and both matter for anyone editing it:

1. **Frozen runtime keys.** The `primitives`, `feature_compositions`, and `derived_metrics` sections are read at *import time* by `crt_engine_v2.py` (via `fm_resolve.bind_phase2 crt_callable()`). They can only grow additively - removing or renaming a key there is a breaking change to running code, not a documentation edit.
2. **Behavior changes still need governance.** "Ontology first" governs where a semantic change *originates*, not an automatic license to mutate runtime behavior. Any change that actually alters production output still has to clear the parity-proof and promotion gate from [Chapter 16](#) - the ontology can't bypass governance, it feeds it.

Downstream reach

Every consumer that computes market meaning is expected to bind to this file rather than re-derive it: `crt_engine_v2.py` at import time (frozen sections), the feature registry / formula registry facade that [Chapter 7](#) depends on, and - per the ontology's own evolution contract - effectively everything else in the pipeline traces its semantic meaning back here. This is why it's Chapter 6, not buried in an appendix: it's upstream of every later Part's vocabulary.

Classification

Concept	Status
Ontology as the semantic authority for meaning	Production, actively governed (CLAUDE.md §6.6)
Frozen runtime keys (<code>primitives/feature_compositions/derived_metrics</code>)	Production, additive-only
<code>canonical_unknowns</code> section	Explicit, honest placeholder for open questions - not a gap in the doctrine, a designed feature of it

Authoritative sources

- `configs/formulas/market_ontology.yaml` - the ontology itself (2,600+ lines, version 1.4, 2026-07-24).
- `docs/governance/MARKET_ONTOLOGY_EVOLUTION_CONTRACT.md` - the full evolution contract this chapter summarizes.
- CLAUDE.md §6.6 - the Canonical Market Ontology Evolution Contract (the two mechanical constraints, the refinement ladder, the Automatic Semantic Discovery ritual).
- `src/features/registry/` (formula registry facade), `src/config_layer/crt_engine_v2.py` (import-time binding).
- `docs/current-findings.md` F-047 - the ontology's authoritative status, source-verified.

Unresolved questions

None for this pass - the ontology's own evolution contract is unusually explicit about its own scope and limits, which made this chapter straightforward to write faithfully.

Previous: [Chapter 05 - Data Ingestion and the No-Lookahead Discipline](#) * **Next:** [Chapter 07 - The Feature Pipeline and the Canonical Vector](#)

Related: [Chapter 17 - Truth Maintenance](#) (the Authority Ladder that governs when a node earns production certification)

Memory: `docs/memory/feature-memory.md`.

Chapter 07 - The Feature Pipeline and the Canonical Vector

Part II - Market Understanding

Status of this chapter: Written

Why this chapter exists

The four scoring engines in Part IV don't see a candle - they see a fixed-length numeric vector. This chapter covers where that vector comes from, what's in it, and - because this is where the book caught a live piece of documentation drift while researching it - exactly how many dimensions it actually has today, as opposed to what the reference doc still says.

What problem it solves

Explains the single most-consumed data contract in the codebase: `CANONICAL_FEATURES`. Every engine, BitNet, and most of the research pipeline binds to this vector's shape.

What you need to already know

[Chapter 6](#) - the feature math this pipeline runs is derived from the ontology's `primitives/feature_compositions/derived_metrics`, not invented locally.

The idea

What the pipeline does

`src/features/feature_pipeline.py`'s `FeaturePipeline.run()` takes an enriched candle stream and produces `(enriched_df, vectors)` - a dataframe plus the fixed-length numeric vectors that engines actually consume. The vector's *shape* - which fields, in which order, at what dimensionality - is defined once, in `src/features/feature_schema.py`, as `CANONICAL_FEATURES`. Nothing downstream is supposed to reconstruct that shape independently; `core/engine_runner.py` imports it directly, as do `engines/zone_gate_engine.py`, `engines/heuristic_gaussian_engine.py`, and `crt_engine_v2.py`.

A live doc-drift, caught and flagged (not silently fixed)

`docs/reference/schemas.md` §4 still documents a **38-dimension, schema v3.0** feature tuple - `CANONICAL_FEATURE_DIM = 38`, with a single `macd_hist` field and a `wick_size` field. The code that actually runs today, `src/features/feature_schema.py`, is at **39 dimensions, schema v4.0** (`CANONICAL_FEATURE_DIM: int = 39`): `macd_hist` was split into `macd_hist_raw` and `macd_hist_z` (indices 18-19), and `wick_size` was renamed to `candle_range` (index 28). A `SCHEMA_V3_ALIASES` mapping exists solely so old v3 records can still be *read*; it is not what new code produces. The pipeline module's own docstring calls this out explicitly, warning readers to quote the code constant rather than a number, because the docstring itself had briefly carried a stale "38" through the v2->v3->v4 migrations.

This is a textbook `DOC_DRIFT` case per `CLAUDE.md` §6.2 (code wins, doc needs fixing) - the fix itself (F-062, 2026-07-31) already shipped on the *code* side; what appears to remain is that `docs/reference/schemas.md`'s table wasn't updated to match. **This book flags it and points at the authoritative constant; per this chapter's own scope, it does not silently edit `schemas.md` as a side effect of being written** - that's a separate, explicit doc-drift turn for whoever picks it up next (tracked in [A2](#)).

The rule this leaves you with: when you need the current dimensionality or field order, read `src/features/feature_schema.py`'s `CANONICAL_FEATURES` constant directly. Don't quote a number from a doc - quote the constant.

Who consumes the vector

`core/engine_runner.py` is the hub: it builds the 39-dim vector once per candle and hands it to all four engines uniformly. This is what makes "all four engines or nothing" ([Chapter 2](#)'s invariant #2) mechanically enforceable - every engine is looking at the exact same input surface, so a missing engine is a missing *scorer*, never a missing *input*.

Classification

Concept	Status
<code>feature_pipeline.py</code> / <code>FeaturePipeline.run()</code>	Production
<code>CANONICAL_FEATURES</code> @ 39-dim, schema v4.0 (<code>feature_schema.py</code>)	Production, current
<code>docs/reference/schemas.md</code> 's 38-dim v3.0 table	Stale documentation - <code>DOC_DRIFT</code> , code already fixed (F-062), doc not yet reconciled
<code>SCHEMA_V3_ALIASES</code>	Production, read-compatibility only - not what new code emits

Authoritative sources

- `src/features/feature_pipeline.py` - the pipeline entry point (`FeaturePipeline.run()`).
- `src/features/feature_schema.py` - **the single source of truth** for `CANONICAL_FEATURES` and `CANONICAL_FEATURE_DIM`.
- `docs/reference/schemas.md` §4 - the reference doc, currently lagging the code (flagged above).
- `docs/current-findings.md` - F-062 (the code-side rename that created this v3/v4 gap).
- `docs/topics/feature-schema.md` - the always-synced topic doc for this concept.

Unresolved questions

- Should `docs/reference/schemas.md` §4 be updated to 39-dim/v4.0 now?** This is a real, low-risk `DOC_DRIFT` fix (per `CLAUDE.md` §6.2's gate calibration, an unambiguous stale-fact fix like this can be auto-fixed without user approval) - it was deliberately left out of this book-writing pass to keep this task's scope to the book itself. Recorded in [A2](#) as the single most actionable follow-up this book surfaced.

Encyclopedia: [E1b - Features & Registry](#) * machine twin `encyclopedia/encyclopedia_rows.jsonl` (phase=E1b)

Previous: [Chapter 06 - The Market Ontology](#) * **Next:** [Chapter 08 - The CRT State Machine](#)

Related: [Chapter 10 - The Four Scoring Engines](#) (the vector's consumers)

Memory: `docs/memory/feature-memory.md`.

Chapter 08 - The CRT State Machine: the Spine

Part III - Market Semantics

Status of this chapter: Written

Why this chapter exists

CRT (Candle Range Theory) is described everywhere in this repository as "the spine" - not because it's the most powerful of the four scoring engines, but because unlike the other three, it isn't a single number-producing function. It's a stateful lifecycle that gives a *sequence* of candles structural meaning, and that structure is what the rest of Part IV's decision-making ultimately reasons about.

What problem it solves

A single candle's numeric features (Chapter 7) can't tell you "this is currently mid-setup, three candles into a retest of a liquidity sweep." Only a state machine that persists across candles can. CRT is that state machine.

What you need to already know

[Chapter 6](#) and [Chapter 7](#) - CRT's frozen ontology sections and its input vector are both prerequisites already covered.

The idea

Two files named "CRT engine" - know which one you're reading

There are two distinct modules worth telling apart precisely, because confusing them is an easy mistake:

- `src/config_layer/crt_engine_v2.py` - the full deterministic state machine. Internally composed as `RangeDetector -> StateMachine -> (internal risk/execution helpers) -> ...`, and this is the module that binds the ontology's frozen sections at import time (see [Chapter 6](#)).
- `src/engines/crt_engine.py` - a thinner wrapper that `EngineRunner` (Part IV) actually calls as one of the four scoring engines. Its `compute()` delegates to `engines/scoring_engine.py`'s `compute_scores`, and what it returns to Fusion is a single number - CRT's *score*, not its full state.

So: the **state** lives in `crt_engine_v2.py`; the **score** that Fusion sees comes from the thinner `engines/crt_engine.py` wrapper. Both matter, but they answer different questions - "where is this setup in its lifecycle" versus "how good does this setup look right now."

The nine-state lifecycle

Already introduced in [Chapter 2](#):

```
RANGE -> SWEEP -> DISPLACEMENT -> EXPANSION -> RETEST -> EXECUTION -> RESOLUTION
```

plus two branches:

- `RANGE ↔ SHADOW_PENDING -> SWEEP` - an early-detection branch that lets the engine track a candidate setup before it's confirmed.
- `EXPANSION -> EXPIRED -> RANGE` - a TTL-based soft-archive: a setup that goes stale before reaching RETEST ages out rather than lingering indefinitely.

This is the **entire** legal transition graph (invariant #6 from Chapter 2) - there is no path through the state machine that isn't one of these edges. The authoritative transition table lives at `state_identity.py:69` (extracted from `crt_engine_v2` to break an import cycle, re-exported for compatibility) with an immutable view at `state_topology.py:109`, and is narrated in `docs/architecture/event-taxonomy.md §3`.

CRT is a closed research surface - what that does and doesn't mean

CRT carries the repository's only **CLOSED** status in the Closure & Authority Index (see [Chapter 17](#) for what that index means in general). The declared boundary is narrow and precise: *OHLCV -> CRT TRADE_OPENED, and only that*. It does not mean "CRT is proven profitable" - the Research Programs in Part VII repeatedly test CRT-adjacent hypotheses (completed sweep->displacement->retest sequences, weekly-calendar sweep variants) and find no economic edge. CLOSED here means: the *mechanism itself* - candle in, state transitions, `TRADE_OPENED` event out - has been audited end-to-end and is considered a stable, trustworthy piece of machinery. Whether what it detects is *worth trading* is a separate, open question, owned by Part VII.

Classification

Concept	Status
The 9-state CRT lifecycle + transition graph	Production, CLOSED (mechanism boundary only)
<code>crt_engine_v2.py</code> (full state machine)	Production
<code>engines/crt_engine.py</code> (score wrapper for EngineRunner)	Production
"Is a completed CRT structure economically tradeable"	Research - repeatedly tested, currently null (see Ch.19)

Authoritative sources

- `src/config_layer/crt_engine_v2.py` - the full state machine.
- `src/engines/crt_engine.py`, `src/engines/scoring_engine.py` - the EngineRunner-facing score wrapper.
- `src/config_layer/state_identity.py:69`, `state_topology.py:109` - the authoritative transition graph.
- `docs/architecture/event-taxonomy.md §3` - the narrated transition table.
- `docs/governance/crt_closure_report.md` - the CLOSED boundary declaration and its exact scope.
- `docs/topics/crt-spine.md` - the always-synced topic doc.

Unresolved questions

None for the mechanism itself - CRT is the single most thoroughly audited surface in the repository. Its *economic* status is intentionally left open here and answered properly in Part VII.

Previous: [Chapter 07 - The Feature Pipeline](#) * **Next:** [Chapter 09 - Interpreters and the Pattern Contract](#)
Related: [Chapter 10 - The Four Scoring Engines](#) * [Chapter 17 - Truth Maintenance](#) * [Chapter 19 - The Research Programs](#)
Memory: `docs/memory/engine-memory.md`.

Chapter 09 - Interpreters and the Pattern Contract

Part III - Market Semantics

Status of this chapter: Written

Why this chapter exists

CRT (Chapter 8) is one way of giving candle sequences structural meaning. It is not the only pattern vocabulary a trader might reach for - Point & Figure, Wyckoff, Market Profile, Order Flow are all classical alternatives. This chapter covers how this repository lets such ideas in *without* letting them claim intelligence they haven't earned.

What problem it solves

Prevents "we added a pattern detector" from silently becoming "we trust this pattern detector." Every interpreter has to prove itself the same way, before it can influence anything.

What you need to already know

[Chapter 1](#)'s "measured, never asserted" principle, and
[Chapter 8](#) - interpreters are a parallel idea to CRT, not a replacement.

The idea

The Interpreter Contract (Level 4)

`src/interpreters/contract.py`, `adapter.py`, and `reference.py` define a fixed contract any pattern-recognition module must satisfy before it's allowed to matter. A candidate pattern is wrapped as an `InterpreterHypothesis` and run through the same `forward_walk` evaluation plus `QualificationGate` machinery used elsewhere in the research pipeline - the same yardstick, reused, not a bespoke one invented per interpreter. This is deliberate: it means "does this pattern help" is answered the same way every time, and a pattern can't get a friendlier evaluation just because its author believed in it more.

The one interpreter that's actually been run through it

Point & Figure (P&F, implementation id `PNF-v1`, double-top/double-bottom patterns) is the first interpreter to complete this process on real data. The result was a clean **REJECT**: shadow-measured on crypto majors, it carried no standalone edge, and scored worse than random controls (a negative delta-expectancy of about -0.039). This is reported here not as a failure of the interpreter framework but as evidence the framework *works as intended* - it correctly declined to grant authority to an idea that didn't earn it, exactly the discipline [Chapter 1](#) described. Per the repository's own research discipline, this closes P&F under the current ontology; reopening it needs a genuinely new ontology, not another sweep of the same one.

What this means for other interpreters

Nothing else currently in `src/interpreters/` has cleared the contract - until one does, the honest status of the whole subsystem is: infrastructure exists, one candidate has been tested and rejected, the rest are unevaluated. None of them influence the live decision spine.

Classification

Concept	Status
Interpreter Contract (<code>contract.py/adapters/reference.py</code>)	Production infrastructure
P&F / PNF - v1	Research - tested, REJECTED (F-028)
Wyckoff, Market Profile, Order Flow interpreters	Unknown / not yet run through the contract

Authoritative sources

- `src/interpreters/contract.py`, `adapter.py`, `reference.py` - the contract itself.
- `docs/topics/interpreter-contract.md` - the always-synced topic doc, entry point, and test references.
- `docs/current-findings.md` F-028 - the P&F result, with its exact scope and confidence.
- `docs/architecture/goal.md` - the "measured against the Goal Layer, never asserted" framing this chapter draws its principle from.

Unresolved questions

- Whether Wyckoff/Market Profile/Order Flow interpreters have any code beyond scaffolding, and if so how far along the contract pipeline they are, wasn't verified in this pass - flagged in [A2](#).

Previous: [Chapter 08 - The CRT State Machine](#) * **Next:** [Chapter 10 - The Four Scoring Engines](#)

Related: [Chapter 19 - The Research Programs](#) (the same falsification discipline, at system scale)

Memory: `docs/memory/engine-memory.md`.

Chapter 10 - The Four Scoring Engines

Part IV - Decision Making

Status of this chapter: Written

Why this chapter exists

`EngineRunner` (invariant #2, [Chapter 2](#)) runs exactly four engines against every candle, unconditionally, and rejects the candle outright if any one is missing. This chapter is the canonical explanation of the other three - Gaussian, Zone Gate, RR - since CRT already owns [Chapter 8](#). It's also where this book delivers its most important warning about the decision spine: three of the four engines have been formally audited and found to contribute far less signal than their names suggest.

What problem it solves

Explains what each engine actually computes (as opposed to what its name implies), and gives each its correctly scoped status - because getting this wrong is exactly how a system ends up trusting a number that's secretly closer to a constant.

What you need to already know

[Chapter 7](#) (the 39-dim vector every engine consumes) and [Chapter 8](#) (CRT, already covered).

The idea

CRT - see Chapter 8

The CRT engine's score comes from `src/engines/crt_engine.py`, delegating to `engines/scoring_engine.py`. Its full explanation, including the state-machine/score-wrapper distinction, is [Chapter 8](#) - this chapter doesn't repeat it, per the book's Canonical Concept Rule.

Gaussian - a channel that has largely collapsed to a constant

Two implementations exist behind a config switch (`gaussian_impl`): `HeuristicGaussianEngine` (`src/engines/heuristic_gaussian_engine.py`, the live default, an EMA/momentum kernel) and `MLGaussianEngine` (`src/engines/ml_gaussian_engine.py`, a trained Naive-Bayes model, fail-open to 0.5 on any error). Both ultimately route through the central scorer, `src/config_layer/crt_gaussian_scorer.py`.

The audited finding here is stark: the live Gaussian channel is an **unparameterized kernel that degenerates to a near-constant score of roughly 0.8825**. All eleven entries in `gaussian_registry.json` lack `mu/sigma`, so the score-normalization code's defaults (0/1) fire even on a *successful* registry load - meaning no learned parameter reaches the score at all, not even a mistrained one. Compounding this, `tanh(momentum_score)` saturates on 93-100% of bars because of a since-registered dimensional-mix defect upstream in the feature math, discarding the sign of momentum entirely. An ablation test (pinning the channel at its saturated value, at neutral 0.5, and at live) produced byte-identical decisions across all three settings on four crypto majors - proof the channel is currently non-pivotal on both an information axis and a level axis. The model artifact that trained these registry entries was also confirmed to have trained on contaminated labels (the same F-022 labeling artifact discussed in [Chapter 17](#)).

Zone Gate - a real geometric hard gate, honestly measured as not adding edge

`src/engines/zone_gate_engine.py` scores candle geometry against `models/zone_registry.json` (8 zones), fail-open on registry errors, via `_compute_soft_zone_score()`. Unlike Gaussian, this channel is not degenerate - it's a working geometric filter that passes roughly 85% of candles it evaluates. But a fusion-weight and threshold sweep (weight $\in \{0, 0.2, 0.4, 0.6\}$, threshold $\in \{0, 0.25, 0.5\}$) produced byte-identical trade sets across every combination - the zone channel is redundant with what the rest of fusion already decides, not under-weighted. A separate re-derivation of its stored win/loss labels through an honest forward-walk (rather than the contaminated F-022 labeling) found the same story at the label level: zero of the eight zones clear a positive expectancy honestly. None of this implies live risk - the gate is purely geometric and fails open - but it does mean tuning its weight or threshold is currently a lever with no effect to pull.

RR - not actually a reward:risk engine

`src/engines/rr_engine.py`'s own module header states, plainly: it is "formerly misnamed 'RR Engine'" - it does not compute forward-looking reward:risk at all. What it actually scores is candle-close-position *commitment*: `max(upper_body, lower_body)` against the candle's range, i.e. how decisively a candle closed toward one extreme. The book keeps its file/class name (`RREngine`) for traceability but flags the naming mismatch here so a reader doesn't go looking for reward:risk math in the wrong file - the actual reward:risk economics for a trade live entirely in [UltronRiskGate](#).

A parallel line of investigation found a second, distinct issue with a now-disabled sibling component, `rr_fusion`: its confidence-gate math was mis-scaled for the dimensionality of its input (a 27-degree-of-freedom Mahalanobis form whose in-distribution confidence floor sits near $1.4e-6$, far below its own 0.3 bypass threshold) - so severely that **100% of its own training data** failed to pass its gate. That component stays disabled in the active config; a follow-up shadow test on its underlying model (bypassing the broken gate) found apparent out-of-sample discrimination, but on labels later confirmed to be F-022-contaminated - leaving its true economic value formally indeterminate, not proven and not disproven.

Classification

Concept	Status
CRT score	Production - see Ch.08
Gaussian (<code>HeuristicGaussianEngine</code> / <code>MLGaussianEngine</code>)	Production (wired, live), AUDITED: non-pivotal, near-constant
Zone Gate	Production (wired, live), AUDITED: redundant, no marginal edge
RR / <code>RREngine</code> (candle-commitment score)	Production (wired, live), AUDITED: correctly named commitment score, not RR
<code>rr_fusion</code> (disabled sibling)	Built, disabled , economic value indeterminate

Authoritative sources

- `src/engines/heuristic_gaussian_engine.py`, `ml_gaussian_engine.py`, `src/config_layer/crt_gaussian_scorer.py`.
- `src/engines/zone_gate_engine.py`, `models/zone_registry.json`.
- `src/engines/rr_engine.py`.
- `docs/topics/scoring-engines.md` - the always-synced topic doc covering Gaussian/Zone-Gate/RR together.

- `docs/current-findings.md` - F-036 (Zone Gate redundancy), F-038 (RR-fusion mis-gate origin), F-041B (honest zone labels), F-044 (dof-aware gate diagnosis), F-045/F-059 (RR clean-label re-test), F-060 (Gaussian constant collapse), F-061 (dimensional-mix mechanism generalized).
- `docs/governance/gaussian_lineage_audit.md`, `zonegate_lineage_audit.md`, `rr_lineage_audit.md` - the full per-engine AUDITED closure artifacts (see [Chapter 17](#) for what "AUDITED" means as a closure status).

Unresolved questions

- RR's clean-label re-test (F-059) reached a "keep-candidate, research-only" verdict - whether a future pass changes `rr_fusion`'s disabled status is an open governance decision, not something this book resolves.

Previous: [Chapter 09 - Interpreters and the Pattern Contract](#) * **Next:** [Chapter 11 - Fusion](#)

Related: [Chapter 08 - The CRT State Machine](#) * [Chapter 17 - Truth Maintenance](#)

Memory: `docs/memory/engine-memory.md`.

Chapter 11 - Fusion: Combining Independent Signals

Part IV - Decision Making

Status of this chapter: Written

Why this chapter exists

Four engines each produce a number ([Chapter 10](#)). Something has to turn four numbers into one. This chapter covers that step - and the one place in the entire synchronous spine where a language model is allowed to touch a live decision, under tightly constrained conditions.

What problem it solves

Explains how Fusion combines scores, why a neural-model slot exists but does nothing yet, and exactly how narrow the LLM's involvement is - directly enforcing invariant #3 from [Chapter 2](#) ("the LLM is advice, never a trigger").

What you need to already know

[Chapter 10](#) - Fusion's primary input is the Gaussian channel, whose audited near-constant behavior matters directly to how much this step is really doing.

The idea

A layered scorer, not a weighted average

`src/core/fusion_engine.py`'s `FusionEngine.evaluate()` is structured as a sequence of layers rather than a flat weighted sum:

1. **Gaussian scorer as the primary anchor.** Given [Chapter 10](#)'s finding that Gaussian has largely collapsed to a near-constant, this is worth sitting with: the layer Fusion leans on hardest is currently the least informative of the four.
2. **A neural-model slot** - reserved for TradeNet v2, which is **built but never wired in** (F-005). This is a permanent, intentional stub, not an oversight in progress; wiring it up is a separate, explicitly gated qualification protocol (TN_QUAL_V1), not something that happens by accident.
3. **An LLM gate as an uncertainty tie-breaker** - and only that. It fires exclusively in a narrow confidence band, default `[0.45, 0.65]`. Any signal that's clearly strong or clearly weak never reaches the LLM at all. This is the concrete mechanism behind invariant #3: the LLM never decides a clear case, only helps break a tie in an ambiguous one, and (per [Chapter 2](#)) falls back to a neutral score after repeated failures rather than blocking.

What Fusion is, honestly, right now

Given the state of the channels feeding it - Gaussian near-constant, Zone Gate redundant with the rest of fusion ([Chapter 10](#)), RR correctly understood as a commitment score rather than reward:risk - Fusion's practical job today is closer to "pass CRT's signal through, sanity-checked by a narrow LLM tie-breaker on ambiguous cases" than "genuinely synthesize four independent opinions." That's not a criticism of the design - the architecture is exactly right for a system meant to *add* independent signal as each one earns its keep (Authority Ladder, [Chapter 17](#)) - it's just the honest current state, which the Research Programs in Part VII are directly aimed at changing.

Classification

Concept	Status
<code>FusionEngine.evaluate()</code> layered structure	Production
Gaussian-as-anchor	Production, but anchor is itself AUDITED near-constant (Ch.10)
Neural-model slot (TradeNet v2)	Partial - built, unwired (F-005)
LLM tie-breaker gate	Production, narrow-band only, fail-open

Authoritative sources

- `src/core/fusion_engine.py` - `FusionEngine.evaluate()`.
- `docs/topics/fusion-decision.md` - the always-synced topic doc (covers Fusion and Decision together).
- `docs/current-findings.md` F-005 (TradeNet stub) and F-060 (Gaussian anchor's audited status).
- `docs/governance/tradenet_lineage_audit.md`, `docs/governance/tradenet_qualification_protocol.md` - the wire-up path for the neural slot, if it's ever taken.
- `docs/architecture/goal.md` §3, invariant #3 - the LLM-as-advice rule this chapter's gate enforces.

Unresolved questions

None for this pass - Fusion's structure and its channels' statuses are both well-evidenced by Chapter 10's audits and the TradeNet lineage audit.

Previous: [Chapter 10 - The Four Scoring Engines](#) * **Next:** [Chapter 12 - The Decision Engine](#)

Related: [Chapter 21 - The AI Automation Agent](#) (the other place an LLM touches this repository, entirely off the synchronous spine)

Memory: `docs/memory/engine-memory.md`, `docs/memory/architecture-memory.md`.

Chapter 12 - The Decision Engine: Semantic Approval Only

Part IV - Decision Making

Status of this chapter: Written

Why this chapter exists

Fusion (Chapter 11) produces one combined score. Something has to turn that score into a binary ACCEPT/REJECT. This chapter covers that module - and a genuinely interesting piece of the repository's history: a multi-year-old bug that made this exact module a structural no-op, found and fixed by re-deciding *who owns what*, not by tuning a threshold.

What problem it solves

Explains what the Decision Engine actually decides today (semantic validity, not economics), and why that split exists.

What you need to already know

[Chapter 11](#) - the Decision Engine consumes Fusion's output directly.

The idea

What it decides

`src/core/decision_engine.py`'s `DecisionEngine` is described in its own source as "central execution authority - only this module decides execute vs reject." What it actually evaluates is purely semantic: is this a valid market opportunity, based on the fused score, the win-probability estimate, and zone validity? It uses a dynamically calibrated threshold (the 85th percentile of recent scores, clamped to `[0.45, 0.65]`) rather than a single fixed cutoff.

What it used to decide, and why that broke everything silently

Until it was fixed, the Decision Engine also carried a reward:risk gate - comparing an RR-like signal against a `rr_threshold` of 1.5. The problem: the value it was actually comparing was candle *polarity*, a quantity bounded in `[0.5, 1]` (this is the "commitment score" from [Chapter 10](#)'s RR engine discussion - a value that structurally can never exceed 1). Compared against a threshold of 1.5, this gate rejected as `low_rr` on **every single candle**: across a 70,002-candle test corpus, the gated code path executed zero times.

How it was resolved - an ownership decision, not a threshold tweak

The fix (F-048, 2026-07-24) wasn't to adjust the threshold or fix the polarity/RR mismatch in place - it was to ask *which module should own reward:risk economics at all*, and answer it cleanly: the Decision Engine's RR gate was **removed**. The Decision Engine is now semantic-approval-only (score / win-probability / zone / weak-component checks); real, cost-taxed reward:risk economics (`min_rr_ratio`, computed after the Execution Planner has determined actual SL/TP geometry) are owned solely by `UltronRiskGate`. The old `rr_threshold` config key is retained but retired (kept for hash-neutrality, no longer read for a decision). Because no real production path had ever actually fed genuine economic RR into the Decision Engine - only test injectors had - the removal was verified byte-identical against the live spine's historical output.

This is a good worked example of the repository's own governance discipline in Part VI: a bug that had silently zeroed out a gate for years was resolved by clarifying architecture (who owns reward:risk)

rather than papering over the symptom (retuning a threshold that was comparing the wrong quantities in the first place).

Classification

Concept	Status
<code>DecisionEngine.decide()</code> , semantic-approval-only	Production, current design (post F-048)
The old RR gate inside Decision Engine	Removed (F-048, 2026-07-24) - retained config key is retired, not read
Dynamic threshold calibration (85th percentile, clamped)	Production

Authoritative sources

- `src/core/decision_engine.py` - `DecisionEngine.decide()`.
- `docs/current-findings.md` F-048 - the full resolution, including the byte-identical parity proof.
- `docs/topics/fusion-decision.md` - the always-synced topic doc.
- [Chapter 14 - Ultron Risk Gate](#) - the module that now solely owns `reward:risk` economics.

Unresolved questions

None - F-048 is an unusually well-documented, closed piece of history with its own parity proof already on record.

Previous: [Chapter 11 - Fusion](#) * **Next:** [Chapter 13 - The Execution Planner](#)
Related: [Chapter 14 - Ultron Risk Gate](#) * [Chapter 16 - Config-First Doctrine and the Promotion Path](#) (the governance discipline this fix exemplifies)
Memory: `docs/memory/engine-memory.md`.

Chapter 13 - The Execution Planner: From Decision to Order Geometry

Part V - Execution

Status of this chapter: Written

Why this chapter exists

An ACCEPT from the Decision Engine (Chapter 12) is still just an approval - it's not yet an order. Something has to turn "yes, trade this" into a concrete entry price, and classify *what kind* of trade this is. This chapter covers that step, and flags two live, unresolved issues that anyone touching this code should know about before assuming the geometry it produces is trustworthy.

What problem it solves

Explains what `ExecutionPlannerV1_2` actually computes - and, just as importantly, what it deliberately does *not* compute.

What you need to already know

[Chapter 12](#) - the planner only runs on an ACCEPT decision.

The idea

A classifier, not a geometry engine

`src/config_layer/execution_planner.py`'s `ExecutionPlannerV1_2.plan()` is, at its core, an intent classifier plus a gate. It sorts an accepted setup into one of four intents - BREAKOUT, PULLBACK, LIQ_SWEEP, REVERSAL - derives an entry price appropriate to that intent, and delegates final approval to `GateIntelligence`. **Stop-loss, take-profit, and reward:risk are explicitly not computed here.** That geometry comes from a different module entirely: CRT's own `compute crt levels` (inside `crt_engine_v2.py`), which owns the SL/TP math because it's the module that understands the structural levels (sweep point, displacement extent) the stop and target are actually measured against.

An open architectural caveat: which config the planner's geometry actually sees

There is a documented split-brain in how `CRTConfig` gets resolved on the *programmatic* path (tests, the tuner, embedders - anywhere `BacktestRunner(cfg)` is called with `crt_config=None`): in that case, `ConfigBuilder.build()` runs with no overrides, and the router's hardcoded FOREX/CRYPTO default profiles win - the production JSON config is never actually opened. Concretely, `expansion_atr_min_distance` ends up at 0.08 on this path versus 0.30 in the production JSON - a 3.75x difference on the very gate that governs the DISPLACEMENT->EXPANSION transition ([Chapter 8](#)'s state machine). The CLI entry point is unaffected - this is specifically a programmatic-construction gap. It's open, not yet fixed, and worth knowing about before trusting a programmatically-run backtest's structural-gate behavior at face value.

A second open caveat: SL geometry may be using the wrong units

A separate, still-unresolved issue concerns `compute crt levels` itself: it appears to treat the `atr` feature as if it were already in price units when computing the stop-loss buffer, but - per [Chapter 7](#)'s feature pipeline - `atr` in the canonical vector is close-relative, not an absolute price quantity. On XAUUSD this produces stop-loss buffers on the order of thousands of times too small (sub-cent stops on LIQ_SWEEP entries), and the same unit

confusion is implicated in unbounded position sizing downstream. This is a real, identified, **currently unfixed** issue awaiting an explicit user decision on remediation - it is not a hypothetical, and it is not something this book fixes as a side effect of documenting it.

Proposed remediation (Grok review pass - options only; no code in this book)

The review of this book correctly noted that flagging without a path feels passive. Below are **proposal options with trade-offs**. None of these is authorized by this chapter: authority still requires an explicit owner decision + construction protocol + parity proof ([Chapter 16](#) Authority Ladder).

ATR unit mismatch - proposed options

Option	What it does	Pros	Cons / risks
A. Multiply by close	Treat pipeline <code>atr</code> as relative; convert at use-site: <code>atr_price = atr_rel * close</code> inside <code>compute_crt_levels</code> (and any sibling buffer sites)	Surgical; matches the known closed form used elsewhere for FM-022/023-style defects; easy to parity-test	Must inventory every consumer of the same <code>atr</code> key in geometry; risk of double-scaling if any caller already converts
B. Pass absolute ATR separately	Geometry API takes <code>atr_abs</code> (from true-range SMA in price units) and never reads the relative feature for stops	Clean separation of feature vector vs geometry units; hard to misuse	New argument surface through CRT geometry callers; dual sources of ATR can drift if periods differ
C. Config-gated dual path	<code>geometry_atr_basis:</code> <code>relative_legacy \ </code> <code>absolute_price</code> with default = current behavior (byte-identical)	Matches F-061/F-066 pattern (strict require + default freeze); enables A/B shadow	Longer-lived dual code; still needs freeze-pin re-certification before activating on production

Recommended default for a future implement turn: C then A - land the conversion behind a strict config key with `relative_legacy` default (hash-neutral), prove ledger parity, then open a shadow config that flips to absolute and measures $\Delta G001$ before any promotion. Do **not** flip the active production config in the same turn as the wiring.

F-057 CRTConfig programmatic split-brain - proposed options

Option	What it does	Pros	Cons / risks
1. Always load ACTIVE_VERSION	When <code>crt_config</code> is <code>None</code> , <code>BacktestRunner</code> / <code>ConfigBuilder</code> loads production JSON via <code>get_prod_config()</code> instead of bare FOREX/CRYPTO profiles	Closes the 3.75x <code>expansion_atr_min_distance</code> gap; CLI and programmatic paths agree	Any test that silently relied on hardcoded profiles will change behavior - needs a test inventory
2. Fail-closed	Raise if <code>crt_config</code> is <code>None</code> on programmatic construction	Forces every caller to be explicit	Breaks tuner/tests until each injects config; noisier migration
3. Explicit profile enum	Require <code>profile="production" \ "router_default"</code> - no implicit default	Makes the choice auditable	API churn; still easy to pick the wrong profile

Recommended default for a future implement turn: 1 with a loud WARN log on the migration window, plus a focused regression that asserts programmatic build equals CLI-loaded production for the structural gates that F-057 names. Treat as backtest Phase-2 work (as the finding already scopes it) - not a silent one-line patch.

What this book still will not do

These remediations are **not implemented by documenting them**. Implementing either issue is a governed code change (construction protocol, tests, parity, owner approval on behavior change). Status remains: **OPEN / awaiting decision**.

Classification

Concept	Status
ExecutionPlannerV1_2.plan() intent classification	Production
SL/TP geometry (compute_crt_levels, lives in CRT engine)	Production, known defect, unfixed (ATR unit mismatch)
Programmatic CRTConfig resolution (non-CLI paths)	Production, known split-brain, OPEN (F-057)

Authoritative sources

- src/config_layer/execution_planner.py - ExecutionPlannerV1_2.plan().
- src/config_layer/crt_engine_v2.py - compute_crt_levels (the SL/TP/RR geometry, owned by CRT not the planner).
- docs/topics/execution-planning.md - the always-synced topic doc.
- docs/current-findings.md F-057 - the CRTConfig programmatic split-brain, in full.
- docs/topics/ai-automation-agent.md §GateIntelligence cross-references (if present) - the approval delegate.

Unresolved questions

- **The ATR unit mismatch in compute_crt_levels** - identified, scoped, proposed above (options A/B/C); still awaiting owner pick + implement authority. Rolled up in [A2](#).
- **F-057's CRTConfig split-brain** - status OPEN; proposed options 1/2/3 above; still "backtest Phase 2" not implemented as of this writing.

Previous: [Chapter 12 - The Decision Engine](#) * **Next:** [Chapter 14 - Ultron Risk Gate](#)
Related: [Chapter 08 - The CRT State Machine](#) (owns the SL/TP math this chapter describes)
Memory: [docs/memory/architecture-memory.md](#).

Chapter 14 - Ultron Risk Gate: the Final Capital Check

Part V - Execution

Status of this chapter: Written

Why this chapter exists

Everything before this point in the spine - ontology, features, four engines, fusion, a semantic decision, a planned entry - can be exactly right and the trade can still be wrong for the *portfolio*. This chapter covers the last, deterministic checkpoint before an order would leave the system, and the one module every other live-trading path in the repository is required to pass through.

What problem it solves

Separates "is this a valid opportunity" (Decision Engine, semantic) from "can we safely afford this trade right now" (Ultron, capital/portfolio). Chapter 12 already explained why that separation exists structurally (F-048); this chapter covers the module that resulted from it.

What you need to already know

[Chapter 12](#) - specifically, why reward:risk economics live here and not in the Decision Engine.

The idea

What it checks

`src/core/ultron_risk_gate.py`'s `UltronRiskGate.evaluate()` is the final capital-protection layer: position sizing, drawdown caps, exposure limits, daily-trade limits, and correlation thresholds across open positions. It is deterministic and hard-rejects on any breach - its own documentation is explicit that **no fallback path is permitted** here. Where the LLM gate in Fusion ([Chapter 11](#)) is allowed to fail open to a neutral score, Ultron is not allowed to fail open at all; a breach is a breach.

Where reward:risk economics actually live

Per [Chapter 12](#)'s account of F-048: real, cost-taxed reward:risk economics (`min_rr_ratio`, computed after the Execution Planner has determined actual SL/TP geometry) are owned solely here, not in the Decision Engine. This makes Ultron the single place in the spine where "is this trade good enough, economically, net of costs" is actually answered.

The one module INOUT touches

Of the four async "kitchen feeders" from [Chapter 2](#), INOUT (the parallel live-execution strategy rail, currently archived - see [Chapter 15](#)) is unusual: it doesn't feed the spine asynchronously the way Governance or Training do. It joins the *synchronous* spine directly, but only at this one point - `UltronRiskGate.evaluate()` - meaning whatever INOUT proposes still has to clear the exact same capital-safety check as the main CRT spine. It is explicitly not one of the four `EXPECTED_ENGINES`; it's a second decision source feeding the same final gate.

Classification

Concept	Status
<code>UltronRiskGate.evaluate()</code>	Production - the terminal approve/reject authority
Reward:risk ownership (<code>min_rr_ratio</code> , cost-taxed)	Production, here since F-048
Shared join-point with INOUT	Production mechanism; INOUT itself is archived (see Ch.15)

Authoritative sources

- `src/core/ultron_risk_gate.py` - `UltronRiskGate.evaluate()`.
- `src/core/_wrapper.py` - the wrapper referenced by the topic doc's entry-point trace.
- `docs/topics/ultron-risk-gate.md` - the always-synced topic doc.
- `docs/architecture/signal-flow.md` §2.4 - the INOUT join-point documentation.
- `docs/current-findings.md` F-048 - reward:risk ownership.

Unresolved questions

None for this chapter - Ultron's own scope is unusually crisply documented (a hard-reject-only module by explicit design), and its relationship to F-048 and to INOUT is already covered precisely by other chapters this one links to.

Previous: [Chapter 13 - The Execution Planner](#) * **Next:** [Chapter 15 - Live Execution and INOUT](#)

Related: [Chapter 12 - The Decision Engine](#)

Memory: `docs/memory/architecture-memory.md`.

Chapter 15 - Live Execution and INOUT

Part V - Execution

Status of this chapter: Written (INOUT ambiguity **resolved** in Grok review pass, 2026-08-07)

Why this chapter exists

Everything in Parts II-V so far describes the backtest path faithfully, and the live path *mostly* the same way - but "mostly" is doing real work in that sentence, and this chapter is where the book is most honest about the gap between the two. It closes Part V by describing what actually runs on a live tick, and by resolving a naming collision that earlier drafts left open: `src/inout/` vs `archive/inout_legacy/`.

What problem it solves

Distinguishes "what the backtest replays" from "what a live tick actually triggers," and is explicit about which of the two the rest of this book's Parts II-V chapters were describing (mostly the former, since it's the better-evidenced, more heavily tested path).

What you need to already know

[Chapter 14](#) - the one point where a live-execution rail rejoins the synchronous spine.

The idea

The live tick entry point

The confirmed entry point for a live candle is `HookedLiveEngine.process` in `src/runtime/live_engine_hook.py:582`. This is what actually gets called on a live tick, and it draws on `src/live/`, `src/inout/`, and `src/execution/loop.py` as supporting modules.

`src/execution/loop.py` - real code, no caller yet

`src/execution/loop.py` (plus `alert_manager.py`, `override_handler.py`) implements a multi-signal execution loop, but per the repository's own topic tracking it is explicitly **scaffolding - no caller yet**. It has tests (`test_execution_loop`) but isn't wired into a live path that actually invokes it.

INOUT naming collision - RESOLVED (Grok review pass, 2026-08-07)

Two paths share the word "inout" but are **not the same subsystem**. Direct comparison of both trees resolves the earlier open question without inventing a third story:

Path	What it actually is	Status
<code>archive/inout_legacy/ARCHIVED_2026-05-02</code>	Parallel execution pipeline that bypassed the canonical spine: <code>scanner.py</code> , <code>state_machine.py</code> , <code>controller.py</code> , <code>executor.py</code> , <code>runner.py</code> , <code>probability_engine.py</code> , plus a <code>SQLite db.py</code>	Legacy / Archived 2026-05-02 - README states it violated spine + no-DB conventions; no production <code>src/</code> importers at archival time

src/inout/	Market-data I/O only - candle/funding fetchers: <code>mt5_candle_fetcher.py</code> , <code>alphavantage_candle_fetcher.py</code> , <code>hummingbot_candle_fetcher.py</code> , <code>perp_funding_fetcher.py</code> (4 modules + package cache)	Production data-ingestion helpers - not a strategy rail, not a second decision path
------------	--	--

Verdict: `src/inout/` is **not** the successor of the archived INOUT rail. It is a **name reuse** for a different responsibility (fetch OHLCV / funding into CSV loaders). The archived rail's replacement for *execution* is stated in its own README: all valid live orchestration routes through `src/runtime/live_engine_hook.py -> HookedLiveEngine.process()`. CLAUDE.md's placement note that "live-mode code" may land under `src/inout/` is therefore best read as historical folder convention for I/O, not an invitation to revive the archived parallel spine.

`docs/architecture/goal.md`'s phrase "INOUT currently archived" correctly describes the **strategy rail** now under `archive/inout_legacy/`. It does **not** mean the current `src/inout/` fetchers are dead.

Classification

Concept	Status
<code>HookedLiveEngine.process</code> (live tick entry point)	Production
<code>src/execution/loop.py</code> (multi-signal execution loop)	Partial - built, tested, no live caller
INOUT strategy rail (<code>archive/inout_legacy/</code>)	Legacy / Archived (2026-05-02)
<code>src/inout/</code> data fetchers	Production (I/O) - distinct from the archived rail

Authoritative sources

- `src/runtime/live_engine_hook.py:582` - `HookedLiveEngine.process`.
- `src/execution/loop.py`, `alert_manager.py`, `override_handler.py`.
- `docs/topics/live-execution.md`, `docs/topics/execution-loop.md` - the always-synced topic docs.
- `docs/architecture/goal.md` §2 - "INOUT currently archived" (strategy rail, not fetchers).
- `archive/inout_legacy/ARCHIVED_2026_05_02/README.md` - archival reason + replacement path.
- `src/inout/*.py` - current fetcher modules (MT5 / Alpha Vantage / Hummingbot / perp funding).

Unresolved questions

- Whether residual `tests/inout/` still exercise the archived pipeline (the archive README suggested moving them to `tests/archive/`) was not re-audited this pass - hygiene only, not a live-path risk.
- `src/execution/loop.py` remains scaffolding with no live caller (unchanged).

Previous: [Chapter 14 - Ultron Risk Gate](#) * **Next:** [Chapter 16 - Config-First Doctrine and the Promotion Path](#)
Related: [Chapter 02 - The Invariants and the Happy Flow](#) (where INOUT was first introduced as a kitchen feeder)
Memory: `docs/memory/runtime-memory.md`.

Chapter 16 - Config-First Doctrine and the Promotion Path

Part VI - Governance

Status of this chapter: Written

Why this chapter exists

Parts II-V described the engine as it exists at one point in time. This chapter starts Part VI by explaining how that engine is meant to *change over time* - through configuration, under a formal gate - rather than through direct code edits chasing whatever the latest backtest suggested.

What problem it solves

Answers two related questions: what's allowed to be a config knob versus what has to stay in code, and what a config has to prove before it's allowed to become the one running in production.

What you need to already know

[Chapter 2](#)'s invariant #4 (no config reaches production without governance) - this chapter is that invariant's full mechanism.

The idea

Config-First: the target end-state

The doctrine's stated target is "**Frozen Engine + Mutable Behavior + Governed Evolution.**" Code changes should become rare; config changes should become routine; and the system should improve by *generating configs* - sweeps, tuning, promotion - rather than by rewriting engines. Every constant in the codebase is meant to be classified into one of three tiers before it's touched:

Class	Examples	What happens to it
STRUCTURAL	interfaces, enum members, the CRT transition graph, execution order, vector dimensions	Stays frozen in code - changing it is expected to be rare and expensive
BEHAVIORAL	thresholds, weights, percentiles, clamps, tier boundaries	Externalized into a config section
GOAL-SEEKING	candidate-config generation, sweeps, promotion itself	Generated as config; the engine underneath stays untouched

A knob climbs a maturity ladder - `HARD_CODED` -> `CONFIG_WIRED` -> `CONFIG_DRIVEN` - with goal-seeking (automated search) as an orthogonal, optional further step, not a requirement for every knob.

The hard rule: no silent defaults

New behavioral knobs must be introduced through a strict `_require()` boundary - a missing config key is an **error**, never a soft `.get(key, some_literal)` fallback. This closes a specific, previously-real failure class where an active production config quietly lagged the code schema and ran on undocumented defaults without anyone noticing. Every migration to config additionally needs a parity proof: the new JSON value must equal the prior hardcoded literal, read strictly, producing a byte-identical backtest ledger - any drift means the JSON default was wrong, and the determinism gate catches it.

The Authority Ladder - tunable is not the same as trusted

A knob becoming config-driven grants **tunability**, never **authority**. The doctrine draws four distinct rungs, and conflating them is exactly the mistake this ladder exists to prevent:

```
Information exists -> Economic usefulness exists -> Authority earned -> Architecture justified
```

A statistically detectable signal is not automatically economically useful; an economically useful signal doesn't automatically earn production/fusion weight; and even a successful consumer of a signal doesn't automatically justify adding more architectural complexity around it. Only a *demonstrated* improvement to the system's own economic goal metric (G001, [Chapter 1](#)) earns a rung. This is the same discipline behind [Chapter 9](#)'s Interpreter Contract and [Chapter 19](#)'s research falsification programs, stated here at the level of a general rule.

The promotion path itself

A new config's journey to production is a fixed pipeline:

```
AutoTuner -> results/tuner/checkpoint_multi.json -> ConfigValidator.validate(params, csv_paths, config_id) - runs a per-instrument backtest against quality gates: HARD: min trade count, max drawdown SOFT: win rate, expectancy, cross-instrument variance -> ValidationReport{decision: APPROVE | REJECT} REJECT -> results/validation/rejected/ APPROVE -> results/validation/approved/ -> PromotionManager.promote_from_report() - computes a SHA-256 config hash - archives the currently-active config - writes the new configs/production/{version}.json - appends an immutable line to configs/promotion_log.jsonl
```

Promotion refuses outright on a non-APPROVE decision, a duplicate version name, a hash-computation failure, or a write failure - and a write failure triggers an automatic rollback plus a PROMOTION_FAILED log entry, never a silent partial write. This is what makes configs/production/ACTIVE_VERSION (the Tier-0 runtime truth per CLAUDE.md §4.0) trustworthy: it can only ever point at something that passed this exact gate.

Classification

Concept	Status
Config-First Doctrine (classification tiers, maturity ladder, no-silent-defaults rule)	Production doctrine, actively enforced (behavior_census.py + test floor)
The Authority Ladder	Production doctrine, cited throughout Parts VI-VII
PromotionManager / promotion pipeline	Production
ConfigValidator.validate() hard/soft gates	Production

Authoritative sources

- CLAUDE.md §6.5 - the full Config-First Doctrine, including the Authority Ladder and the behavioral-vs-structural classification table.
- docs/reference/governance.md - the full PromotionManager API (promote_from_report, promote_from_tuner_checkpoint, promote_direct, list_versions, load_version, rollback procedure) - this chapter summarizes §1 of it; the rest is a direct reference, not duplicated here.
- src/governance/promotion_manager.py, src/config_layer/config_validator.py.
- scripts/analysis/behavior_census.py + tests/test_behavior_census.py - the enforcement contract pinning each migrated module's maturity so it can't silently drift back into code.

- `docs/topics/promotion-governance.md`, `docs/topics/config-validation.md` - the always-synced topic docs.

Unresolved questions

None - this is one of the most thoroughly, explicitly documented doctrines in the repository.

Encyclopedia: [E3 - Governance Tooling](#)

Previous: [Chapter 15 - Live Execution and INOUT](#) * **Next:** [Chapter 17 - Truth Maintenance](#)

Related: [Chapter 12 - The Decision Engine](#) (F-048 as a worked Config-First-adjacent example) * [Chapter 19 - The Research Programs](#) (the Authority Ladder applied to research findings)

Memory: `docs/memory/governance-memory.md`.

Chapter 17 - Truth Maintenance: Findings, Closure, and Authority

Part VI - Governance

Status of this chapter: Written

Why this chapter exists

This chapter explains the machinery behind a vocabulary this book has already used in nearly every chapter - "AUDITED," "CLOSED," confidence levels, F-0xx finding IDs. Rather than define these piecemeal, this is their one canonical home: how the repository decides what it believes, how confident it is, and how it prevents a claim from quietly rotting into a false one.

What problem it solves

A repository this size, worked on across years by multiple humans and multiple LLMs, faces a specific failure mode: the same fact stated three different ways in three different docs, one of which is now wrong, with no way to tell which. This chapter covers the doctrine built specifically to prevent that - "silent truth divergence."

What you need to already know

[Chapter 3](#) - this chapter fills in the classification vocabulary that chapter promised to explain in full.

The idea

The seven rules

The Repository Truth Maintenance Doctrine (CLAUDE.md §6.2) states seven rules, worth knowing as a set because they compound: (1) find the doc that already owns a topic before creating a new one; (2) classify every claim as `ALIGNED`, `DOC_DRIFT` (code wins, fix the doc), `CODE_DRIFT` (doc wins, fix the code), or `AMBIGUOUS`; (3) never silently resolve an `AMBIGUOUS` conflict - surface it and ask; (4) never delete disproven history - mark it `SUPERSEDED/INVALIDATED_BY` and keep the row; (5) minimize doc count; (6) synchronize related artifacts rather than editing in isolation; (7) state truth per-branch, never globally. [Chapter 7](#)'s 38-vs-39-dim finding is a live, worked example of rule 2 in action - this book classified it and flagged it rather than silently patching it, per rule 3's caution and this chapter's own scope discipline.

Findings: the unit of validated belief

`docs/current-findings.md` is where a validated or overturned conclusion gets recorded, the same turn it's discovered. Every finding carries a confidence level - `Certain`, `Likely`, or `Possible` - and cites its evidence. Findings are never deleted, only marked `SUPERSEDED` or `RETIRED` with the row kept intact, so the repository's history of what it *used* to believe stays legible even after that belief changes. This book's own chapters cite specific findings (F-005, F-028, F-036, F-038, F-039, F-041B, F-044, F-048, F-057, F-060, F-062, and others) exactly this way - as evidence, with the confidence and scope the finding itself declares, never inflated.

Epistemic Integrity - catching overclaims before they're registered

Before any finding is registered, a six-question pre-registration check runs: what artifact supports this (file:line required)? Could insufficient power explain the observation? Am I upgrading sign noise into meaning? Is this statistical or economic (a distinct authority tier)? Is the parent

claim stronger than its children? Would I phrase this differently after seeing raw counts? If any answer exposes uncertainty, the finding's confidence gets downgraded or its status set to HYPOTHESIS rather than accepted as-is. When this check catches a real overclaim, the required phrasing is explicit and disarming rather than defensive: *"Caught me overclaiming; I owe you a correction."* A pre-registration correction under this ritual is treated as evidence the discipline worked, not as a failure - Chapter 10's Gaussian-audit ablation and this book's own 38-vs-39-dim flag in Chapter 7 are both examples of the same instinct: report the uncertainty rather than round it away.

Correction is fixing the source, not just the chat. If a false claim is retracted, the same turn also has to fix every recorded instance of it - the finding's `Reversal` field, the doc, the memory file - marked `CORRECTED: <old> -> <new>`, never silently deleted.

Closure & Authority: what "CLOSED" and "AUDITED" actually mean

`docs/governance/closure_authority_index.json` tracks closure status per subsystem, governed by one non-negotiable invariant: **closure is boundary-scoped and non-transitive**. A `CLOSED` upstream subsystem does not make anything downstream of it `CLOSED` - only the specific artifact that declares closure for a *specific, named boundary* counts. Three distinctions worth memorizing, because conflating them is the exact mistake this index exists to prevent:

- AUDITED != CLOSED - reviewed and characterized is not the same as end-to-end proven.
- LINEAGE CLOSED != ECONOMICALLY VALIDATED - knowing a model loads the artifact it claims to is not the same as that artifact being worth anything.
- UPSTREAM CLOSED != DOWNSTREAM CLOSED - see the non-transitivity invariant above.

Concretely, in this book's own vocabulary (Chapter 3): CRT is `CLOSED` for its mechanism boundary only (Chapter 8); Gaussian, Zone Gate, RR, BitNet, and TradeNet are all `AUDITED` (Chapter 10, Chapter 11) - reviewed end-to-end, with clear, documented findings, but not certified as economically proven. A `CLOSED` or `AUDITED` surface reopens only under specific conditions: a governed dependency inside its declared boundary changes, a closure invariant fails, new contradictory evidence appears, or the artifact's own stated revalidation condition fires - never just because someone feels like revisiting it.

Classification

Concept	Status
Truth Maintenance Doctrine (7 rules)	Production doctrine, actively enforced (test-guarded)
Findings registry (<code>current-findings.md</code>)	Production, living document
Epistemic Integrity ritual (E-001)	Production doctrine
Closure & Authority Index	Production, machine-readable + test-guarded

Authoritative sources

- `CLAUDE.md` §6.2 (Truth Maintenance, the seven rules, the Documentation Drift Protocol, the Findings Mandate) and §6.5 (the Authority Ladder, cross-referenced from Chapter 16).
- `docs/current-findings.md` - the living findings registry.
- `docs/governance/EPISTEMIC_INTEGRITY.md` - the full six-question ritual.
- `docs/governance/closure_authority_index.json` - the machine-readable closure registry.
- `docs/governance/DOCUMENTATION_DRIFT_PROTOCOL.md` - the full drift-classification process.
- `tests/test_current_findings.py`, `tests/governance/test_epistemic_invariants.py`, `tests/test_closure_authority_index.py` - the enforcement floors.

Unresolved questions

None - this doctrine is unusually self-documenting; its own charter is the primary source and this chapter is a faithful compression of it.

Previous: [Chapter 16 - Config-First Doctrine and the Promotion Path](#) * **Next:** [Chapter 18 - A Field Guide to docs/governance/](#)

Related: [Chapter 03 - How This Book Fits](#) (the classification vocabulary this chapter grounds)

Memory: docs/memory/governance-memory.md.

Chapter 18 - A Field Guide to docs/governance/

Part VI - Governance

Status of this chapter: Orientation-level

Why this chapter exists

`docs/governance/` has roughly 250 files - by far the largest single documentation subtree in the repository. Chapters 16 and 17 already gave you the *doctrine* (Config-First, Truth Maintenance, Closure & Authority). This chapter is different in kind: it's a map of the directory itself, so that when you land in it - chasing a citation from another chapter, or investigating a specific subsystem's audit trail - you know which cluster you're in and what kind of document to expect.

What problem it solves

Without a map, `docs/governance/` reads as an undifferentiated pile of dated JSON+MD pairs. It isn't - it's several distinct evidence ledgers, each with its own internal logic, that happen to share a directory.

What you need to already know

[Chapter 17](#) - nearly everything in this directory is either a closure artifact, a lineage audit, or a certification record in that chapter's sense.

The idea

This is mostly an evidence ledger, not narrative documentation

The distinction from [Chapter 3](#) matters most here: most of `docs/governance/` is machine-oriented certification/evidence material - dated, often paired JSON+MD, produced by a specific audit or census run - not curated prose meant to be read start to finish. Treat this chapter as a directory of *drawers*, not a table of contents to read in order.

The clusters

- **Promotion / production lifecycle.** The mechanics are [Chapter 16's](#) territory; this directory holds supporting evidence like `IMPLEMENTATION_VALIDATION_V1.md`.
- **Closure reports.** End-of-thread declarations for a specific boundary - `crt_closure_report.md` ([Chapter 8's](#) primary source), `canonical_feature_code_surface_closure-2026-07-11.md`, various `feature_pipeline_*_closure_manifest` and `feature_surface_closure_audit` pairs.
- **Lineage audits.** Does the runtime actually load the artifact its registry claims to, and did the trainer actually produce it honestly - one file per model family:
`gaussian_lineage_audit.md`, `zonegate_lineage_audit.md`, `rr_lineage_audit.md`,
`bitnet_lineage_audit.md`, `tradenet_lineage_audit.md`, plus a cross-model `model_lineage_rollup.md`.
These are [Chapter 10](#) and [Chapter 11's](#) primary evidence.
- **Epistemic integrity / adjudication.** `EPISTEMIC_INTEGRITY.md` ([Chapter 17's](#) primary source) plus dated adjudication records where two or more authorities disagreed and the disagreement had to be formally resolved (e.g. `three_authority_drift_adjudication-2026-07-11.md`, `geometry_semantic_adjudication.jsonl`).
- **Ontology / market-ontology contracts.** `MARKET_ONTOLOGY_EVOLUTION_CONTRACT.md` ([Chapter 6's](#) primary source), plus formula- and geometry-specific contracts (`crt_formula_contract.md/json`, `geometry_family_registry.json`).
- **Measurement contract.** `MEASUREMENT_CONTRACT.md` and its schema - a frozen, not-yet-implemented program for making research claims formally comparable (see [Chapter 19](#)).

- **Feature-DAG / FM certification.** A large cluster tracking the certification status of individual feature-math (FM-0NN) nodes as they climb the ontology's refinement ladder ([Chapter 6](#)) - `feature_dag_layers-*`, `feature_certification_ledger.jsonl`, per-feature certification files.
- **CRT-specific census/diagnostic artifacts.** `crt_executable_surface.md/json`, `crt_executable_state_graph.md/json`, `CRT_CONFIG_CONSTRUCTION_PROTOCOL.md`, and dated funnel/fail-reason diagnostics - deep supporting evidence for [Chapter 8](#).
- **OHLCV corpus authority.** A dated 2026-07-10 cluster establishing exactly what the raw candle corpus is, its lineage, and its temporal semantics - `CORPUS_AUTHORITY.md` plus census/closure/contradiction/fingerprint artifacts. Supporting evidence for [Chapter 5](#).
- **MSIP (Market State Interpretation Platform).** See the dedicated brief below (Grok review pass).
- **Script inventory / authority.** `script_canonical_allowlist.json`, `MODEL_INTENT_AUTHORITY_REGISTER.md` - governs which scripts are recognized, registered surfaces (referenced by `CLAUDE.md` §3.1's SITS registration requirement).
- **Documentation governance itself.** `DOCUMENTATION_DRIFT_PROTOCOL.md` ([Chapter 17](#)'s primary source), plus `finding_dependency_audit.md` and `user-progress-registry.md`.
- **Ten packaged decision-bundle subdirectories** (`build_manifests/`, `crt_architecture_adjudication_v1/`, `msip_shadow_design_v1/`, `rr_l1_freeze/`, `xauusd crt_baseline_trace/`, and others) - self-contained evidence packages for a specific past decision, not meant to be browsed loose.

MSIP brief - Market State Interpretation Platform (Grok review pass)

MSIP is large enough that earlier drafts deferred it to A2. It does **not** replace CRT; it is a **shadow / observational** representation layer for continuous market state, designed to answer whether state vectors carry information *beyond* CRT lifecycle labels - without earning production cutover by existing.

Piece	Role
Design contract	<code>docs/governance/MSIP_SHADOW_DESIGN_CONTRACT_V1.md</code> (+ packaged bundle under <code>msip_shadow_design_v1/</code>)
Implementation plan	Shadow plan reviewed under G-PLAN-01; implementation only after explicit G-IMPL-01 owner OPEN
Code surface	<code>src/msip/</code> - <code>market_state_vector.py</code> , <code>interpretation_config.py</code> , <code>shadow_emitter.py</code> , <code>disagreement.py</code> , <code>isolation.py</code>
Shadow driver	Research scripts such as <code>run_msip_shadow.py</code> (observational; CRT path untouched when shadow off)
Hypotheses	H-MSIP-001 / H-017 (micro matched-subpopulation SUPPORTIVE, research-only); H-MSIP-002 / H-018 (MATCHING_DEPENDENT robustness)
Whole-representation prereg	MSIP_WHOLE series - design/prereg work has been parked pending owner un-park; no production authority

Status vocabulary (honest): infrastructure for shadow observation is **implemented**; several gates (e.g. G-SHADOW-01) have been declared on specified XAUUSD populations; **threshold change, migration, concurrency, and production cutover remain NOT_AUTHORIZED**. Findings from MSIP hypotheses are research-only micro effects or matching-dependent results - they do **not** reopen CRT's CLOSED mechanism boundary and do **not** grant fusion weight ([Chapter 16](#) Authority Ladder).

A full MSIP chapter (hypothesis ladder, gate names, evidence packages) remains optional future book work; this brief is enough to stop treating MSIP as an undocumented mystery.

How to actually use this directory

Don't browse it top-down. Arrive with a specific question - "is Gaussian's model artifact actually loaded correctly," "what does the CRT closure boundary formally cover," "why is `rr_fusion` disabled" - and use [Chapter 17](#)'s Closure & Authority Index or `docs/current-findings.md`'s Evidence: fields to find the specific file. This directory rewards targeted lookup, not linear reading.

Classification

Concept	Status
<code>docs/governance/</code> as a whole	Production evidence ledger - actively maintained, test-guarded per-cluster
Individual clusters	Mixed - closure reports and lineage audits are authoritative for their narrow boundary; census/diagnostic artifacts are point-in-time evidence, not living truth

Authoritative sources

- The clusters listed above, by name, are the entry points - there is no single index file for this directory beyond `docs/current-findings.md`'s Evidence: cross-references and the Closure & Authority Index in [Chapter 17](#).

Unresolved questions

- MSIP full chapter (optional)** - the brief above covers design posture and authority bounds; a deeper chapter (every gate id, every H-MSIP run artifact) remains future book work only if MSIP re-enters active execution. Tracked in [A2](#).
- This chapter's cluster list is a first-pass grouping, not an exhaustive index of all ~250 files - a future deepening pass could build a proper searchable index if that turns out to be worth the maintenance cost (weighed against `CLAUDE.md` §6.2 rule 5's "minimize doc count").

Previous: [Chapter 17 - Truth Maintenance](#) * **Next:** [Chapter 19 - The Research Programs](#)

Related: [Chapter 06](#), [Chapter 08](#), [Chapter 10](#) (the chapters whose primary evidence lives in this directory)

Memory: `docs/memory/governance-memory.md`.

Chapter 19 - The Research Programs: Falsification as a Discipline

Part VII - Research

Status of this chapter: Written (research data table expanded Grok pass)

Why this chapter exists

Chapter 1 promised that pattern interpreters and, by extension, trading ideas in general are "measured, never asserted." This chapter is where that promise gets tested at scale: a numbered sequence of Research Programs, each pre-registered, each run to a decisive conclusion, and - almost without exception so far - each one closing as an honest null result. That's not a failure of the research effort. It's the effort working exactly as designed.

What problem it solves

Gives a reader the shape of the research history **with concrete result data** (not only slogans): what was tested, sample sizes / key metrics where registered, the verdict, and why "0 PROMOTE" across program after program is a *credible* finding rather than a broken research pipeline.

What you need to already know

Chapter 16's Authority Ladder - every program below is that ladder applied to a specific hypothesis: information existing is not the same as economic value, which is not the same as earned authority.

The idea

Research results at a glance (data table)

Primary authority for every row: docs/current-findings.md (findings F-0xx below). Shared measurement standard for economic arms unless noted: **intraday fixed exit model + 12 bps** round-trip cost, absolute expectancy gate (E must clear 0 OOS to PROMOTE), plus controls / permutations as each program pre-registered. All economic cells below are **research-only** - none of these results grants production fusion weight.

Prog	Question (one line)	Key data (registered)	Verdict	Finding
1	Any next-bar directional edge on crypto majors?	Toys: baseline_delta vs random ~0 (-0.076...+0.033); spine 5-13 trades/inst -> n<30; pooled n=30 E=-0.399	0 PROMOTE / KILLED	F-019
1b	Conditional pockets (sessionxvolxmom, h<=20)?	25/25 partitions BH-significant but IG~0.001-0.0036 bits; Stage-3 economics on 74 cells -> 0 pockets	ENTROPY_LEAD_NO ECON	F-020

1c	Spine RETEST selection skill beyond session?	Selected–rejected $\Delta E = +1.17R$ but SESSION=ALL rejects; ZONE 0/110; SCORE p=0.327	Session filter only	F-021
1d	Exit/cost grid rescue expectancy?	Universe n=53,629; every SLxTP cell E_oos<0; best –0.271; reality_gap +4.16R (entry info, not exit)	Exit is not the lever	F-025
2	Completed sweep->disp->retest > bare sweep?	Funnel retest n=47 (~1% completion); excursion_asym(h8)= –0.319 ; loses all 4 controls	INSUFFICIENT_POWER / FROZEN	F-026
3	H1/H4 rescue directional edge?	0 PROMOTE H1 and H4; H1 E_oos –0.136...–0.266; H4 expansion_breakout ~gross 0 then pooled –0.0009	CLOSED null	F-027
P&F	Point-and-figure double-top/bottom edge?	BNB n=8554, PF=0.528, E= –0.454 , WR=0.327; loses random_uniform $\Delta E = -0.039$	REJECT / FROZEN	F-028
4	Vol-regime LEVEL consumable directionally?	H_atr=0.885 (vol memory real); 0 REGIME_EXPLOITABLE ; toys n=3.7k-35k still E~–0.14...–0.35	Informative != money	F-030
4b	Markov P^H transition forecast?	0 REGIME_EXPLOITABLE / 15 scopes; toys REGIME_REDUNDANT vs level+lag	CLOSED redundant	F-043
4d	MTF compression->expansion (Stage1 info / Stage2 econ)?	Stage1 UNIVERSAL p~0.0005; Stage2 directional consumer 0 PROMOTE (crypto E –0.25...–0.60R)	Info yes / money no	F-040
5	Cross-sectional relative-value monetizable?	5 interpreters, n=722-8,758; momentum PF=0.335; 0 PROMOTE	KILLED (panel axis)	F-032
6	Carry/basis as signal on spot dispersion?	8 interpreters n=722-4,373; E $\in$ [–0.0032, –0.0016]; 0 PROMOTE	KILLED	F-033

6b	Funding cashflow harvest net of costs?	4 harvest_full REJECT n=104-729; pre-cost +1...+6 bps < ~24 bps RT cost	KILLED / STOP	F-034
FX	Program-1 null generalizes off crypto?	5 FX majors; toys n=7.7k-17k PF 0.02-0.10 E -1.5...-2.8R; 0 PROMOTE	Null generalizes	F-035
8	Weekly Mon-Fri liquidity-sweep ontology (FX)?	6 scopes; pooled n=512 E=-0.634R PF=0.457 ; 0 PROMOTE	CLOSED null	F-042
7	Open Interest	Named out-of-scope for P6; not run in findings ledger as of book sources	NOT RUN	-
9	M5 multi-TF expansion + OCO straddle	Pre-reg frozen; on-disk results/research/m5_mtf/ has resample_parity PASS only; Stage-1/2 verdicts not sealed into findings	PRE-REGISTERED	H-016 / prereg

How to read the table: "0 PROMOTE" means *no cell cleared the fixed absolute-expectancy bar under the pre-registered cost/exit model* - not "the research code failed." Several rows are statistically informative (entropy, vol memory, Stage-1 transition IG) yet economically dead for spot directional (or carry) constructions. That split is the point of the Authority Ladder.

Where the raw artifacts live (examples; full paths in each finding's Evidence field):

Family	Typical artifact roots
Program 1 qualify	results/research/qualification/
Conditional entropy	results/research/phase_b/
Selection effect	results/research/phase_s/
Exit grid	results/research/phase_d/
Structural asymmetry	results/research/phase_e/
HTF qualify	results/research/qualification_htf/
Regime / transition	results/research/regime/, results/research/candle_state/
Cross-section / carry	results/research/cross_sectional_*.json, carry_*.json, harvest_*.json
Weekly sweep	results/research/weekly_sweep/
Program 9 M5	results/research/m5_mtf/

The gate every program runs through

Nearly every program in this ledger is tested against the same standard: a pre-registered hypothesis, an intrabar-realistic exit model with real transaction costs (not close-only fills), and a decision rule that requires *absolute* expectancy above zero out-of-sample - not just beating a control. A program is only allowed to "PROMOTE" a finding into production authority if it clears that bar; anything else is REJECT, INSUFFICIENT_POWER, or REDUNDANT (statistically real but fully explained by something already known).

The ladder, program by program (narrative)

- **Program 1 - Next-Bar Directional Ontology.** The foundational sweep: is there *any* directional edge in next-bar entries across crypto majors, at any conditioning level (session, volatility, momentum)? Ran through qualification, conditional-entropy pocket search, a selection-effect isolation, a full trade-anatomy audit, and an exit/cost grid - every stage closed null. **CLOSED / KILLED.** This is the foundation the rest of the ladder builds on.
- **Program 2 - Structural Asymmetry.** Does a *completed* sweep->displacement->retest sequence add forward asymmetry beyond a bare sweep? One experiment, a negative point estimate, badly underpowered (n~47, roughly 1% funnel completion). **FROZEN after one experiment** - explicitly not run further until power improves.
- **Program 3 - Higher-Timeframe Directional Ontology.** Does moving to H1/H4 rescue the directional edge Program 1 killed at M15? No - zero promotions at either timeframe. **CLOSED.**
- **Program 4 / 4b - Volatility Regime and Transition.** Does contemporaneous volatility-regime *level*, or a forward Markov *transition* forecast, offer a consumable directional edge? Both informative (volatility genuinely has memory) but neither consumable - the transition-forecast variant (4b) turned out to be statistically redundant with plain current-volatility level, adding nothing beyond what was already known. **CLOSED**, both variants.
- **Program 5 - Cross-Sectional Dispersion.** The first program to leave the per-instrument frame entirely and ask whether relative-value/market-neutral positioning across instruments is monetizable. All five interpreters tested REJECT, well-powered. **KILLED.**
- **Program 6 / 6b - Carry / Basis and Carry Harvest.** Is funding-rate carry a usable signal (6), or - separately - is the raw cash-and-carry payoff itself harvestable net of costs (6b)? Both null: all eight carry/basis interpreters REJECT, and all four harvest constructions fail to clear their own turnover cost even before considering price drag. **Both KILLED**, with an explicit stop called after 6b rather than continuing to iterate on cost assumptions.
- **Program 7 - Open Interest.** Named as explicitly out of scope for Program 6 (a distinct payoff), but not yet run as of the evidence this chapter draws on.
- **Program 8 - Weekly Liquidity-Sweep Ontology.** A genuinely new calendar-locked geometry (weekly accumulation/sweep/reversal, distinct from Program 1/2's local-structural frame) tested on FX majors. Zero promotions across all six scopes tested. **CLOSED.**
- **Program 9 - M5-base Multi-TF Expansion Forecasting (Non-Directional Ontology).** See dedicated section below (Grok review pass). **Pre-registered; research authority only.**
- **A cross-asset generalization check (not itself numbered as a Program):** does Program 1's crypto-majors null generalize to FX majors? Yes - the same entry-information null replicated on five FX majors with well-powered, decisively negative toy results.

Program 9 in depth (Grok review pass - closes the coverage gap)

Earlier book drafts only noted Program 9 from commit history. The frozen pre-registration is authoritative: docs/research/preregistration-program-9.md (../research/preregistration-program-9.md).

Why it exists. Program 4 (F-040) found expansion forecasting **informative but not consumable** by a spot *directional* consumer. Its reopen clause allows only new data, a new domain, or a new

non-directional ontology - never parameter archaeology. Program 9 exercises two reopen keys:

- 1. **New data:** strict M5 corpus (Binance crypto ~2y; MT5 FX ~270d) that did not exist when F-040 registered ("no M5 on disk").
- 2. **New ontology:** a both-sided stop-entry **OCO straddle** at compression-box edges (long-volatility payoff), which spot long/short cannot express.

Two-stage gate (same Authority Ladder as every other program):

Stage	Question	Authority level
Stage 1	Does the M5^M15^H1^H4 conjunction carry <i>incremental</i> information beyond the M15-base key at the same instant? (Gates A raw IG + B within-K15-cell incremental)	Level 1 - information only
Stage 2	Sole consumer <code>compression_box_straddle</code> through the unchanged M4 QualificationGate	Level 2 - economic

Stage 2 runs **only if** Stage 1 clears. **Gate A pass + Gate B fail** $\Rightarrow$ **M15_REDUNDANT = Stage-1 FAIL** (F-043's "real but already-known" lesson, renamed for this program).

Frozen construction highlights (D1-D7): reject-bar both-edges cancel (no invented touch order); no fill-bar TP credit; unfilled straddles excluded from n; first-compression-bar gating only; chronological 50/50 stability split; sole-consumer rule - no straddle-v2 archaeology inside Program 9.

Closure rule: Program 9 is **CLOSED** once hypotheses 9a (vol expansion), 9b (range expansion), and 9c (compression->expansion transition) each resolve to Stage-1 FAIL (incl. **M15_REDUNDANT**) **or** Stage-1 PASS + Stage-2 sole-consumer verdict - under the fixed thresholds. No within-program iteration.

Status at book time: pre-registration is frozen and present in-repo; hypothesis registry seed tracks H-016 as open/pre-reg. This book does **not** invent Stage-1/Stage-2 numeric results - verify artifacts under `results/research/m5_mtf/` (if present) and `docs/current-findings.md` before claiming a PROMOTE/REJECT. Authority remains **research/docs only** until a sealed finding says otherwise.

Related implementation surfaces (orientation, not a full walk): `src/research/candle_state/` (M5 Stage-1 kernels), `src/research/hypotheses/compression_box_straddle.py` (Stage-2 consumer), `src/research/resample.py` (calendar OHLCV resampler shared with Program 3).

Why this is the right outcome, not a broken pipeline

Every one of these closures follows the same discipline as [Chapter 9](#)'s Interpreter Contract: pre-register the test, apply a fixed, honest cost/exit model, require *absolute* expectancy, and accept the answer even when it's negative. A research program that only ever produced PROMOTE verdicts on this codebase's own history would be the actual red flag - it would mean the bar wasn't being held. Programs 1 through 8 closing null, one after another, under the same rigorous standard, is what a functioning falsification discipline looks like from the outside.

Classification

Concept	Status
---------	--------

Programs 1, 2, 3, 4, 4b, 5, 6, 6b, 8	Research - all CLOSED/KILLED/FROZEN , null or insufficient-power results
Program 9 (M5 multi-TF non-directional straddle)	Research - PRE-REGISTERED ; Stage results not asserted here
Program 7 (Open Interest)	Named / out-of-scope for Program 6; not independently verified as run
FX cross-asset generalization of Program 1's null	Research - confirmed generalizes

Authoritative sources

- `docs/current-findings.md` - the Funding Ledger section, Programs 1 through 8, each with its own F-0xx findings and confidence levels - this chapter's primary source, summarized not reproduced.
- `docs/research/preregistration-program-9.md` - Program 9 frozen pre-registration (primary for P9).
- `docs/research-readiness/program-*-preregistration.md` - the pre-registration for each program (e.g. `program-4-nondirectional-preregistration.md`, `program-8-weekly-crt-sweep-preregistration.md`).
- `CLAUDE.md` §6.5 - the Authority Ladder this whole chapter is an application of.

Unresolved questions

- **Program 9 run artifacts** - whether Stage-1/Stage-2 JSON under `results/research/m5_mtf/` is present and sealed into a finding was not re-validated against a fresh run this pass; the *design* is now chaptered, the *verdict* must still come from artifacts + findings.
- Program 7 (Open Interest) is named but its current status (planned, in progress, or not started) wasn't independently verified this pass.

Previous: [Chapter 18 - A Field Guide to docs/governance/](#) * **Next:** [Chapter 20 - The Research Platform](#)

Related: [Chapter 09 - Interpreters and the Pattern Contract](#) (the same discipline, one interpreter at a time) * [Chapter 16 - Config-First Doctrine](#) (the Authority Ladder)

Memory: none dedicated - see the research-family memory entries indexed in `MEMORY.md` under this session's project files (e.g. cross-sectional, carry-basis, carry-harvest programs).

Chapter 20 - The Research Platform: src/research/

Part VII - Research

Status of this chapter: Written (platform map + artifact index expanded Grok pass)

Why this chapter exists

Chapter 19 covered the *findings* and **numeric results** the research effort produced. This chapter covers the *machinery* that produces them: `src/research/`, the single largest subpackage in the entire codebase (416 files - more than double the next largest), plus its companion pre-registration corpus in `docs/research-readiness/` and the on-disk result trees under `results/research/`.

What problem it solves

Explains why there's so much code here, what shape it takes, **where research configs and outputs live**, and - importantly - flags a real gap between the research platform's ambition and its current Measurement Contract implementation state.

What you need to already know

Chapter 19 - this chapter is that one's supporting infrastructure.
Read the **Research results at a glance** table there before this map.

The idea

Research data map (where things live on disk)

Layer	Path	What you find
Living conclusions	<code>docs/current-findings.md</code>	F-0xx rows with Evidence, Confidence, Status
Pre-registrations	<code>docs/research/preregistration-*.md</code> <code>docs/research-readiness/program-*.preregistration.md</code>	Frozen hypotheses before runs Per program / per-asset research knobs
Research configs	<code>configs/research/research_config*.json</code> <code>configs/research/experiments/</code>	Per program / per-asset research knobs
Measurement profiles	<code>configs/research/measurement_contract-*.mp.json</code>	Per program / per-asset research knobs
Code kernels	<code>src/research/**</code>	Drivers, adapters, controls, consumers
CLI drivers	<code>scripts/research/**</code>	Thin entry points that write artifacts
Run artifacts	<code>results/research/**</code>	JSON/JSONL ledgers (qualification, phase_, regime, m5_mtf, weekly_sweep, h_msip_, ...)
Family registry	<code>docs/governance/research_family_registry.json</code>	Discovery layer atlas; most cells Contract:UNKNOWN

Rule: a claim without a file under `results/research/` (or an Evidence path in `current-findings.md`) is not a research result - it is a hypothesis.

The shape of `src/research/`

Structured as ~17 subpackages plus loose top-level modules. Map of **what research data each cluster owns**:

Subpackage / module	Research data role	Tied programs
<code>adapters/</code>	Spine / signal sources for research harnesses	P1 spine arm
<code>candle_state/</code>	MTF conjunction, transition targets, M5 kernels	P4 / P9 Stage-1
<code>controls/</code>	Negative controls every economic arm must beat	All M4 programs
<code>hypotheses/</code>	Economic consumers (e.g. <code>compression_box_straddle</code>)	P9 Stage-2
<code>weekly_sweep/</code>	Calendar-locked weekly sweep geometry	P8
<code>clean_labels/</code>	Honest label re-derivation (<code>forward_walk</code> class)	RR / Zone audits
<code>cross_sectional.py</code>	Panel / relative-value kernel	P5 / P6 / P6b
<code>exit_grid.py</code> , <code>costs.py</code>	SL/TP grid + cost model	P1 exit falsification
<code>conditional_entropy_grid.py</code>	Entropy + economics grid	P1 conditional pockets
<code>resample.py</code>	Deterministic OHLCV resampler	P3 HTF, P9 MTF
<code>measurement/</code>	Measurement-contract scaffolding	MC path (OPEN)
<code>ic002_*</code> / <code>ic003_*</code>	Interpreter-candidate shape research	shape / sequence IC tracks
<code>zone_mapping/</code>	Zone assignment research	ZoneGate audits
<code>secondlow_v1/</code> , <code>path/</code> , <code>episodes/</code>	Path / episode / second-low experiments	specialized tracks
<code>synthetic/</code> , <code>envelope_offline/</code> , <code>model_runners/</code> , <code>forensics.py</code>	Synthetic data, offline envelope, model runners, forensics	supporting

Loose shared modules also include: `experiment_spec.py`, `goal_alignment.py`.

The Edge Research Platform (ERP)

`docs/research-readiness/` documents a broader design - the "Edge Research Platform" - covering how research should flow end-to-end: an information-class boundary, a phased plan, a "promise ladder" (explicitly the doctrine-side counterpart of the Authority Ladder from [Chapter 16](#)), and how multiple LLMs collaborate on research work (tying into [Chapter 23](#)). This directory also holds every individual program's pre-registration document (`program-N-*.preregistration.md`) and a series of individual hypothesis pre-registrations (`h-*` files) for smaller, single-hypothesis tests that don't rise to full-Program scale.

An honest gap: the Research Measurement Contract exists on paper, not yet in code

One specific piece is worth flagging precisely because it's easy to miss:
`docs/governance/MEASUREMENT_CONTRACT.md`
defines a frozen (as of 2026-07-10) schema meant to make every research claim's measurement basis - label derivation, cost model, gate mode, clock basis - explicit and comparable across experiments, backed by a 27-seed adversarial mutation test matrix. As of the most recent verification available to this book, **zero MC- * instances have been sealed and none of the 27 probes have been implemented** - every finding in `docs/current-findings.md` currently carries `Contract: UNKNOWN`. This does not invalidate the Program 1-8 findings in [Chapter 19](#) - they were run under their own program-specific rigor (pre-registration, honest costs, absolute-expectancy bar) - but it does mean the *cross-program comparability* layer the Measurement Contract is meant to provide doesn't exist yet. Anyone tempted to compare two programs' results side by side should treat that comparison as informal until this layer is built.

Proposed path to close the gap (Grok review - roadmap, not an implement claim)

The review correctly treats 0 sealed MC- * instances as a **research-infrastructure** gap, not a failure of Programs 1-8. A minimal credible path (still owner-gated; this book does not execute it):

Step	Deliverable	Why it matters
1. Seal one profile-bound MC	One MC- * instance bound to an existing measurement profile (<code>configs/research/measurement_contracts/*.v1.json</code> , MP- * namespace) for a <i>closed</i> program (e.g. Program 8 or F-040's Stage-2)	Proves the schema is fillable from real artifacts, not only from theory
2. Implement E-MT-00 + E-MT-01	First two probes of the 27-seed mutation matrix	Turns "frozen schema" into "adversarially checked schema"
3. Bind registry rows	Update <code>docs/governance/research_family_registry.json</code> objectxlayer cells from <code>Contract:UNKNOWN</code> -> sealed id for the programs just covered	Makes the atlas machine-queryable
4. Stop the bleed	New research programs refuse to register findings without an MC- * id (policy + optional test floor)	Prevents the UNKNOWN set from growing while history is backfilled
5. Backfill selectively	Only re-seal programs when a comparison claim is needed - do not require rewriting all of F-019...F-043 before any new work	Matches L0 sufficiency (\$10 of the contract): hygiene spend is bounded

What this does *not* claim: sealing contracts does not reverse any null result, does not grant fusion/production authority, and does not replace per-program pre-registration. It only makes "these two experiments measured the same thing" a checkable statement.

Reports that check the platform's own hygiene

`docs/research-readiness/` also holds a set of integrity reports that audit the research *infrastructure* itself rather than any one finding: `determinism-report.md` (is the same experiment reproducible), `metric-integrity-report.md`, `config-reachability-report.md`, `test-gap-report.md`, `behavior-census-report.md`. These exist so that a null result can be trusted as a genuine null, rather than an artifact of broken plumbing.

Classification

Concept	Status
src/research/ subpackages	Research infrastructure, actively used (Programs 1-8)
ERP design docs (docs/research-readiness/edge-research-platform-*)	Design doctrine, partially implemented
Research Measurement Contract	Frozen schema, unimplemented - 0/27 probes built, all findings Contract: UNKNOWN
Research-readiness integrity reports	Production tooling (self-audits the research platform)

Authoritative sources

- `src/research/` - the 17 subpackages and shared modules listed above.
- `docs/research-readiness/README.md` and its ~60 sibling files (ERP design series, program and hypothesis pre-registrations, integrity reports).
- `docs/governance/MEASUREMENT_CONTRACT.md` - the frozen, unimplemented contract schema.
- `docs/topics/research-measurement-contract.md` - the always-synced topic doc for this specific gap.

Unresolved questions

- **Sealed mc- * count** - re-verify against `docs/governance/research_family_registry.json` and `MEASUREMENT_CONTRACT.md` before claiming any seal; book snapshot remains `OPEN` / 0 sealed unless those artifacts say otherwise.
- This chapter did not individually verify every one of the 17 `src/research/` subpackages' current wiring status (research-only vs. touching any promotable artifact) - flagged in [A2](#).

Encyclopedia: [E2 - Research Utilities Index](#) (269-file program/driver map)

Previous: [Chapter 19 - The Research Programs](#) * **Next:** [Chapter 21 - The AI Automation Agent](#)

Related: [Chapter 23 - Multi-LLM Coordination](#) (the ERP's multi-LLM research-collaboration design)

Memory: the session memory entry on the research family registry + measurement profiles (title: "Research family registry + measurement profiles") records the Measurement Contract's frozen-since-2026-07-10, zero-implementation status - consult it via the assistant's memory index, not a repo-relative path (it lives outside this repository).

Chapter 21 - The AI Automation Agent

Part VIII - Agent Intelligence

Status of this chapter: Orientation-level

Why this chapter exists

Everything in Parts II-VII describes a system a human (or a script) drives directly. This chapter covers the layer that lets natural language drive it instead - a deterministic, auditable agent built specifically to avoid the failure mode of "the LLM improvised something." Keep this distinct from [Chapter 23](#): that chapter covers *humans and multiple LLMs collaborating on developing this repository*; this chapter covers *one in-repo agent operating the trading system itself* through a fixed set of tools.

What problem it solves

Explains how a natural-language command becomes a specific, bounded sequence of tool calls against the trading system - and why that sequence is looked up, not generated fresh, on every request.

What you need to already know

[Chapter 2](#) - the agent is one of the four async kitchen feeders, and per invariant #3 it can drive the spine (Pipeline mode) but never silently mutate a decision in flight (Copilot mode is explicitly read-only).

The idea

Five modes, each with its own tool surface

`src/agent/modes/` implements five distinct operating modes: **Pipeline** (drives the spine through a backtest loop - 6 tools), **Copilot** (taps the Fusion/Decision step read-only, cannot mutate - 7 tools, all read-only by construction), **Governance** (interacts with the promotion machinery from [Chapter 16](#) - 5 tools), **Findings**, and **Log Query**. Two additional tools are available cross-mode. This mode separation is itself a safety mechanism: a Copilot-mode session is structurally incapable of triggering a trade or a promotion, regardless of what it's asked to do, because the tools that could do that simply aren't in its registry.

Deterministic planning, not generated planning

This is the agent's central design decision, and it's worth dwelling on: when a natural-language command comes in, an `IntentRouter` classifies it (a regex-first pass, with an LLM fallback for ambiguous phrasing) into one of a fixed catalogue of intents. A `PlanCompiler` then looks that intent key up in `PLAN_REGISTRY` - a hardcoded dictionary in `src/agent/plan_compiler.py`, "the single source of truth for what steps each intent runs." **The LLM never generates the tool-call sequence itself.** It only helps identify *which* pre-written sequence applies. This is the same "LLM is advice, never a trigger" invariant from [Chapter 2](#), applied at the level of *what the agent is allowed to do* rather than just *what decision it can influence*.

An `Executor` then runs the compiled plan, gated by a confirm-step and a path-guard (preventing the agent from writing outside its authorized surface), tracked through `AgentState`.

Implementation depth sketch (Grok review pass - still orientation, not a full rewrite)

The review asked for more than a brochure on Part VIII. The following is the **load-bearing call chain** a reader should keep in mind; the complete tool tables remain in `docs/reference/agent-reference.md`.

```
User natural language -> IntentRouter.classify() # regex first, LLM fallback only for ambiguity ->
PlanCompiler.build(intent) # PLAN_REGISTRY[intent] -> ordered ToolSteps -> Executor.run(plan) #
confirm gate + path-guard + AgentState -> tool_registry handlers (mode-scoped) -> dual JSONL audit
(per-step + session intent summary)
```

Intents (order-of-magnitude, not a freeze of the catalogue): pipeline / copilot / governance / findings / log-query families - roughly a dozen named intents, each mapped to a **fixed** tool sequence. Adding a capability is a code change to `tool_registry.py` **and** a new or extended `PLAN_REGISTRY` entry - never "the model decided to call arbitrary tools."

REPL entry: `src/agent/cli.py` (and related agent package CLI wiring) is the human-facing loop; config knobs live under the production config's `agent` section (fail-fast load patterns apply).

What the agent cannot do by design: promote a config without the same `ConfigValidator` / `PromotionManager` path a human uses ([Chapter 16](#)); invent a new plan graph at runtime; write outside path-guard; run Copilot tools that mutate the spine.

Everything is logged, twice

Every agent action writes to `logs/agent_audit.jsonl` (a per-step record of exactly what tool ran with what arguments) and `logs/agent_intent_log.jsonl` (a session-level summary of what was asked and how it was classified). This gives the same replayability guarantee ([Chapter 2's](#) invariant #1) to agent-driven work that the core spine already has for candle-driven work.

Classification

Concept	Status
Five-mode structure (Pipeline/Copilot/Governance/Findings/Log Query)	Production
PLAN_REGISTRY deterministic lookup	Production - the core safety mechanism
IntentRouter (regex + LLM fallback)	Production
Dual audit logging	Production

Authoritative sources

- `docs/reference/agent-reference.md` - the complete reference: all tools per mode, the intent catalogue, the full `PLAN_REGISTRY` tables, the `Executor`'s confirm-gate and path-guard, audit log field semantics, and the procedure for adding a new tool. This chapter summarizes it; that document is the one to read for implementation detail.
- `src/agent/plan_compiler.py` - `PLAN_REGISTRY` (line ~43) and `PlanCompiler.build()`.
- `src/agent/intent_router.py`, `tool_registry.py`, `executor.py`, `state.py`, `audit.py`.
- `src/agent/modes/` - the five mode implementations.
- `docs/topics/ai-automation-agent.md` - the always-synced topic doc.

Unresolved questions

None for the architecture itself - `agent-reference.md` is a comprehensive, well-structured primary source. Whether every documented tool in the reference is currently reachable end-to-end (vs. scaffolded) wasn't independently re-verified this pass.

Previous: [Chapter 20 - The Research Platform](#) * **Next:** [Chapter 22 - The Control Plane](#)

Related: [Chapter 23 - Multi-LLM Coordination](#) (a different, human-orchestrated layer - don't conflate the two)

Memory: `docs/memory/agent-memory.md`.

Chapter 22 - The Control Plane

Part VIII - Agent Intelligence

Status of this chapter: Orientation-level

Why this chapter exists

The repository has no REST API by design ([Chapter 2](#)'s invariant

7 - no database, no broker, no cloud dependency extends to "no network service surface" as a

default posture). But someone still needs a single, discoverable catalogue of every command this system supports, and a place for a human-facing dashboard to hang off of. That's the control plane.

What problem it solves

Gives every CLI-invocable command in the system one canonical, machine-readable declaration - instead of a human needing to grep `scripts/` to discover what's runnable.

What you need to already know

[Chapter 21](#) - the AI Agent's tool registry and the control plane's command registry are siblings in spirit (both are declarative catalogues), though distinct in scope (the agent's tools operate the trading system; the control plane catalogues every runnable command).

The idea

A declarative catalogue, not an execution engine

`src/control_plane/registry.py` defines a `CommandSpec` for every registered command (its `ArgSpecs`, from `cp_types.py`, and an `argv` template) - a catalogue, not the execution logic itself. It also carries a `WORKFLOW_STAGE_ORDER` tuple laying out the canonical operational pipeline in six stages: **Data Prep -> Tuning -> Model Training -> Validation & Promotion -> Replay & Backtest -> Live Runner** - and a mapping from roughly 19 concrete command IDs (`data.prepare_data`, `tuning.auto_tuner_multi`, `governance.orchestrator`, `promotion.manager`, `backtest.v2`, `live.inout_runner`, and others) to the stage they belong to, plus a small set of quickstart usage notes per command. This is effectively the machine-readable backbone behind `docs/reference/cli-matrix.md`'s human-readable command catalog.

The localhost dashboard

A small HTTP surface (`server.py`, `dashboard_api.py`, `report_api.py`) exposes this registry to a browser-based UI at `localhost:8787`, picking up new registered commands automatically via `ControlPlaneAPI.commands_payload()` - adding a `CommandSpec` is enough to make a command appear in the UI, with no separate UI-side wiring needed. This surface is explicitly **localhost-only, with no authentication layer and no TLS** - it is not meant to be exposed beyond the machine it runs on, and should never be.

Supporting modules

`code_context_extractor.py`, `context_report.py`, and `dot_graph_context.py` build the contextual information the AI Agent's Copilot mode ([Chapter 21](#)) can surface to an LLM - dependency-graph context, code excerpts - while `jobs.py` and `monitors.py` handle running and observing long-lived background operations the control plane kicks off.

Classification

Concept	Status
CommandSpec registry (<code>registry.py</code> , <code>cp_types.py</code>)	Production
Localhost dashboard (<code>server.py</code> + UI)	Production, deliberately unauthenticated/local-only
Context-extraction support modules	Production, primarily in service of the AI Agent's Copilot mode

Authoritative sources

- `src/control_plane/registry.py`, `cp_types.py` - the `CommandSpec` catalogue, `WORKFLOW_STAGE_ORDER`, and stage/command mapping.
- `docs/reference/control-plane.md`, `docs/reference/cli-matrix.md` - the full reference and the auto-generated command catalog.
- `src/control_plane/server.py`, `dashboard_api.py`, `report_api.py`, `jobs.py`, `monitors.py`.
- `CLAUDE.md` §4 - the localhost-only, no-auth constraint, stated as a hard operational limit.

Unresolved questions

None for this pass - the control plane's scope (declarative catalogue + localhost dashboard) is narrow and was confirmed directly against source.

Previous: [Chapter 21 - The AI Automation Agent](#) * **Next:** [Chapter 23 - Multi-LLM Coordination](#)
Related: [Chapter 04 - Architecture at a Glance](#)
Memory: none dedicated - see `docs/memory/architecture-memory.md` for cross-package context if needed.

Chapter 23 - Multi-LLM Coordination

Part VIII - Agent Intelligence

Status of this chapter: Orientation-level

Why this chapter exists

This closes Part VIII with the one layer that isn't about the trading system at all - it's about *how this repository itself gets built*, when the work is spread across six different language models operated by hand by one human. Distinct from [Chapter 21](#): that was one in-repo agent operating the trading system; this is six external LLMs collaborating on developing the repository, with the user as the bridge between them.

What problem it solves

Six models drifting into six different mental models of the same codebase is worse than one model working alone. This protocol exists to keep them sharing one reality.

What you need to already know

Nothing new - this chapter is largely self-contained, describing a process layer rather than code that runs inside the trading system.

The idea

The pipeline, and who's never allowed to skip it

Six models, each with a fixed role and an explicit "never does" boundary, connected in sequence by the user manually copy-pasting a structured handoff block between chat windows - no model calls another directly:

```
User (Bridge/Decider) -> DeepSeek (Planner - decomposes the story into a plan) -> Gemini Think (Quant Analyst - math/stats/CRT validation; never optimizes code) -> Gemini Pro (Optimizer - turns analysis into efficient code; never re-derives math) -> ChatGPT (Interpreter/Architect - explains, builds a JSON task plan; never executes tasks) -> Claude (Executor - writes code, runs validation, logs; never adds unsolicited explanation) -> Reality (tests + findings - VALIDATED / REFUTED / INCONCLUSIVE) -> Grok-2 (Synthesizer - combines everything + Reality into a final answer) -> User decides - approve, reject, or loop
```

Code authority is exclusive; advice is shared

This is the load-bearing rule of the whole layer: **Claude is the sole actor that writes or edits code and runs execution in this repository.** The other five models may propose code, diffs, or commands - but those are inputs to Claude, never applied directly. Advice, by contrast, is explicitly bidirectional and shared: every model contributes non-binding recommendations, and - this is stated with equal weight - **no model's advice, including Claude's own, carries authority by itself.** The user weighs it; Reality (tests, findings) settles it. This grants no new authority over the trading system itself: every change Claude makes under this protocol still has to pass the same gates as any other change - the promotion pipeline ([Chapter 16](#)), the path-guard and confirm-gate ([Chapter 21](#)'s Executor pattern, reused here), and explicit user approval for anything irreversible.

GATHER / ENHANCE / IMPLEMENT

Every work cycle follows a three-phase shape: **GATHER** (map affected files and constraints before touching code), **ENHANCE** (sharpen the prompt going to the next model - context, exact output format, an explicit "if unsure, write # UNKNOWN: - never invent" guard, role+constraint pairing), **IMPLEMENT** (a SpecWriter -> parallel DiffGen/TestGen -> Auditor chain, ending in commit/PR/changelog writers). Work moves through six status codes - PENDING, IN_PROGRESS, BLOCKED, UNKNOWN, DONE, NEEDS_REVISION - tracked per cycle in a lightweight state object, with the *authoritative* record living in `multi_llm/turn_ledger.jsonl` and `HANDOFF.md`, not the convenience state object itself.

The infrastructure underneath

- `context/*.md` - the "Portable Mind." Generated, gitignored derived views regenerated by `python scripts/context/build_context.py`. Truth stays in the repo (`CLAUDE.md`, `docs/current-findings.md`, the queue); these files are never hand-edited.
- `multi_llm/build_queue.jsonl` - the single backlog feeding what each model works on next.
- `multi_llm/turn_ledger.jsonl` - every model turn, captured verbatim, indexed for rewind and digest tooling (`scripts/context/discussion.py`).
- **Two session logs**, distinguished by dimension, not chronology: `assistant_project.md` for codebase/engineering work (this is the log every chapter of this book, and every response in the session that produced it, is required to append to - see `CLAUDE.md` §6), and `llm_project_assistant.md` for the *operation* of the multi-LLM workflow itself (handoff cycles, cross-model coordination decisions).

Classification

Concept	Status
The 6-model role pipeline + handoff protocol	Production process (human-operated, not automated)
Code & Execution Authority rule (Claude-exclusive)	Production doctrine
<code>context/*.md</code> Portable Mind	Production tooling, generated/gitignored
<code>multi_llm/turn_ledger.jsonl</code> , <code>build_queue.jsonl</code>	Production, authoritative state

Authoritative sources

- `CLAUDE.md` §13 - the full pipeline, role cards, phase framework, status codes, state object shape, and the Code & Execution Authority rule, verbatim.
- `multi_llm/MULTI_LLM_PROTOCOL.md`, `multi_llm/README.md` - the canonical protocol spec.
- `multi_llm/roles/` - the five per-agent role files (`ROLE_CHATGPT.md`, `ROLE_CLAUDE.md`, `ROLE_DEEPSEEK.md`, `ROLE_GEMINI.md`, `ROLE_USER.md`).
- `src/multi_llm/` - the code layer: `context_pack.py`, `discussion.py`, `tokens.py`, `turn_ledger.py`.
- `ChatGpt workflow/OrchestartedPrompts/` - plain-text/markdown prompt-stage artifacts (the rest of that directory is largely `.docx` illustrative material, not an authoritative source).
- `tests/test_context_compiler.py`, `tests/test_handoff_state.py` - the enforcement floors.

Unresolved questions

None - `CLAUDE.md` §13 is an unusually complete, self-contained primary source for this entire layer.

Encyclopedia: [Chapter 24 - Repository Encyclopedia](#) (remaining-file plan)

Previous: [Chapter 22 - The Control Plane](#) * **Next:** [Chapter 24 - Repository Encyclopedia](#)

Related: [Chapter 21 - The AI Automation Agent](#) (a different, in-repo agent layer - don't conflate the two)

Memory: none dedicated for this session.

Chapter 24 - Repository Encyclopedia

Part IX - Toward Full Repository Understanding

Status of this chapter: Written (charter + remaining-file map, Grok pass 2026-08-07)

Also known as: the Repository Encyclopedia plan - how the Canonical Knowledge Book (architecture guide) becomes a file-level encyclopedia without treating every unread file as equally important.

Why this chapter exists

The Canonical Knowledge Book (Chapters 00-23 + A1-A2) explains the **production spine and governance story**. It was never meant to name every Python file. After the coverage report (`grok/Book_PDF_File_Coverage_Grok.xlsx`), the natural question is:

What about the remaining files? Are they dormant? Unimportant? Or just not chaptered yet?

This chapter answers that. The remaining files fall into **several categories**. They are **not all equally important**, and the coverage report already hints at that.

What problem it solves

Gives a durable plan for “100% repository understanding” that:

- 1. Preserves the book’s architecture spine (do not rewrite it into a flat file dump).
- 2. Prioritizes **core omissions** over scripts and dormant packages.
- 3. Defines what a true **Repository Encyclopedia** entry looks like per significant file.

What you need to already know

- [Chapter 00](#) - candle -> order spine.
- [Chapter 04](#) - package map + un-chaptered orientation table.
- [Appendix A2](#) - open book gaps.
- Coverage spreadsheet: `grok/Book_PDF_File_Coverage_Grok.xlsx` (regenerate via `python grok/book_file_coverage_report.py`).

The idea

Current book vs encyclopedia

Mode	What it optimizes for	File coverage style
Canonical Knowledge Book (Ch.00-23)	Architecture, invariants, production path, research discipline	Concepts + load-bearing files; one chapter can explain dozens of files without naming each
Repository Encyclopedia (this chapter’s target)	Every significant file: purpose, relationships, entry points, relevance	File- or micro-module entries, prioritized by group A->D below

The book was written around the **architecture**, not around every source file:

Feature Pipeline ↓ CRT ↓ Fusion ↓ Decision ↓ Execution ↓ Risk Gate

...instead of listing file A, file B, file C, That is why only on the order of ~31 src/ files are exactly cited (~7% of the 456-file src inventory), while loose package orientation reaches roughly 60% of src/ - the major concepts and production spine, not every helper.

If the long-term goal is “100% repository understanding”: treat the current book as roughly 60-70% of the knowledge (concept + spine depth), not because 60-70% of files were named, but because the major architectural concepts and production path were captured. The remaining work is the implementation surfaces under those concepts.

1. Not covered by the book (~403 files)

From the coverage report (exact NOT_IN_BOOK rows against the functionality Excels):

Area	Not in Book
src/	189
scripts/	214
Total	403

This does not mean they are useless. It only means the book never discussed them as named files. Many still sit under packages the book oriented (DIR_ORIENTED) - research (142 files), features, agent, engines, runtime - where the chapter explains the system without enumerating every module.

Rule: NOT_IN_BOOK != dead code. Code and active config win on “is this live?”

2. They naturally split into four groups

A. Core but omitted (highest priority)

These are the most important omissions: the book explains the architecture but not every implementation file in the spine packages.

Package	Inventory files	Exact cited	Not in book (exact)	Notes
config_layer/	27	7	20	CRT helpers, routers, scorers, builders beyond the named few
core/	20	4	16	Fusion/decision/risk are cited; many orchestration helpers not
control_plane/	11	2	9	Beyond registry.py / server.py
execution/	5	1	4	Beyond loop.py (alerts, overrides, ...)
engines/	13	6	0*	Most engines named; support modules often DIR-only

runtime/	10	2	0*	backtest_v2 / live hook oriented; internals not file-listed
features/	28	2	0*	Pipeline + schema cited; registry/math surface oriented

* “0 not-in-book” can still mean **DIR_ORIENTED** (package mentioned, individual files not deep-read).

Encyclopedia priority: future chapters or encyclopedia entries for remaining core implementation files, engine support, planner helpers, runtime internals.

Should become: future book chapters *or* encyclopedia micro-entries under Part IV-V packages.

B. Sidecar / support modules

These support the system but are not required to understand the trading spine on day one.

Examples from the coverage report (mostly 0 exact cites, full package unread at file level):

Package	Files	Role
bitnet/	23	Low-bit model surface (inert on active config when disabled)
utils/	17	Shared helpers
retrieval/	9	Retrieval helpers
training/	8	Training pipelines
expansion/	6	Parameter expansion / search
portfolio/	6	Portfolio allocation (built; spine still largely single-candle)
replay/	6	Replay harness support
analytics/	5	Analytics
regime/	4	Regime observers (research-adjacent)
events/	2	Event utilities
search/	1	Search helpers
msip/	6	Shadow market-state (oriented in Ch.18; not spine)
multi_llm/	5	Multi-LLM coordination runtime (Ch.23 orients protocol)

Encyclopedia priority: medium - document when a task touches them; do not block spine literacy.

C. Dormant / archived

The book intentionally deprioritized these because they are not part of the active production story (or live only as archive).

Package	Files	Why deprioritized
strategies/	18	S01-S10 wrappers; mostly scaffold / non-spine
scanner/	6	Sidecar opportunity scanning
journal/	6	Journal path; dormant class
data_ingestion/	6	Alternate ingest surface
llm_research/	5	LLM research experiments
uat/	4	UAT
cognitive/	2	Cognitive bus - sidecar (F-012 class)
feedback/	2	Feedback loop - dormant class
monitoring/	2	Monitoring sidecar
ui/	1	UI remnants
archive/inout_legacy/	(archived tree)	Parallel execution rail retired 2026-05-02 (Ch.15)

Encyclopedia priority: low - one-line “status: DORMANT / ARCHIVED + replacement path” is enough unless reopening.

D. Operational scripts (largest numeric gap)

214 scripts are NOT_IN_BOOK (of 335 in `scripts_business_functionality.xlsx`). Only ~4 scripts are exactly cited. The book focuses on **what the system is**, not how every utility script works.

Typical script categories (from the scripts functionality inventory + book intent):

- migrations and repair tools
- validation utilities
- report generators
- governance / census / certification runners
- one-off research runners (`scripts/research/**`)
- data preparation and fetch wrappers
- automation / maintenance

Encyclopedia priority: index by **category + entry point + when to run**, not a narrative chapter per script. Prefer linking `docs/reference/cli-matrix.md` and `docs/reference/script-matrix.md` (when present) as the machine index; encyclopedia entries add purpose and danger notes (write vs read-only).

3. Why only ~35 files named?

Because the book teaches **flow and purpose**:

Concept chapter	Explains without listing every file under...
Ch.07 Feature pipeline	src/features/ (28 files)
Ch.08 CRT	most of crt_engine_v2 surface
Ch.10 Engines	src/engines/ (13 files)
Ch.11-14 Decision / execution / risk	core spine modules
Ch.19-20 Research	src/research/ (142 files) as a map, not 142 essays

One chapter can explain the purpose of **dozens** of files without naming each one individually. That is a feature of an architecture guide - and a **gap** for an encyclopedia.

4. Target: true Repository Encyclopedia

Transform the knowledge book from an **architecture guide** into a **repository encyclopedia**, where essentially every significant file has a documented:

Field	Meaning
Purpose	Why the file exists (user intent / economic role if any)
Relationships	Imports / callers / callees (point at module-roles / code-map when generated)
Entry points	CLI, tests, agent tools, control-plane commands
Relevance	Group A/B/C/D + LIVE / SIDECAR / DORMANT / ARCHIVED
Authority	Research-only vs production-touching; config keys if any
Coverage	CITED / DIR_ORIENTED / NOT_IN_BOOK from the coverage report

Out of encyclopedia scope (still real files): pure __pycache__, generated binaries, bulk market CSV/XLSX data dumps, and duplicate archives - index existence only, do not narrate.

Recommended build order (priority)

Phase	Scope	Outcome	Status
E0	This charter + coverage Excel	Shared vocabulary A-D	DONE
E1	Group A spine packages (core, config_layer, runtime, execution, engines, live, inout, control_plane)	File-level purpose map for ~94 modules	DONE - encyclopedia/E1-spine-implementation
E1b	Deep src/features/ (pipeline + ontology registry, 28 modules)	Feature math authority map	DONE - encyclopedia/E1b-features-registry

E2	Research utilities under src/research/ (143) + scripts/research/ (126) - index, not per-file essays	Research encyclopedia layer	DONE - encyclopedia/E2-research-utilities.
E3	Governance tooling (src/governance/ 19 + scripts/governance/ 31 + scripts/analysis/ 115 + maintenance 9)	Ops + truth maintenance tools	DONE - encyclopedia/E3-governance-tooling.
E4	Group B sidecars (~100 files: bitnet, utils, training, portfolio, replay, analytics, expansion, regime, msip, multi_llm, retrieval, ...)	Support library map	DONE - encyclopedia/E4-sidecar-modules.md
E5	Group C dormant / archived (~69 src/ + archive/**)	Honest “do not reopen without intent”	DONE - encyclopedia/E5-dormant-modules.md
E6	Remaining operator scripts (~70: data/training/context/backtest/export/...)	Operator encyclopedia	DONE - encyclopedia/E6-remaining-scripts.m

E1 note: Architecture chapters still own deep narrative for CITED spine modules (EngineRunner, CRT, Fusion, ...). E1 fills the **omitted implementation files** so Group A is no longer a blank space on the coverage map.

Entry format (proposed template for future encyclopedia rows)

```
### `src/path/to/module.py` - **Group:** A | B | C | D - **Relevance:** LIVE | SIDECAR | DORMANT | ARCHIVED - **Purpose:** ... - **Entry points:** ... - **Related chapters:** ... - **Config keys:** ... | none - **Tests:** ... - **Do not confuse with:** ...
```

Machine twin (future, optional): append-only JSONL under docs/book/encyclopedia/ or a generated sheet sibling to Book_PDF_File_Coverage_Grok.xlsx - **generated from inventory + hand overlays**, never hand-edited mass dumps as sole truth.

5. Relationship to existing record systems

This encyclopedia does **not** replace:

System	Still owns
Code	Behavior
docs/topics/	Always-synced concept dossiers
docs/memory/*	Edit-time reading order
docs/current-findings.md	Validated conclusions
Functionality Excels	File inventory + summaries
Coverage Excel	Book citation status
CLI / script matrices	How to run X

The encyclopedia **joins** them: file -> purpose -> chapter -> tests -> run entry.

Classification

Concept	Status
Canonical Knowledge Book (Ch.00-23)	Production architecture guide
Remaining-file groups A-D	Charter - authoritative prioritization for future work
Full prioritized encyclopedia	E0-E6 + E1b + JSONL DONE
E1 spine implementation map	encyclopedia/E1-spine-implementation.md
E1b features / registry map	encyclopedia/E1b-features-registry.md (28 modules)
E2 research utilities index	encyclopedia/E2-research-utilities.md (269 files)
E3 governance tooling index	encyclopedia/E3-governance-tooling.md (~174 files)
E4 sidecar modules map	encyclopedia/E4-sidecar-modules.md (~100 files)
E5 dormant / archived map	encyclopedia/E5-dormant-modules.md (~69 + archive)
E6 remaining operator scripts	encyclopedia/E6-remaining-scripts.md (~70 scripts)
Machine JSONL twin	encyclopedia/encyclopedia_rows.jsonl (~813 rows; regenerate via <code>grok/build_encyclopedia_jsonl.py</code>)
Coverage report	Generated tooling (<code>grok/book_file_coverage_report.py</code>)

Authoritative sources

- `grok/Book_PDF_File_Coverage_Grok.xlsx` - CITED / DIR / NOT_IN_BOOK counts.
- `results/analysis/src_business_functionality.xlsx` - 456 src files.
- `scripts_business_functionality.xlsx` - 335 scripts.
- `docs/book/README.md` - book TOC.
- `docs/architecture/module-roles.generated.md` - one-line roles when regenerated.
- `docs/reference/cli-matrix.md` / script matrix - runnable surfaces.
- [Chapter 04](#), [Chapter 15](#), [A2](#).

Unresolved questions

- Whether encyclopedia entries live only as chapters, as a JSONL registry, or both (recommended: hand-written priority entries + generated inventory join).
- Whether Group D should be 100% prose or mostly matrix + danger tags (recommended: matrix-first).
- E1 Group A map is written; optional **E1b** for full `src/features/` file rows remains open.
- **E0-E6 + E1b + JSONL twin complete**. Optional later: richer per-row relationship graphs; agent query UI over JSONL.

Previous: [Chapter 23 - Multi-LLM Coordination](#) * **Next:** [Appendix A1 - Testing](#) (or continue encyclopedia E1)

Related: [Chapter 00 - Quick Start](#) * [Chapter 04 - Architecture at a Glance](#) * [A2 - Unresolved Questions](#)

Memory: `docs/memory/architecture-memory.md` before expanding spine package entries.

Repository Encyclopedia

Parent charter: [Chapter 24 - Repository Encyclopedia](#)

Coverage report: [grok/Book_PDF_File_Coverage_Grok.xlsx](#)

Purpose: File-level purpose, relationships, entry points, and relevance - complementing the architecture book (Ch.00-23), not replacing code.

Status by phase

Phase	Scope	Status	Document
E0	Charter + Groups A-D vocabulary	DONE	Ch.24
E1	Group A spine implementation (core, config_layer, runtime, execution, engines, live, inout, control_plane)	DONE (file map)	E1-spine-implementation.md
E1b	Deep src/features/pipeline + registry (28 modules)	DONE	E1b-features-registry.md
E2	Research utilities (src/research/ 143 + scripts/research/ 126)	DONE (index)	E2-research-utilities.md
E3	Governance tooling (src/governance + gov/analysis/maintenance scripts)	DONE (index)	E3-governance-tooling.md
E4	Group B sidecars (~100 files: bitnet, utils, training, portfolio, ...)	DONE	E4-sidecar-modules.md
E5	Group C dormant / archived (~69 files + archive/)	DONE	E5-dormant-modules.md
E6	Remaining operator scripts (~70 outside research/gov/analysis/maintenance)	DONE	E6-remaining-scripts.md
JSONL	Machine twin of all encyclopedia module rows	GENERATED	encyclopedia_rows.jsonl

Series status: E0-E6 + E1b + JSONL twin complete.

Machine twin (JSONL)

Item	Value
Artifact	encyclopedia_rows.jsonl (~813 rows)
Regenerate	<code>python grok/build_encyclopedia_jsonl.py</code>
Validate	<code>python grok/build_encyclopedia_jsonl.py --check</code>

Schema (per line)	id, path, kind, phase, group, relevance, purpose, package, classes, source_doc, book_status, bytes, generated_at
-------------------	--

Do not hand-edit the JSONL as sole truth - regenerate from the builder (mirrors GENERATED tier discipline).
 Enrichment lives in the markdown encyclopedias + builder overrides.

How to read an entry

Each significant file has:

Field	Meaning
Group	A (core) * B (sidecar) * C (dormant) * D (scripts)
Relevance	LIVE * SIDECAR * DORMANT * ARCHIVED * OBSERVE_ONLY * PARTIAL
Book	CITED * DIR_ORIENTED * NOT_IN_BOOK (from coverage report)
Purpose	Why it exists
Relationships	Who calls it / what it calls
Entry points	CLI / tests / agent / control plane
Chapter	Architecture chapter if any

Code always wins on conflict with this encyclopedia.

Inventories used

- results/analysis/src_business_functionality.xlsx
- scripts_business_functionality.xlsx
- grok/Book_PDF_File_Coverage_Grok.xlsx (regenerate: python grok/book_file_coverage_report.py)

Encyclopedia E1 - Spine Implementation (Group A)

Phase: E1
Status: DONE (file-level map; not line-by-line reverse engineering)
Date: 2026-08-07
Parent: [Chapter 24](#) * [Encyclopedia index](#)

Scope

Group A packages that implement or host the candle -> order spine and its immediate I/O / control surfaces:

Package	Files mapped	Architecture chapters
src/core/	20	Ch.11-14, Ch.02
src/config_layer/	28 (incl. rr/)	Ch.06-08, Ch.13, Ch.16
src/engines/	13	Ch.10
src/runtime/	10	Ch.05, Ch.15, Ch.02
src/execution/	5	Ch.15
src/live/	3	Ch.15
src/inout/	4	Ch.05, Ch.15
src/control_plane/	11	Ch.22
Total	~94	

Not in E1: full src/features/ registry depth (package oriented in Ch.06-07; file encyclopedia deferred to a follow-on spine-features pass if needed), src/research/ (E2), governance (E3).

Legend - Relevance

Tag	Meaning
LIVE	On or adjacent to production spine / promotion path
PARTIAL	Built + tested; wiring conditional or incomplete
SIDECAR	Optional / advisory / non-veto on default path
OBSERVE_ONLY	Telemetry or probes; must not change decisions
SHIM	Compatibility alias; prefer canonical module
INERT	Present but disabled / unwired on active config

1. `src/core/` - Decision spine orchestration

Package role: Orchestrate scoring -> fusion -> decision -> capital gate; hold shared types and optional controllers.

Already in the architecture book (CITED)

File	Classes	Purpose (short)	Chapter
<code>engine_runner.py</code>	<code>EngineRunner</code>	Orchestrates engines, fusion, decision path	Ch.02, Ch.10-12
<code>fusion_engine.py</code>	<code>FusionEngine</code> , ...	Multi-engine score fusion / completeness	Ch.11
<code>decision_engine.py</code>	<code>DecisionEngine</code>	Semantic accept/reject (no economic RR)	Ch.12
<code>ultron_risk_gate.py</code>	<code>UltronRiskGate</code>	Final capital / RR / exposure check	Ch.14

Encyclopedia entries - omitted / supporting

`src/core/__init__.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Package exports for the decision spine.
- **Relationships:** Imported by runtime, agents, tests.
- **Entry points:** Import surface only.
- **Chapter:** Ch.04 package map.

`src/core/types.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Canonical TypedDict / contract types for the execution spine (`EngineContext`, engine results, etc.).
- **Relationships:** Consumed by `EngineRunner`, engines, tests that pin shapes.
- **Entry points:** Imported across `src/core`, `src/engines`, tests.
- **Chapter:** Complements `docs/reference/schemas.md`.

`src/core/feature_store.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Canonical feature store - single source of truth for feature dictionaries used on the spine.
- **Relationships:** Fed by feature builders / pipeline; read by engines and gates.
- **Entry points:** `EngineRunner` / live hook paths; unit tests.
- **Chapter:** Ch.07 (vector) without naming this module.

`src/core/gate_intelligence.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Pure signal gate used by `ExecutionPlanner` (intent / geometry-adjacent approval), not the capital gate.
- **Relationships:** Called from `execution_planner.py`; uses CRT-level geometry helpers (`compute_crt_levels` chain).
- **Entry points:** Planner path after Decision ACCEPT; tests under planner/gate.
- **Chapter:** Ch.13 (planner) - this file is the missing named helper.

- **Note:** ATR unit issues in CRT levels surface here via geometry (Ch.13 open defect).

`src/core/model_registry.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Versioned model registry with promotion margin / GOV-style quality gates for model artifacts.
- **Relationships:** Training / promotion paths; loads model JSON under `models/`.
- **Entry points:** Training scripts, promotion flows; `tests` around registry.
- **Chapter:** Ch.16 (promotion) adjacent.

`src/core/collector.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** NOT_IN_BOOK
- **Purpose:** Append structured records to `logs/collector.jsonl` (telemetry for fusion shadow / analysis).
- **Relationships:** Written from EngineRunner / runtime; read by `runtime/analyze_fusion_shadow.py`.
- **Entry points:** Runtime with collector enabled; analysis scripts.
- **Chapter:** None (ops telemetry).

`src/core/acceptance_controller.py`

- **Group:** A * **Relevance:** SIDECAR / PARTIAL * **Book:** NOT_IN_BOOK
- **Purpose:** Adaptive acceptance control - modulates accept rates / thresholds over time.
- **Relationships:** Optional layer near fusion/decision; config-driven when wired.
- **Entry points:** Config sections if present; focused tests.
- **Chapter:** None - not required to understand baseline spine.

`src/core/convergence_controller.py`

- **Group:** A * **Relevance:** SIDECAR / PARTIAL * **Book:** NOT_IN_BOOK
- **Purpose:** Convergence layer for FusionEngine (stability of multi-engine agreement).
- **Relationships:** Fusion path when enabled.
- **Entry points:** Fusion config flags; tests.
- **Chapter:** Ch.11 optional depth.

`src/core/dynamic_threshold.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** NOT_IN_BOOK
- **Purpose:** Percentile-based adaptive thresholds (not fixed hardcodes).
- **Relationships:** Controllers / gates that need regime-aware cutoffs.
- **Entry points:** Config + unit tests.
- **Chapter:** None.

`src/core/hierarchical_meta_fusion.py`

- **Group:** A * **Relevance:** SIDECAR / INERT-ish * **Book:** NOT_IN_BOOK
- **Purpose:** Multi-layer hierarchical meta-fusion for capital allocation quality scoring (HMF).
- **Relationships:** Sidecar to main fusion; findings class F-012 (sidecar-only consumers).
- **Entry points:** Explicit HMF callers / research; not default spine.
- **Chapter:** Ch.04 dormant/sidecar class.

`src/core/regime_governor.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** NOT_IN_BOOK

- **Purpose:** Regime-aware governor (`UltronGovernor` naming in module) - regime context for sizing/gates.
- **Relationships:** Adjacent to Ultron / risk; not a substitute for `UltronRiskGate`.
- **Entry points:** Config-gated integrations; tests.
- **Chapter:** Ch.14 adjacent.

`src/core/signal_audit.py`

- **Group:** A * **Relevance:** OBSERVE_ONLY * **Book:** NOT_IN_BOOK
- **Purpose:** Per-bar signal audit + leak detection recorder.
- **Relationships:** Observes spine outputs; should not mutate decisions.
- **Entry points:** Audit-enabled runs; analysis.
- **Chapter:** None.

`src/core/signal_belief_tracker.py`

- **Group:** A * **Relevance:** PARTIAL * **Book:** NOT_IN_BOOK
- **Purpose:** Temporal belief accumulator that can gate `DecisionEngine` calls (multi-bar confidence).
- **Relationships:** Upstream of `DecisionEngine` when enabled.
- **Entry points:** Config + tests; may be inactive on default configs.
- **Chapter:** None.

`src/core/governance_mode.py`

- **Group:** A * **Relevance:** LIVE (LLM layer) * **Book:** NOT_IN_BOOK
- **Purpose:** `GOVERNANCE_MODE` switch hardening for the LLM layer (Trd-M5) - when LLM is advisory vs restricted.
- **Relationships:** LLM client / agent governance; not a trade veto by itself.
- **Entry points:** Config / env governance mode; agent paths.
- **Chapter:** Ch.21 / Ch.16 adjacent.

`src/core/backtest_port.py`

- **Group:** A * **Relevance:** LIVE (interface) * **Book:** NOT_IN_BOOK
- **Purpose:** Dependency-inversion port for backtest results (Trd-M4) - clean boundary between runtime and consumers.
- **Relationships:** Implemented/used by backtest harnesses and validators.
- **Entry points:** Backtest / config validator.
- **Chapter:** Ch.05 / Ch.16.

`src/core/ultron_risk_gate_wrapper.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Wrapper around `ultronRiskGate` for alternate call shapes / integration adapters.
- **Relationships:** Prefer `ultron_risk_gate.py` as canonical; wrapper for callers needing a facade.
- **Entry points:** Live / strategy adapters; tests.
- **Chapter:** Ch.14.

2. `src/config_layer/` - Config, CRT, planner, models, LLM transport

Package role: Production config load, CRT state machine, execution planner, goal schema, model path resolution, RR offline stack, optional LLM transport.

Already in the architecture book (CITED)

File	Purpose	Chapter
<code>crt_engine_v2.py</code>	Full CRT state machine	Ch.08
<code>execution_planner.py</code>	Intent classifier + gate	Ch.13
<code>config_validator.py</code>	Pre-promotion validation	Ch.16
<code>goal_schema.py</code> / <code>goal_validator.py</code>	Goal layer	Ch.01
<code>state_identity.py</code>	CRTState / transitions seed	Ch.08
<code>crt_gaussian_scorer.py</code>	CRT-side Gaussian scoring helper	Ch.10 adjacent

Encyclopedia entries

`src/config_layer/production_config.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Load active production config; `get_active_version()` / `get_prod_config()` - **Tier-0 runtime truth** (ACTIVE_VERSION).
- **Relationships:** Nearly every production path; fail-fast if version missing.
- **Entry points:** Import from engines, backtest, promotion, CLI.
- **Chapter:** CLAUDE.md §4.0; Ch.16.

`src/config_layer/config_builder.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Single source of truth for building `CRTConfig` (and override validation). Site of F-057 split-brain when programmatic path skips production JSON.
- **Relationships:** `market_router` profiles; `BacktestRunner` / CRT engine.
- **Entry points:** Backtest, tuner, tests; CLI path usually via production JSON.
- **Chapter:** Ch.13 (F-057), Ch.08.

`src/config_layer/market_router.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Market-type CRT profiles (crypto/forex) and routing into `ConfigBuilder` defaults.
- **Relationships:** Feeds `ConfigBuilder` when production overrides absent.
- **Entry points:** `ConfigBuilder`; instrument onboarding.
- **Chapter:** Ch.13 F-057 mechanism.

`src/config_layer/production_bundle.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Immutable executable production state as one object (bundle of config + identity members).
- **Relationships:** Promotion / runtime loaders that want a sealed snapshot.
- **Entry points:** Governance / production load paths.
- **Chapter:** Ch.16.

`src/config_layer/stack_version.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Composite identity over WHAT / HOW / WHO / EXECUTION stack versions.

- **Relationships:** Bundle / promotion identity checks.
- **Entry points:** Governance tooling.
- **Chapter:** Ch.16 adjacent.

`src/config_layer/model_paths.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** CODE layout authority for where model artifacts live on disk.
- **Relationships:** `model_resolver`, registries under `models/`.
- **Entry points:** Trainers, resolvers, EngineRunner model load.
- **Chapter:** Ch.10 / Ch.16.

`src/config_layer/model_resolver.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Resolve model family -> registry active entry -> artifact path (+ identity parity).
- **Relationships:** Zone / RR / Gaussian / BitNet loaders.
- **Entry points:** Engine construction; tests for registry parity (e.g. zone manifest).
- **Chapter:** Ch.10, F-041 class.

`src/config_layer/state_topology.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Immutable CRT state topology view (`VALID_TRANSITIONS` runtime topology).
- **Relationships:** Breaks import cycles vs full `crt_engine_v2`; used by tests and resolvers.
- **Entry points:** CRT tests (`test_crt_state_invariants`), topology consumers.
- **Chapter:** Ch.08.

`src/config_layer/state_contract.py` / `state_contract_loader.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Typed immutable CRT state contract + loader for declarative/YAML-backed contract surfaces.
- **Relationships:** Semantic/state resolver pipelines; CRT governance.
- **Entry points:** Contract tests; semantic pipeline.
- **Chapter:** Ch.08 / Ch.06 adjacent.

`src/config_layer/crt_config_provenance.py`

- **Group:** A * **Relevance:** OBSERVE_ONLY / LIVE-meta * **Book:** NOT_IN_BOOK
- **Purpose:** Track how a CRTConfig was constructed (construction mode) - observability for F-057 class bugs.
- **Relationships:** ConfigBuilder / BacktestRunner diagnostics.
- **Entry points:** Probes and governance checks.
- **Chapter:** Ch.13 F-057.

`src/config_layer/crt_sweep_taxonomy.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Pure-function geometric classification of CRT sweeps (taxonomy helpers for engine/research).
- **Relationships:** CRT engine and research classifiers.
- **Entry points:** CRT tests; research scripts.
- **Chapter:** Ch.08 adjacent.

`src/config_layer/llm_inference_client.py`

- **Group:** A * **Relevance:** SIDECAR (fail-open) * **Book:** NOT_IN_BOOK
- **Purpose:** LLM HTTP transport with timeout / failure disable; returns neutral scores after N failures.
- **Relationships:** Agent, optional gates; **not** hard dependency of trade path.
- **Entry points:** Agent, llama/LLM config sections.
- **Chapter:** CLAUDE.md constraints; Ch.21.

`src/config_layer/llm_scorer.py / llm_narrative.py / insight_reporter.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** NOT_IN_BOOK
- **Purpose:** Score computation, narrative generation, and insight reporting over LLM transport.
- **Relationships:** Optional engines (`llm_engine`), dashboards, reports.
- **Entry points:** Control-plane report APIs; optional engine path.
- **Chapter:** Ch.21 / Ch.22 adjacent.

`src/config_layer/rr/rr_dataset_builder.py`

- **Group:** A * **Relevance:** RESEARCH / LIVE-train * **Book:** NOT_IN_BOOK
- **Purpose:** Build RR training datasets from features + outcomes (label path historically F-022-sensitive).
- **Relationships:** `rr_pattern_miner`, training scripts.
- **Entry points:** `scripts/training/*`, research.
- **Chapter:** Ch.10 RR lineage; F-022/F-045.

`src/config_layer/rr/rr_pattern_miner.py`

- **Group:** A * **Relevance:** RESEARCH / ARTIFACT * **Book:** NOT_IN_BOOK
- **Purpose:** Train / pure-Python inference for RR pattern model (Mahalanobis-style confidence).
- **Relationships:** Produces `models/rr_model*.json`; F-044 gate mis-spec documented in findings.
- **Entry points:** Training scripts; offline probes.
- **Chapter:** Ch.10; F-044/F-038.

`src/config_layer/rr/rr_fusion.py`

- **Group:** A * **Relevance:** INERT on active (disabled) * **Book:** NOT_IN_BOOK
- **Purpose:** Advisory RR fusion layer; **disabled** in active production config (`rr_fusion.enabled: false`) so base RREngine flows unmutated.
- **Relationships:** EngineRunner only constructs when enabled.
- **Entry points:** Config flag; regression tests for disabled identity.
- **Chapter:** Ch.10-11; F-038/F-044.

`src/config_layer/rr/__init__.py`

- **Group:** A * **Relevance:** LIVE (package) * **Book:** NOT_IN_BOOK
- **Purpose:** RR subpackage marker / exports.
- **Chapter:** Ch.10.

`src/config_layer/__init__.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Package docstring / exports for config layer.
- **Chapter:** Ch.04.

3. `src/engines/` - Scoring engines + gates

Package role: Four mandatory engines (CRT, Gaussian, Zone Gate, RR) plus optional validators and meta layers.

Already CITED

`crt_engine.py`, `heuristic_gaussian_engine.py`, `ml_gaussian_engine.py`, `rr_engine.py`, `scoring_engine.py`, `zone_gate_engine.py` - see **Ch.10**.

Encyclopedia entries - support / optional

`src/engines/__init__.py`

- **Group:** A * **Relevance:** LIVE * **Book:** DIR_ORIENTED
- **Purpose:** Package exports for scoring engines.
- **Chapter:** Ch.10.

`src/engines/gaussian_engine.py`

- **Group:** A * **Relevance:** SHIM * **Book:** DIR_ORIENTED
- **Purpose:** Backward-compatibility shim; canonical implementations are heuristic / ML Gaussian modules.
- **Relationships:** Prefer `heuristic_gaussian_engine.py` / `ml_gaussian_engine.py`.
- **Chapter:** Ch.10.

`src/engines/live_engine.py`

- **Group:** A * **Relevance:** LIVE * **Book:** DIR_ORIENTED
- **Purpose:** Live trading decision helper + zone load path used by live hook (not the full HookedLiveEngine).
- **Relationships:** `live_engine_hook` subclasses / wraps live engine behaviors; zone registry load.
- **Entry points:** Live path; tests.
- **Chapter:** Ch.15.

`src/engines/zone_cluster_score.py`

- **Group:** A * **Relevance:** LIVE * **Book:** DIR_ORIENTED
- **Purpose:** Pure zone cluster scoring shared by EngineRunner and historical zone tooling.
- **Relationships:** Zone gate / `discover_zones` pipelines.
- **Chapter:** Ch.10 Zone Gate.

`src/engines/trap_validator_engine.py`

- **Group:** A * **Relevance:** PARTIAL * **Book:** DIR_ORIENTED
- **Purpose:** Data integrity / trap validation pre-gate (quality of setup, not fusion weight).
- **Relationships:** Optional pre-filter before or beside scoring.
- **Entry points:** Config when enabled; tests.
- **Chapter:** None dedicated.

`src/engines/llm_engine.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** DIR_ORIENTED
- **Purpose:** Parallel engine wrapper around `llm_score_safe()` - advisory, fail-open class.
- **Relationships:** `llm_scorer` / inference client.
- **Chapter:** Ch.21 constraints (LLM not hot-path hard dep).

`src/engines/tradenet_meta_engine.py`

- **Group:** A * **Relevance:** INERT / UNWIRED * **Book:** DIR_ORIENTED
- **Purpose:** TradeNet meta-cognition layer - fusion neural slot historically stubbed (F-005).
- **Relationships:** Not on active production path without TN qualification protocol.
- **Chapter:** Ch.10 / F-005; tradenet lineage audit.

4. src/runtime/ - Backtest, live hook, baselines, probes

Package role: Run the spine over historical or live candles; capture baselines; optional observation tools.

Already CITED

File	Purpose	Chapter
backtest_v2.py	Candle-by-candle backtest harness	Ch.05, Ch.02
live_engine_hook.py	HookedLiveEngine.process live tick entry	Ch.15

Encyclopedia entries

src/runtime/__init__.py

- **Group:** A * **Relevance:** LIVE * **Book:** DIR_ORIENTED
- **Purpose:** Package marker.
- **Chapter:** Ch.04.

src/runtime/baseline_capture.py

- **Group:** A * **Relevance:** LIVE (governance hygiene) * **Book:** DIR_ORIENTED
- **Purpose:** Capture reproducible baseline manifest (schema hash, models, config SHA).
- **Relationships:** Pre-training / pre-migration hygiene.
- **Entry points:** python src/runtime/baseline_capture.py --label ...
- **Chapter:** CLAUDE.md Phase 0; Ch.16.

src/runtime/unified_replay_harness.py

- **Group:** A * **Relevance:** LIVE (research/ops) * **Book:** DIR_ORIENTED
- **Purpose:** Run execution-truth backtest (v2) and BitNet gate modes on same dataset; emit comparable artifacts.
- **Relationships:** Backtest + BitNet shadow configs.
- **Entry points:** Research / replay CLIs; determinism tests.
- **Chapter:** Ch.05; F-055 class.

src/runtime/backtest_bitnet.py

- **Group:** A * **Relevance:** RESEARCH / SHADOW * **Book:** DIR_ORIENTED
- **Purpose:** Row-by-row backtest with BitNet as hard gatekeeper (shadow experiments).
- **Relationships:** BitNet package; shadow production configs.
- **Entry points:** Research scripts; F-055 shadow.
- **Chapter:** Ch.10 BitNet; F-004/F-055.

src/runtime/analyze_fusion_shadow.py

- **Group:** A * **Relevance:** OBSERVE_ONLY * **Book:** DIR_ORIENTED

- **Purpose:** Analyze EngineRunner fusion shadow telemetry from collector logs.
- **Relationships:** collector.py JSONL.
- **Entry points:** Offline analysis CLI.
- **Chapter:** None.

src/runtime/crt_baseline_trace.py

- **Group:** A * **Relevance:** OBSERVE_ONLY * **Book:** DIR_ORIENTED
- **Purpose:** Behavior-preserving CRT baseline observation / operand traces.
- **Relationships:** CRT engine diagnostics.
- **Entry points:** Forensic / baseline programs.
- **Chapter:** Ch.08 diagnostics.

src/runtime/crt_fail_reason_counters.py

- **Group:** A * **Relevance:** OBSERVE_ONLY * **Book:** DIR_ORIENTED
- **Purpose:** Count CRT try_* fail reasons - observation only.
- **Relationships:** CRT funnel diagnostics.
- **Entry points:** Governance / CRT census tools.
- **Chapter:** Ch.08 / governance CRT artifacts.

src/runtime/exit_model_band.py

- **Group:** A * **Relevance:** RESEARCH * **Book:** DIR_ORIENTED
- **Purpose:** Dual-bound exit-model reporting (trust-layer exit bands).
- **Relationships:** Research exit/cost studies (F-025 class).
- **Entry points:** Research scripts.
- **Chapter:** Ch.19 exit falsification.

5. src/execution/ - Multi-signal execution loop (partial)

Already CITED

loop.py - multi-signal execution loop; **built, tested, no live caller** (Ch.15).

Encyclopedia entries

src/execution/__init__.py

- **Group:** A * **Relevance:** PARTIAL * **Book:** NOT_IN_BOOK
- **Purpose:** Package marker.
- **Chapter:** Ch.15.

src/execution/alert_manager.py

- **Group:** A * **Relevance:** PARTIAL * **Book:** NOT_IN_BOOK
- **Purpose:** Alerts for execution loop events (human-visible notifications).
- **Relationships:** ExecutionLoop; may bridge to Telegram later.
- **Entry points:** Loop when wired; tests.
- **Chapter:** Ch.15.

src/execution/override_handler.py

- **Group:** A * **Relevance:** PARTIAL * **Book:** NOT_IN_BOOK
- **Purpose:** Human/system overrides against automated execution decisions.
- **Relationships:** Execution loop; safety rail.
- **Entry points:** Loop when wired; tests.
- **Chapter:** Ch.15.

`src/execution/execution_intent_v1_0.py`

- **Group:** A * **Relevance:** PARTIAL * **Book:** NOT_IN_BOOK
- **Purpose:** Human-in-the-loop execution intent state machine (v1.0 contract).
- **Relationships:** Distinct from CRT `ExecutionPlanner` intents; for loop-level intent lifecycle.
- **Entry points:** Execution loop / UAT; tests (`test_execution_*`).
- **Chapter:** Ch.15.

6. `src/live/` - Live broker / alert bridges

`src/live/__init__.py`

- **Group:** A * **Relevance:** LIVE * **Book:** DIR_ORIENTED
- **Purpose:** Package marker for live bridges.
- **Chapter:** Ch.15.

`src/live/mt5_bridge.py`

- **Group:** A * **Relevance:** LIVE * **Book:** DIR_ORIENTED
- **Purpose:** MetaTrader 5 order execution bridge (IPC to terminal).
- **Relationships:** Live path after Ultron approve; requires MT5 running.
- **Entry points:** Live engine / operator setup.
- **Chapter:** Ch.15; external MT5.

`src/live/telegram_bridge.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** DIR_ORIENTED
- **Purpose:** Telegram bot alerts for signals / status.
- **Relationships:** Live engine optional notifications (not capital authority).
- **Entry points:** Live config with bot token (secrets in `.env` - never commit).
- **Chapter:** Ch.15.

7. `src/inout/` - Market data I/O (not the archived strategy rail)

Resolved in Ch.15: fetchers only; archived parallel spine is `archive/inout_legacy/`.

`src/inout/mt5_candle_fetcher.py`

- **Group:** A * **Relevance:** LIVE * **Book:** DIR_ORIENTED
- **Purpose:** Fetch M15 (etc.) OHLCV from MT5 -> CSV for `CandleLoader`.
- **Entry points:** `scripts/data/fetch_candles_mt5.py`.
- **Chapter:** Ch.05, Ch.15.

`src/inout/alphavantage_candle_fetcher.py`

- **Group:** A * **Relevance:** LIVE (data) * **Book:** DIR_ORIENTED
- **Purpose:** Alpha Vantage FX intraday -> CSV (premium limits apply).
- **Chapter:** Ch.05.

`src/inout/hummingbot_candle_fetcher.py`

- **Group:** A * **Relevance:** LIVE (data) * **Book:** DIR_ORIENTED
- **Purpose:** Exchange OHLCV via hummingbot-supported venues -> CSV.
- **Chapter:** Ch.05.

`src/inout/perp_funding_fetcher.py`

- **Group:** A * **Relevance:** RESEARCH / LIVE-data * **Book:** DIR_ORIENTED
- **Purpose:** Binance USD-M perpetual funding/basis inputs for carry research (Programs 6/6b).
- **Chapter:** Ch.19 P6; Ch.05.

8. `src/control_plane/` - Localhost command catalog + dashboard

Already CITED

`registry.py` (CommandSpec catalog), `server.py` (stdlib HTTP localhost:8787) - **Ch.22.**

Encyclopedia entries

`src/control_plane/__init__.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Package exports for control plane.
- **Chapter:** Ch.22.

`src/control_plane/cp_types.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** ArgSpec, CommandSpec, RunRecord dataclasses for the catalog and job history.
- **Relationships:** `registry.py`, `jobs.py`, `server.py`.
- **Chapter:** Ch.22; schemas.

`src/control_plane/jobs.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** JobManager - queue and run control-plane commands as subprocesses.
- **Relationships:** Server API -> subprocess argv from CommandSpec.
- **Entry points:** Dashboard “run command”; localhost only.
- **Chapter:** Ch.22.

`src/control_plane/dashboard_api.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Read-only data API for the CRT trading dashboard UI.
- **Relationships:** Server routes; log/artifact readers.
- **Chapter:** Ch.22.

`src/control_plane/monitors.py`

- **Group:** A * **Relevance:** LIVE * **Book:** NOT_IN_BOOK
- **Purpose:** Dashboard monitor specs (jsonl-tail / json / regex / file-stat sources).
- **Relationships:** Dashboard live panels.
- **Chapter:** Ch.22.

`src/control_plane/report_api.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** NOT_IN_BOOK
- **Purpose:** Run history reports (Excel) + optional LLM analysis.
- **Relationships:** Job history; LLM client.
- **Chapter:** Ch.22 adjacent.

`src/control_plane/context_report.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** NOT_IN_BOOK
- **Purpose:** ContextReportAPI - architecture analyst pack for a run (LLM-assisted).
- **Relationships:** `code_context_extractor`, `dot_graph_context`.
- **Chapter:** Ch.22 / multi-LLM ops.

`src/control_plane/code_context_extractor.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** NOT_IN_BOOK
- **Purpose:** AST-based code extraction for context reports.
- **Chapter:** Ch.22.

`src/control_plane/dot_graph_context.py`

- **Group:** A * **Relevance:** SIDECAR * **Book:** NOT_IN_BOOK
- **Purpose:** Flow resolution + architectural graph context (`.dot`) for reports.
- **Relationships:** `graph.dot` / pyan outputs when present.
- **Chapter:** Ch.04 / Ch.22.

E1 coverage rollup

Package	Exact CITED (pre-E1 book)	E1 encyclopedia entries added	Residual risk
core	4	16 supporting modules mapped	Controllers may be config-off - verify before assuming LIVE
config_layer	7	21 modules mapped	F-057 / rr_fusion disabled still apply
engines	6	7 support modules mapped	TradeNet/LLM inert unless enabled
runtime	2	8 modules mapped	Observation tools must stay OBSERVE_ONLY
execution	1	4 modules mapped	Loop still unwired to live
live	0	3 modules mapped	Secrets never in docs
inout	0	4 fetchers mapped	!= archived strategy rail

control_plane	2	9 modules mapped	localhost-only, no auth
---------------	---	------------------	-------------------------

E1 exit criterion (charter): every Group A spine-package `.py` file has a purpose/relevance row above (or is a trivial empty `__init__` noted as package marker). **Met.**

Not claimed: full call-graph proof for every optional controller; economic authority for any sidecar.

Next

- **E2** - `src/research/` + research scripts index (category + driver map, not 142 essays).
- Optional **E1b** - `src/features/` file encyclopedia (registry package depth) if spine feature math is the next editing focus.

Related chapters: [00](#) * [08](#) * [10](#) * [11-15](#) * [16](#) * [22](#) * [24](#)

Encyclopedia E1b - Feature Pipeline & Registry (deep map)

Phase: E1b
Status: DONE (file-level map of `src/features/` - 28 modules)
Date: 2026-08-07
Parent: [Chapter 24](#) * [E1 spine](#) * [Encyclopedia index](#)
Architecture narrative: [Chapter 06](#) * [Chapter 07](#)

Scope

Surface	Count	Role
<code>src/features/*.py</code> (top-level)	23	Pipeline, schema, semantic layers, builders, monitors
<code>src/features/registry/</code>	5	Authoritative feature-math implementation (no eval)
Total	28	Group A feature surface (was DIR_ORIENTED under E1 only)

Authority model (immutable):

```
configs/formulas/market_ontology.yaml -> DECLARES meaning (WHAT) src/features/registry/* +  
candle_math/derived_math -> IMPLEMENTS math (HOW) src/features/feature_pipeline.py -> EMITS 39-dim  
vector (v4.0) src/features/feature_schema.py -> NAMES / pins CANONICAL_FEATURES
```

Never re-derive formula math in engines/scripts - use `registry / derived_math / pipeline` (F-047, ownership lint).

Book status: `feature_pipeline.py` + `feature_schema.py` **CITED** in Ch.07; all other modules were package-oriented until this E1b map.

Layer map (semantic pipeline)

Layer	Module(s)	Role
L0 OHLC identity	<code>candle_math.py</code> , <code>registry/primitive_registry.py</code>	Immutable geometry (body, wick, range, ratios)
L1 derived scalars	<code>derived_math.py</code> , <code>registry/derived_registry.py</code>	ATR-relative / normalized metrics (FM-0NN)
L1 compositions	<code>registry/composition_registry.py</code>	Named ratios over primitives
Registry facade	<code>formula_registry.py</code> , <code>registry/__init__.py</code> , <code>registry/_loader.py</code>	FORMULA_REGISTRY + ontology load
FM resolve	<code>fm_resolve.py</code>	FM-id -> callable (fail-closed)
Identity	<code>feature_identity.py</code>	<code>feature_id</code> collision / governed identities
Batch pipeline	<code>feature_pipeline.py</code>	Production 39-dim vector builder

Schema pin	<code>feature_schema.py</code> , <code>schema_validator.py</code>	CANONICAL_FEATURES / dim / fail-fast validate
Online builders	<code>feature_builder.py</code> , <code>crt_feature_builder.py</code>	Tick/row -> input_data / CRT->canonical dict
Structure / PIT	<code>causal_structure.py</code>	FC1-A delayed-confirmed swings (no lookahead)
Session / clock	<code>session_classifier.py</code> , <code>broker_clock.py</code>	Session ownership; MT5 UTC correction (F-066)
Semantic L2	<code>feature_states.py</code>	Feature values -> declared states
Semantic L4	<code>market_context.py</code>	States -> market context
Semantic L5	<code>market_shape.py</code>	Context -> recurring shapes
Semantic L5 CRT	<code>crt_state_resolver.py</code>	Feature states -> CRT 9-state resolve (parity with engine)
Semantic L7	<code>model_evidence.py</code>	Model testimony per bar
Contracts	<code>gaussian_schema_contract.py</code>	Gaussian NB schema under v4 migration
Dataset prep	<code>dataset_builder.py</code> , <code>dataset_validator.py</code>	Train/log feature extract + validate
Drift	<code>feature_monitor.py</code>	Rolling distribution / Z-drift on trades
Package	<code>__init__.py</code>	Exports

1. Math authority (immutable primitives + registry)

`src/features/candle_math.py`

- **Group:** A * **Relevance:** LIVE * **Book:** DIR_ORIENTED (E1b deep)
- **Purpose:** Single immutable source for candle geometry identities (body/wick/range/ratios). Mechanism not policy (F-046 class).
- **Relationships:** Used by pipeline, CRT, registry primitives; parity-tested.
- **Chapter:** Ch.06-07; F-046.

`src/features/derived_math.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Scalar implementations of deterministic normalized metrics (disp_strength/retest/... FM forms). Sibling of candle_math.
- **Relationships:** registry/derived_registry.py maps ontology impl names here (no eval).
- **Chapter:** Ch.06-07; F-047/F-050/F-063.

`src/features/registry/primitive_registry.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Maps ontology primitive impl names -> candle_math callables.
- **Chapter:** Ch.06.

`src/features/registry/composition_registry.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Trading-interpretation ratios (named callables over primitives).
- **Chapter:** Ch.06.

`src/features/registry/derived_registry.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Maps derived FM impl names -> `derived_math` callables + signature dispatch.
- **Chapter:** Ch.06.

`src/features/registry/_loader.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Leaf ontology YAML loader (lazy PyYAML); avoids import cycles.
- **Relationships:** Reads `configs/formulas/market_ontology.yaml`.
- **Chapter:** Ch.06.

`src/features/registry/__init__.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Authoritative aggregated feature-math registry API (`FORMULA_REGISTRY`, validation helpers).
- **Chapter:** Ch.06-07; F-047.

`src/features/formula_registry.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Back-compat facade re-exporting the registry package (do not re-implement math here).
- **Chapter:** Ch.06.

`src/features/fm_resolve.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Phase-2 FM resolution: FM-id -> callable via ontology + `FORMULA_REGISTRY` (Option A).
- **Entry points:** Certification tooling, consumers needing FM-id dispatch.
- **Chapter:** Ch.06; E3 `feature_dag` tools.

`src/features/feature_identity.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Fail-closed `feature_id` lookup; governs duplicate/collision families (phase-1 identity registry docs).
- **Chapter:** Ch.07 governance adjacent.

2. Production vector emission & schema

`src/features/feature_pipeline.py`

- **Group:** A * **Relevance:** LIVE * **Book:** CITED (Ch.07)
- **Purpose:** Production-grade M15 OHLCV -> deterministic vector of length `CANONICAL_FEATURE_DIM` (39, v4.0). Config-driven indicator periods (`feature_pipeline` section).
- **Relationships:** Backtest + live feature build; BitNet/Zone/CRT consumers of vector slots.
- **Chapter:** Ch.07; F-061/F-066 config gates (`normalization_basis`, `session_timestamp_basis`).

`src/features/feature_schema.py`

- **Group:** A * **Relevance:** LIVE * **Book:** CITED (Ch.07)
- **Purpose:** Canonical feature names, dim, schema objects, hash pins. Source of truth for 39-dim order.
- **Note:** Reference docs may lag (38 vs 39) - code wins.
- **Chapter:** Ch.07; F-062.

`src/features/schema_validator.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Strict fail-fast validator for the canonical feature contract.
- **Entry points:** Dataset/train preflight; tests.
- **Chapter:** Ch.07.

`src/features/gaussian_schema_contract.py`

- **Group:** A * **Relevance:** LIVE-contract
- **Purpose:** Gaussian NB feature-schema contract under SCHEMA-V4-VECTOR-MIGRATION (dim renames).
- **Chapter:** Ch.10 Gaussian; F-062 class.

3. Online / CRT builders

`src/features/feature_builder.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Constructs canonical `input_data` dict from raw tick/row; enforces mandatory OHLCV.
- **Relationships:** Live/engine paths needing lightweight build vs full batch pipeline.
- **Chapter:** Ch.07 adjacent.

`src/features/crt_feature_builder.py`

- **Group:** A * **Relevance:** LIVE / PARTIAL
- **Purpose:** CRT trade/candle/state -> dict with **exactly** all CANONICAL_FEATURES. Historical `body_ratio` outlier path audited (F-046 - prefer `candle_math`).
- **Chapter:** Ch.07-08.

`src/features/causal_structure.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** FC1-A delayed-confirmed structure publication helpers (PIT-safe swings). Batch authority via pipeline `structure/liquidity` columns.
- **Chapter:** Ch.07; F-051 class.

4. Session & broker clock

`src/features/session_classifier.py`

- **Group:** A * **Relevance:** LIVE
- **Purpose:** **Single owner** of “what session is this?” - deliberately separates feature session labels vs CRT filter windows (F-066 doctrine).
- **Chapter:** Ch.07; F-017/F-066.

`src/features/broker_clock.py`

- **Group:** A * **Relevance:** LIVE (config-gated)
- **Purpose:** Convert MT5 broker-server timestamps to true UTC when `session_timestamp_basis=utc_corrected`. Default `broker_local` is byte-identical legacy.
- **Relationships:** `mt5_candle_fetcher` labeling; `feature_pipeline` session path.
- **Chapter:** Ch.07; F-066.

5. Semantic stack (states -> context -> shape -> CRT resolve -> model evidence)

`src/features/feature_states.py`

- **Group:** A * **Relevance:** LIVE-semantic
- **Purpose:** Layer 2 - feature VALUES -> declared semantic STATES (`market_crt_states` / ontology `feature_states`).
- **Chapter:** Ch.06-08 semantic pipeline.

`src/features/market_context.py`

- **Group:** A * **Relevance:** LIVE-semantic
- **Purpose:** Layer 4 - combine states into full market context ("what is happening").
- **Chapter:** Ch.06.

`src/features/market_shape.py`

- **Group:** A * **Relevance:** LIVE-semantic
- **Purpose:** Layer 5 - assign recurring shape identity (`configs/formulas/market_shapes.yaml` class).
- **Chapter:** Ch.06 / Ch.09 adjacent.

`src/features/crt_state_resolver.py`

- **Group:** A * **Relevance:** LIVE-semantic / RESEARCH-parity
- **Purpose:** Resolve 9 CRT states from declared feature states (declarative path). Semantic parity with `crt_engine_v2` is incomplete by construction (F-069) - not a drop-in replacement for the engine.
- **Chapter:** Ch.08; F-069.

`src/features/model_evidence.py`

- **Group:** A * **Relevance:** LIVE-semantic
- **Purpose:** Layer 7 - record what each model says about the bar (testimony), not what the market is.
- **Chapter:** Ch.06 semantic stack.

6. Dataset, validation, drift

`src/features/dataset_builder.py`

- **Group:** A * **Relevance:** TRAIN/RESEARCH
- **Purpose:** Extract feature vectors / BitNet slices from fusion logs for training datasets.
- **Chapter:** Ch.07; training E4/E6.

`src/features/dataset_validator.py`

- **Group:** A * **Relevance:** TRAIN/RESEARCH
- **Purpose:** Validate fusion trade logs before dataset build or model training (entropy, balance, min-n).
- **Chapter:** Ch.16 training quality gates.

src/features/feature_monitor.py

- **Group:** A * **Relevance:** LIVE-ops
- **Purpose:** Rolling feature distribution + drift (Z-scores); wired on TRADE_OPENED in backtest (CLAUDE Phase 2).
- **Chapter:** Ch.07; FeatureMonitor in backtest_v2.

src/features/__init__.py

- **Group:** A * **Relevance:** LIVE
- **Purpose:** Package exports / marker.

Config keys (feature surface)

Config area	Examples	Effect
feature_pipeline	rsi_period, atr_period, ema_*, macd_*, normalization_basis, session_timestamp_basis	Periods + F-061/F-066 gates
Ontology YAML	configs/formulas/market_ontology.yaml	Entity identities / lifecycle
Shapes YAML	configs/formulas/market_shapes.yaml	Shape declarations
CRT states YAML	market_crt_states (semantic)	State predicates for resolver

Tests & lint (pointers)

Surface	Role
tests/test_feature_pipeline.py, tests/test_candle_math.py	Pipeline / geometry
scripts/analysis/feature_math_lint.py	Ownership lint - no new local formula math
E3 feature_dag_*, feature_surface_query	Certification / query API
tests/test_schema_contracts.py	Schema pins

Danger flags

Flag	Meaning
NO_LOCAL_FORMULA_MATH	New quantities register in ontology -> registry first
PIT_SENSITIVE	Swings/structure must stay causal (F-051)
DIM_PIN	Schema hash load-bearing - dim change invalidates baselines

RESOLVER_!=_ENGINE	crt_state_resolver not byte-identical to crt_engine_v2 (F-069)
DOC_DRIFT	schemas.md may still say 38-dim - code is 39

E1b exit criterion

Criterion	Status
All 28 src/features/**/* .py files have purpose rows	Met
Registry authority chain documented	Met
Semantic layers L2/L4/L5/L7 + CRT resolver placed	Met
Linked to Ch.06-07 and F-ids	Met

Related: [Ch.06](#) * [Ch.07](#) * [E1](#) * [E3 feature DAG](#) * Machine twin: `encyclopedia_rows.jsonl`

Encyclopedia E2 - Research Utilities Index

Phase: E2
Status: DONE (category + package + driver index - not 269 prose essays)
Date: 2026-08-07
Parent: [Chapter 24](#) * [Encyclopedia index](#)
Architecture narrative: [Chapter 19](#) * [Chapter 20](#)

Scope

Tree	.py files	Role
src/research/	143	Kernels, hypotheses, measurement, adapters - importable library
scripts/research/	126	Thin CLI drivers that write results/research/**
Total E2 surface	269	Research-only authority unless a finding says otherwise

E2 design rule (from charter): index by **category + entry point + program link**, not a narrative chapter per file. Deep math stays in pre-registrations + findings.

Group: mostly **A-adjacent research** for kernels used by Programs 1-9; **D** for one-off scripts.

Relevance: almost all **RESEARCH** - no production fusion weight without \$6.5 Authority Ladder.

How to use this index

1. Start from a **Program** (table below) -> kernel + driver + artifact.
 2. Or start from a **src/research/ package** -> file list.
 3. Or start from a **script cluster** -> which CLIs exist.
 4. Confirm claims in docs/current-findings.md + results/research/**.
- Rule:** a claim without an artifact path (or finding Evidence) is a hypothesis, not a result.

1. Program -> code -> driver -> artifact (primary map)

Prog	Question (short)	Primary src/research	Typical scripts/research drivers	Typical artifacts
1	Directional edge on crypto majors?	qualification.py, hypotheses/*, adapters/spine_signal_source.py, controls/*, costs.py	qualify_majors.py	results/research/qualification
1b	Conditional pockets?	conditional_entropy	phase_b_conditional	results/research/phase_b/
1c	RETEST selection skill?	selection_effect.py	phase_s_selection_eff	results/research/phase_s/

1d	Exit/cost grid?	exit_grid.py, costs.py	phase_d_exit_grid.py	results/research/phase_d/
2	Sweep->disp->retest asymmetry?	structural_asymmetry.py, adapters/structural_event_source.py	phase_e_structural_asymmetry.py	results/research/phase_e/
3	H1/H4 rescue?	resample.py, adapters + qualification	build_resampled_data.py, qualify_htf.py	results/research/qualification/
P&F / IC	Interpreter edge?	src/interpreters/* + qualification.py	qualify_interpreter.py	Interpreter shadow results
4	Vol-regime level?	regime_conditioning.py	qualify_regime_conditioning.py	results/research/regime/
4b	Markov transition?	regime observers + conditioning	qualify_regime_transition.py	regime transition artifacts
4d / transitions	Compression->expansion info?	candle_state/*, hypotheses/compression	transition_information.py, qualify_transitions.py	results/research/candle_state/
5	Cross-sectional RV?	cross_sectional.py	qualify_cross_sectional.py	cross_sectional/*.json
6 / 6b	Carry signal / harvest?	cross_sectional.py + carry configs	qualify_carry.py, qualify_harvest.py	carry/*.json, harvest/*.json
8	Weekly sweep?	weekly_sweep/weekly_sweep_hypotheses/weekly_sweep_reversal.py	qualify_weekly_sweep.py	results/research/weekly_sweep/
9	M5 multi-TF + OCO straddle?	candle_state/m5_increment_mtf_conjunction.py, hypotheses/compression_resample.py	m5_mtf_information.py, qualify_m5_straddle.py, verify_m5_resample_parity.py	results/research/m5_mtf/
FX gen.	Crypto null on FX?	same M4 stack	qualify_fx_metals.py, qualify_xauusd.py	FX qualification trees
Zone audits	ZoneGate labels / inertness?	zone_mapping/*, zone_label_audit.py	zone_label_audit.py, diagnose_zone_inertness.py, discover_zones.py, ...	zone_* under results
RR clean labels	Honest RR labels?	clean_labels/*, measurement/forward	rr_l3_label_generator.py, rr_shadow_value.py, rr_kill_test_clean_l3.py	results/rr_research/ etc.
MSIP hyps	State vector info?	(shadow in src/msip - E1)	run_h_msip_001.py, run_h_msip_002.py	results/research/h_msip_001/ h_msip_002/
Gate measure	Fusion gate veto rate?	spine adapters	gate_measurement_measurement.py	gate_measurement_*

Shared **M4 gate** implementation center of gravity: `src/research/qualification.py` (+ `costs.py`, `measurement/*`, `controls/*`).

2. Shared research spine (load-bearing root modules)

These top-level modules under `src/research/` are reused across many programs:

File	Purpose	Relevance
qualification.py	M4 QualificationGate - absolute expectancy, controls, permutations, PROMOTE/REJECT	LIVE-research
costs.py	Round-trip cost model (e.g. 12 bps standard)	LIVE-research
exit_grid.py	SLxTP grid evaluations	P1d
cross_sectional.py	Panel / relative-value kernel	P5/P6/P6b
conditional_entropy_grid.py	Entropy + economics grid	P1b
selection_effect.py	Selected vs rejected retest decomposition	P1c
structural_asymmetry.py	Funnel asymmetry vs controls	P2
regime_conditioning.py	Regime x consumer matrix	P4/4b
resample.py	Deterministic calendar OHLCV resampler	P3, P9
experiment_spec.py	Experiment specification objects	ERP hygiene
config.py	Research config load helpers	all
contracts.py	Research contract types	ERP
runner.py / cli.py	Generic research runner / CLI glue	ops
registry.py	Research registry helpers	ERP
provenance.py	Run provenance stamps	hygiene
forensics.py	Trade/path forensics helpers	diagnostics
goal_alignment.py	Goal-layer alignment checks	Ch.01 link
process_diagnostics.py / process_characterization.py	Process / entropy diagnostics	P1b sanity
indicators.py	Research-side indicators (not production feature authority)	support
band_validation.py	Band / threshold validation helpers	FM/feature research
shape_statistics.py	Shape library statistics	IC tracks
mt5_cost_calibration.py	FX/metals cost calibration	F-035 class
xau_metals_protocol.py	XAU metals research protocol	metals track
zone_label_audit.py	Zone label honesty / F-041B class	ZoneGate
__init__.py	Package marker	-

Authority: research kernels must not silently become production fusion weights. Production path remains EngineRunner spine (E1).

3. src/research/ package encyclopedia

3.1 adapters/ (4)

File	Purpose
spine_signal_source.py	Research adapter: CRT spine entries as signal source for M4
structural_event_source.py	Sweep/disp/retest structural events (Program 2)
shape_signal_source.py	Shape/interpreter signal source adapter
__init__.py	Package exports

3.2 controls/ (3)

File	Purpose
random_baseline.py	Random-uniform / shuffle controls
always_long.py	Long-only / market control
__init__.py	Exports

Every economic PROMOTE path is expected to beat appropriate controls (program-specific).

3.3 hypotheses/ (8) - Stage-2 economic consumers

File	Purpose	Programs
expansion_breakout.py	Directional expansion breakout consumer	P1, P3, P4 toys
mean_reversion.py	Mean-reversion consumer	P1, P4
compression_breakout.py	Compression->expansion directional consumer	P4d Stage-2
compression_box_straddle.py	Sole P9 Stage-2 OCO straddle consumer	P9
weekly_sweep_reversal.py	Weekly sweep reversal consumer	P8
spine_hypothesis.py	Spine entry hypothesis wrapper	P1 spine arm
market_shape_hypothesis.py	Market-shape hypothesis	IC / shape
__init__.py	Exports	

3.4 candle_state/ (7) - multi-TF state & Program 9 Stage-1

File	Purpose
encoder.py	Candle-state encoding
mtf_conjunction.py	M15^H1^H4 (and extensions) conjunction builder
transition_target.py	Forward vol/range/transition targets (Program 4 kernels)
m5_incremental.py	M5-base keys + incremental information gate (P9 Stage-1)
info_robustness.py	Information robustness helpers
reporting.py	Stage-1 reporting
__init__.py	Exports

3.5 measurement/ (4) - honest outcomes

File	Purpose
forward_walk.py	Intrabar-aware forward walk labels (anti-F-022 contamination)
metrics.py	Research metrics (E, PF, ...)
bootstrap.py	Bootstrap CIs
__init__.py	Exports

MC path: schema in governance; this package is scaffolding toward sealed mc- * (still OPEN - Ch.20).

3.6 clean_labels/ (3)

File	Purpose
builder.py	Build clean labels from geometry + forward_walk
protocol.py	Clean-label protocol freeze
__init__.py	Exports

3.7 weekly_sweep/ (2)

File	Purpose
weekly_range.py	Calendar-locked weekly range + sweep geometry (P8)
__init__.py	Exports

3.8 zone_mapping/ (13)

Zone census, historical mapping, rare-zone evals, Gaussian-family shadows, CRTxzone crosstabs - research audits of ZoneGate / geometry (F-036, F-041 class). Files:

boundary_hypothesis_eval.py, build_corpus_zone_map.py, collect_trade_opened_features.py, crt_zone_crosstab.py, displacement_zone_event_study.py, gaussian_delta_gap_eval.py, gaussian_family_shadow_eval.py, historical_zone_mapper.py, rare_zone_context_filter_eval.py,

`rare_zone_detection_eval.py`, `rare_zone_fa_characterization.py`, `zone_census.py`, `__init__.py`.

3.9 episodes/ (13+)

Episode store / schema / tensors / projectors for trajectory research (not the live single-candle spine).

Core: `builder.py`, `events.py`, `flat.py`, `policy.py`, `protocol.py`, `query.py`, `schema.py`, `store.py`, `tensors.py`, `projectors/detection.py`, `projectors/spine.py`, `__init__.py`.

3.10 ic002_entry_evolution/ (5)

Trajectory build + separation eval for entry-evolution interpreter candidates: `build_trajectories.py`, `evaluate_separation.py`, `io_util.py`, `schema.py`, `__init__.py`.

3.11 ic003_shapes/ (3)

Shape library: `build_library.py`, `schema.py`, `__init__.py`.

3.12 ic003b_sequence_geometry/ (8)

Sequence geometry / DTW clustering: `build_all.py`, `cluster_dtw.py`, `cluster_euclid.py`, `continuum.py`, `dtw.py`, `schema.py`, `summarize.py`, `__init__.py`.

3.13 secondlow_v1/ (5)

Second-low detector track: `corpus.py`, `depth_metrics.py`, `detector.py`, `regime_metrics.py`, `__init__.py`.

3.14 path/ (2)

`ambiguity_census.py` - path ambiguity census (intrabar / touch-order class issues).

3.15 envelope_offline/ (3)

Offline envelope train/shadow: `train.py`, `shadow.py`, `__init__.py` (TradeNet/envelope research - not live fusion authority).

3.16 model_runners/ (23)

Generic offline model runner stack: `runner.py`, `envelope.py`, `contracts.py`, `require_config.py`, `schema_resolver.py`, `stats.py`, `substrate.py`, plus `adapters/*` for model families. **Research execution substrate**, not EngineRunner.

3.17 synthetic/ (10)

Synthetic story ontology for controlled experiments: `ontology.py`, `story_builder.py`, `story_registry.py`, `story_spec.py`, `stories/*`.

3.18 Root-only modules

Listed in §2 shared spine table (27 root `.py` including package init).

4. scripts/research/ - driver clusters (126 files)

Thin wrappers: **prefer** importing `src/research` kernels; scripts should not reinvent gate math.

4.1 Qualify / M4 gate (16) - primary economic runners

qualify_majors.py, qualify_htf.py, qualify_fx_metals.py, qualify_xauusd.py, qualify_transitions.py, qualify_regime_conditioning.py, qualify_regime_transition.py, qualify_cross_sectional.py, qualify_carry.py, qualify_harvest.py, qualify_weekly_sweep.py, qualify_m5_straddle.py, qualify_interpreter.py, qualify_shape_xauusd.py, qualify_zone_topk.py, xauusd_gaussian_m4_qualify.py.

When to run: only against a **pre-registration** and frozen config; write under `results/research/`.

4.2 Phase / Program 1 sub-drivers (4)

phase_b_conditional_entropy.py, phase_d_exit_grid.py, phase_e_structural_asymmetry.py, phase_s_selection_effect.py.

4.3 Regime / transition information (1+)

transition_information.py (Stage-1 info); economic twin often qualify_transitions.py / regime qualify scripts.

4.4 Program 9 M5 (2+)

m5_mtf_information.py (Stage-1), verify_m5_resample_parity.py, qualify_m5_straddle.py (Stage-2, listed under qualify).

4.5 Zone / ZoneGate research (14)

discover_zones.py, zone_label_audit.py, diagnose_zone_inertness.py, zone_gate_threshold_sweep.py, ablate_zone_thr_xauusd_fusion.py, convert_zones_v1_to_gaussian.py, build_zone_census.py, build_crt_zone_crosstab.py, build_displacement_zone_event_study.py, rare-zone builders, trace_zone_gate_xauusd.py, zone_x_o4_gap_study.py, ...

4.6 RR / clean labels (11)

rr_l3_label_generation.py, rr_l2_feature_truth.py, rr_l4_research_execution.py, rr_shadow_value.py, rr_kill_test_clean_l3.py, build_clean_labels_tn_env.py, build_rr_dataset_from_clean_labels.py, analyze_clean_labels_tn_env.py, h_rr_threshold_001.py, gaussian_rr_scatter.py, bitnet_population_label_audit.py.

4.7 BitNet / Gaussian / TradeNet diagnostics (8)

bitnet_r25_kill_test.py, bitnet_shadow_diagnostic.py, diagnose_gaussian_pivotality.py, Gaussian family/delta builders, XAUUSD Gaussian econ ledgers, TradeNet report patches.

4.8 CRT parity / resolver (7)

crt_parity_*.py, crt_state_confusion_matrix.py, validate_crt_state_resolver.py, crt_range_rebuild_probe.py, crt_resolver_economic_comparison.py.

4.9 IC / shapes (6)

ic002_build_trajectories.py, ic002_evaluate.py, ic003_build_shape_library.py, ic003b_build.py, ic003b_derive_information.py, shape_statistics_survey.py.

4.10 Episodes / corpus / path (13)

build_episodes.py, build_trace_corpus.py, analyze_trace_corpus.py, certify_xauusd_corpus.py, path ambiguity / knn / geometry tracers, enrich helpers.

4.11 XAU / metals / FX cost (8)

`run_xau_metals_protocol_v1.py`, `xauusd_mt5_cost_calibration.py`, `rerun_xau_metals_m4.py`, MT5 CRT runners, library build helpers.

4.12 MSIP hypothesis runners (2)

`run_h_msip_001.py`, `run_h_msip_002.py` - research-only; no CRT production authority.

4.13 Gate measurement (2)

`gate_measurement_m_gate_01.py` (F-070 class), `gate_o_nonlinear_probe.py`.

4.14 Private / underscore helpers (4+)

`_verify_outputs.py`, `_watch_h_rr.py`, `_fix_eval_auc.py`, `_show_top_decile_n9.py`, `_corpus_inspect.py`, ... - operator helpers, not programs.

4.15 Uncategorized residual (~28)

Includes forensics (`bnbusdt_forensics.py`), attribution studies, envelope train/shadow, story library, promotion dry-run, opportunity scanner, dimensional-mix diagnostics, FM030 band validation, model offline runners, etc. **Treat as ops/research tools:** read the module docstring before running; assume **write to results/** unless proven read-only.

5. Configs, pre-reg, artifacts (not code but E2 surface)

Kind	Path
Research configs	<code>configs/research/research_config*.json</code> , <code>configs/research/experiments/</code>
Measurement profiles	<code>configs/research/measurement_contracts/*.v1.json</code>
Pre-registrations	<code>docs/research/preregistration-*.md</code> , <code>docs/research-readiness/program-*.preregistration.md</code>
Findings	<code>docs/current-findings.md</code>
Family registry	<code>docs/governance/research_family_registry.json</code>
Outputs	<code>results/research/**</code>

6. Danger / authority flags

Flag	Meaning
<code>RESEARCH_ONLY</code>	Default for all of E2 - no fusion/production weight
<code>WRITES_ARTIFACTS</code>	Most <code>scripts/research/*</code> write under <code>results/</code>
<code>LABEL_SENSITIVE</code>	RR/zone label scripts - F-022 contamination class; prefer <code>forward_walk</code> / <code>clean_labels</code>

COST_MODEL	Changing bps or exit model invalidates cross-program comparison (Measurement Contract gap)
PRE_REG_REQUIRED	New economic claims need a frozen pre-registration first
OBSERVE_ONLY	Parity/trace/counter scripts must not change spine math

E2 exit criterion

Criterion	Status
Every src/research subpackage has a purpose row	Met (§3)
Shared root kernels listed	Met (§2)
Programs 1-9 mapped to kernels/drivers/artifacts	Met (§1)
All 126 scripts classified into clusters (or residual)	Met (§4)
Per-file essay for all 269	Out of scope (charter: index, not encyclopedia essays)

Next

- **E3** - Governance tooling (src/governance/, scripts/governance/, scripts/analysis/).
 - Optional deep-dives: one program appendix (e.g. P9 only) if Stage-1/2 re-runs are planned.
-

Related: [Ch.19](#) * [Ch.20](#) * [E1 spine](#) * [Ch.24](#)

Encyclopedia E3 - Governance Tooling

Phase: E3

Status: DONE (library + CLI index - not every analysis probe essay)

Date: 2026-08-07

Parent: [Chapter 24](#) * [Encyclopedia index](#)

Architecture narrative: [Chapter 16](#) * [Chapter 17](#) * [Chapter 18](#)

Scope

Tree	.py files	Role
src/governance/	19	Promotion, registries, validators, SITS cores - importable
scripts/governance/	31	Construction protocol, seeds, queries, feature-DAG, promote CLIs
scripts/analysis/	115	Census, probes, generators, instrument forensics (OBSERVE-heavy)
scripts/maintenance/	9	Hash, session-log rotation, hygiene (adjacent ops)
Total E3 surface	~174	Governance / truth / map tooling

E3 design rule: index by **job type** (promote, seed registry, certify features, generate maps, probe drift) - not 174 narratives. Deep doctrine stays in Ch.16-18 and docs/governance/.*

Group: governance libraries = **A-adjacent (ops authority)**; most analysis scripts = **D (operators)** or **OBSERVE_ONLY**.

Relevance: can **write** promotion logs, registries under data/, governance artifacts - treat write paths as gated.

How to use this index

If you need to...	Start here
Promote a config	PromotionManager + scripts/governance/promote_v2.py * Ch.16
Enforce construction protocol	scripts/governance/construction_protocol.py
Query hypotheses / frameworks / scripts	query_hypotheses.py / query_registry.py / query_scripts.py
Seed registries	seed_*_registry.py (PRIMARY in seeds, GENERATED in data/)
Feature certification / DAG	feature_dag_certify.py, feature_certification_state.py, feature_surface_query.py
Findings machine export	export_findings.py -> data/findings.jsonl
Config reachability / behavior census	scripts/analysis/config_reachability.py, behavior_census.py

Regenerate architecture maps	gen_code_map.py, gen_pyan.py, generate_cli_matrix.py, ...
------------------------------	--

1. Job map (primary navigation)

Job	Library (src/governance)	CLI / script	Writes	Chapter
Promote production config	promotion_manager.py	scripts/governance/promote.py	configs/production/*, promotion_log.jsonl	Ch.16
Shadow promotion gate	shadow_promotion_gate.py	python manager / dedicated flows)	shadow reports	Ch.16
Config integrity check	config_integrity.py	analysis / preflight	usually none	Ch.16; F-006 class
Portfolio validation	portfolio_validation.py	multi-market validation paths	validation reports	Ch.16
Multi-strategy validation	multi_strategy_validation.py, strategy_backtest.py	promote_v2 / validators	ValidationReport	Ch.16
Governance loop orchestrator	orchestrator.py	control-plane / CLI wrappers	audit trails	Ch.16 adjacent
Findings -> JSONL	findings_export.py	export_findings.py	data/findings.jsonl (GENERATED)	Ch.17
Hypothesis registry	hypothesis_registry.py	seed_hypothesis_registry.py, query_hypotheses.py	data/hypothesis_registry.jsonl	Ch.17/18
Framework registry	framework_registry.py	seed_framework_registry.py, query_registry.py, update_registry.py	data/framework_registry.jsonl	Ch.17
Script registry (SITS)	script_registry.py, script_census.py, script_seed.py	seed_script_registry.py + query_scripts.py, scripts/analysis/script_census.py	data/script_registry.jsonl	CLAUDE §3.1
Module attribution census	module_census.py, module_attribution.py	module census CLI under analysis	module registry artifacts	Ch.04 adjacent
Construction protocol	-	construction_protocol.py	completion validation	Gate 6 protocol
Feature DAG certify	-	feature_dag_certify.py, feature_certification_state.py	certification ledger	Ch.06-07 / F-054
Feature surface query	-	feature_surface_query.py	year-only	Ch.07 query surface
BitNet governance executor	bitnet_governance_executor.py	bitnet governance flows	gated	Ch.10 BitNet
Meta governor executor	(if present as meta path)	meta-governor flows	gated	Ch.16
Expansion integration	expansion_integration.py	expansion ops	research/ops	sidecar

Reflection buffer	reflection_buffer_advanced.py	advanced-reflection	sidecar	low priority
Runtime import boundary scan	-	scan_runtime_boundary_report	report	hygiene
Model path literal scan	-	scan_model_paths_literal.py	report	ModelPaths boundary
Three-authority surplus census	-	three_authority_surplus_census_artifact	license-artifact	Ch.17 / WHO-WHAT-HOW
CRT config construction census	-	crt_config_construction_census	license (F-057) class	Ch.13
Validation access pack	-	validation_access_cli	results/validation_access_cli	research/* access
Session log / hash hygiene	-	scripts/maintenance/*	logs, hashes	CLAUDE \$6

2. src/governance/ - library encyclopedia (19 files)

Promotion & validation (LIVE ops)

`promotion_manager.py`

- **Relevance:** LIVE * **Purpose:** Only path to production promotion - requires APPROVE ValidationReport, SHA-256, append-only `promotion_log.jsonl`.
- **Entry:** `python ../promotion_manager.py` or `promote_v2.py`; never bypass.
- **Chapter:** Ch.16.

`shadow_promotion_gate.py`

- **Relevance:** LIVE * **Purpose:** Two-gate shadow promotion before full promote.
- **Chapter:** Ch.16.

`config_integrity.py`

- **Relevance:** LIVE (often under-consumed) * **Purpose:** Config lineage / integrity guards (F-006: real check historically orphaned from hot path - still hygiene-critical).
- **Chapter:** Ch.16; findings F-006.

`portfolio_validation.py`

- **Relevance:** LIVE-research/ops * **Purpose:** Multi-market edge universality validation for portfolios.
- **Chapter:** Ch.16 adjacent; F-013 class (allocator orphaned from live single-candle spine).

`multi_strategy_validator.py` / `strategy_backtest.py`

- **Relevance:** PARTIAL * **Purpose:** ValidationReport across strategy modules + per-strategy backtest measurement.
- **Note:** Strategy modules largely DORMANT (Group C) - validators exist for multi-strategy governance path.
- **Chapter:** Ch.16; E5 for strategies themselves.

`orchestrator.py`

- **Relevance:** LIVE-ops * **Purpose:** GovernanceOrchestrator - multi-step governance loop coordination.
- **Chapter:** Ch.16.

Registries & generated truth (LIVE meta)

findings_export.py

- **Relevance:** LIVE * **Purpose:** Parse docs/current-findings.md -> data/findings.jsonl (GENERATED; never hand-edit).
- **CLI:** scripts/governance/export_findings.py
- **Chapter:** Ch.17.

hypothesis_registry.py

- **Relevance:** LIVE * **Purpose:** H-id registry schema/validation; authority locked to research.
- **Seed/query:** seed_hypothesis_registry.py, query_hypotheses.py
- **Chapter:** Ch.17 / E2.

framework_registry.py

- **Relevance:** LIVE * **Purpose:** Architecture component map (kernel/domain/style/...) as JSONL registry.
- **Seed/query/update:** seed_framework_registry.py, query_registry.py, update_registry.py
- **Chapter:** Ch.17.

script_registry.py / script_census.py / script_seed.py

- **Relevance:** LIVE * **Purpose:** SITS - discover scripts, merge stubs/overlays, materialize data/script_registry.jsonl.
- **CLI:** seed_script_registry.py, query_scripts.py, scripts/analysis/script_census.py (+ matrix generators).
- **Chapter:** CLAUDE.md §3.1; conventions.

module_census.py / module_attribution.py

- **Relevance:** LIVE * **Purpose:** Same idea as SITS for every src/**/* .py - ownership / attribution surface.
- **Chapter:** Ch.04 adjacent.

Specialist / sidecar

File	Relevance	Purpose
bitnet_governance_executor.py	SPECIALIST	BitNet-specific governance execution path
expansion_integration.py	SIDECAR	Wire expansion engine into governance loops
reflection_buffer_advanced.py	SIDECAR	Advanced reflection buffer for governance agents
__init__.py	LIVE	Package marker

3. scripts/governance/ - CLI encyclopedia (31 files)

3.1 Construction & promotion

Script	Purpose	Danger
construction_protocol.py	Gate-6 BUILD_IMPACT_MANIFEST / validate-completion / check	WRITE policy enforcement
promote_v2.py	Thin multi-strategy promote CLI	WRITES production configs
crt_config_construction_census.py	ConfigBuilder / CRTConfig construction sites (F-057 observe)	READ-ONLY census
behavioral_constant_authority_trace.py	Behavioral constants on decision spine	READ-ONLY

3.2 Registry seeds & queries

Script	Purpose
seed_hypothesis_registry.py	Seed H-ids from curated ledger
seed_framework_registry.py	Seed framework registry
seed_script_registry.py	SITS seed + PRIMARY overlays
export_findings.py	Regenerate findings JSONL
query_hypotheses.py / query_registry.py / query_scripts.py	Read-only queries
update_registry.py	Append-only framework registry update
framework_registry_report.py / framework_gap_audit.py	Markdown reports from registry

3.3 Feature DAG / FM governance

Script	Purpose
feature_certification_state.py	Per-feature certification ledger seed/resolver
feature_dag_certify.py	Certify / promote / STALE cascade mutations
feature_surface_query.py	Read-only join API over feature surface artifacts
build_fm_ownership_matrix.py	FM ownership / consumer matrix generator

3.4 Authority censuses & boundary scans

Script	Purpose
three_authority_surplus_census.py	WHO/WHAT/HOW declaration surplus
who_numeric_dependency_census.py	WHO numeric threshold dependency census
scan_runtime_boundary.py	Kernel ↔ research import direction + promotion boundary
scan_model_paths_literals.py	Unauthorized models/ path literals
build_g001_consumer_attribution.py	G001 consumer attribution package

3.5 Decision packages / MSIP / RR freeze / zone remap

Script	Purpose	Class
build crt architecture adjudication CRT.py	CRT architecture adjudication package	OBSERVE package
build msip shadow design contract MSIP.py	MSIP shadow design freeze package	design only
build msip1 verification package.py	MSIP-1 verification pack	OBSERVE
patch msip shadow design consistency consistency.py	Consistency patch for MSIP design pack	OBSERVE
rr_l1_freeze_certificate.py	RR L1 freeze certificate hash/assert	certificate
remap_zone_registry_v4.py	Zone registry schema v3->v4 remap	WRITES model JSON (one-shot)
validation_access_cli.py	VA-XAUUSD-M15 dual-surface access ladder	writes results packs
_build_doc_tracking_index.py	DOC_TRACKING_INDEX.xlsx metadata inventory	Excel write

4. scripts/analysis/ - census & probe clusters (115 files)

Default stance: OBSERVE_ONLY / diagnostic. Prefer governance feature-DAG tools for **mutations**; analysis scripts for **measurement**.

Cluster	~Count	Examples	Use when
Feature DAG / FM certification	15	feature_dag_layers.py, feature_math_lint.py, b2a_feature_candidate_certification.py, FM drift/decision-flip probes	Feature surface hygiene, F-053/F-054 class
CRT / state / parity	14	funnel diagnostics, guard ablations, transition audits, soft-conf / shadow probes	CRT CLOSED boundary forensics
RR / Gaussian / Zone / BitNet probes	10	rr_confidence_probe.py, zone parity, gaussian/bitnet runners	Engine AUDITED lineage evidence
XAUUSD / instrument forensics	9	trade anatomy, timestamp gaps, phase1 validation	Instrument-specific audits
Code map / wiring / matrices	7	gen_code_map.py, gen_pyan.py, generate_cli_matrix.py, generate_script_matrix.py, script_census.py	Regenerate architecture/CLI maps
Session / clock / normalization	5	session certification, cost audit, override scoping	F-066 / session policy class
Config / behavior census	2	config_reachability.py, behavior_census.py	Config-first maturity (§6.5)
Blind label / epistemic	2	blind label sample/score	Epistemic / labeling studies

Geometry census	1+	geometry_census.py	Geometry family registry inputs
Other / residual	~50	pattern library, consensus sweeps, log compress, early invalidation A/B, detection sweeps, ...	Read docstring; classify before reuse

Regenerate maps (safe, high-ROI):

```
scripts/analysis/gen_code_map.py scripts/analysis/gen_pyan.py
scripts/analysis/generate_cli_matrix.py scripts/analysis/generate_script_matrix.py
scripts/analysis/script_census.py # often with --write-stubs for SITS
```

5. scripts/maintenance/ - adjacent hygiene (9 files)

Script	Purpose
_compute_hash.py	Recompute production config hash after params edits
rotate_session_log.py	Rotate assistant_project.md / workflow log
check_session_log_commit.py	Commit-hook session log presence
check_governance_invariants.py	Governance invariant checker
check_consolidation_due.py	Consolidation due probe
cleanup_logs.py / reorganize_results.py	Log/results hygiene
fix_bom.py	BOM fixes
_promote_v4_bnb_cutover.py	Historical cutover helper (danger: version-specific)

6. Evidence ledger home (docs, not scripts)

E3 tools **read/write into** the evidence directory mapped in Ch.18:

Cluster	Examples
Closure reports	crt_closure_report.md, feature surface closures
Lineage audits	*_lineage_audit.md
Certification ledgers	feature_certification_ledger.jsonl, DAG LATEST JSON
Measurement contract	MEASUREMENT_CONTRACT.md (still OPEN seals)
Construction protocol	REPOSITORY_CONSTRUCTION_PROTOCOL.md, change_contracts.json
MIAR	MODEL_INTENT_AUTHORITY_REGISTER.md

7. Danger / authority flags

Flag	Meaning
PROMOTION_WRITE	Can change active production config - human confirm required
GENERATED_ONLY	data/*.jsonl - regenerate from seeds/docs; never hand-edit as truth
APPEND_ONLY	Registries / promotion_log / certification ledger - no silent rewrite
OBSERVE_ONLY	Analysis probes must not change production behavior
ONE_SHOT_MUTATION	e.g. zone registry remap - archive + hash after
ORPHANED_CHECK	e.g. config_integrity may not gate runtime - still run in hygiene

E3 exit criterion

Criterion	Status
Every src/governance module has a purpose row	Met (§2)
All scripts/governance listed by job cluster	Met (§3)
scripts/analysis clustered with regenerate shortlist	Met (§4)
Maintenance adjacency noted	Met (§5)
Per-file essay for all 115 analysis scripts	Out of scope (cluster index)

Next

- **E4** - Group B sidecars (bitnet, utils, training, portfolio, replay, ...)
- **E5** - Group C dormant one-liners
- **E6** - Remaining non-research / non-governance scripts catalog

Related: [Ch.16](#) * [Ch.17](#) * [Ch.18](#) * [E1](#) * [E2](#) * [Ch.24](#)

Encyclopedia E4 - Sidecar / Support Modules (Group B)

Phase: E4
Status: DONE (package + file map for non-spine support surfaces)
Date: 2026-08-07
Parent: [Chapter 24](#) * [Encyclopedia index](#)

Scope

Group B packages **support** the system but are **not required** to understand the candle->order spine on day one. They are not all equally “dead” - some are LIVE data/ops helpers; many are INERT, UNWIRED, RESEARCH, or OBSERVE_ONLY on the active production config.

Package	.py files	Default posture
src/bitnet/	23	Built; inert when use_bitnet: (F-004/F-055)
src/utils/	17	LIVE helpers (logging, JSONL, manifests)
src/retrieval/	9	Enterprise RAG - optional tooling
src/training/	8	Offline train pipelines; TradeNet built/unwired (F-005)
src/expansion/	6	Controlled frequency expansion - ops/research
src/portfolio/	6	Allocator built; live spine still single-candle (F-013)

src/replay/	6	Replay memory / timing - sidecar (F-012 class)
src/analytics/	5	Offline metrics / clustering
src/regime/	4	Regime clustering / routing - research-adja
src/msip/	6	Shadow market state - OBSERVE_O (Ch.18)
src/multi_llm/	5	Multi-LLM ops tooling (also root multi_llm/)
src/events/	2	Event fabric
src/search/	1	Regime weight searcher
src/monitoring/	2	Health endpoint (borderline dormant/ops)
Total	~100	

Not in E4: Group C dormant strategies/scanner/journal (-> **E5**); research/gov (E2/E3); spine (E1).

Rule: SIDECAR != deleted. Read when a task touches the package; do not promote into fusion without Authority Ladder evidence.

How to use

Need	Package
BitNet zone gate / low-bit models	bitnet/ + Ch.10

Logging / JSONL / run manifests	utils/
Offline train / TradeNet	training/
Portfolio multi-instrument	portfolio/
Replay memory / pattern timing	replay/
Offline performance metrics	analytics/
Shadow MSV	msip/
Multi-LLM turn ledger / packs	multi_llm/ + Ch.23
Codebase RAG	retrieval/
Parameter expansion	expansion/

1. src/bitnet/ - BitNet / zone intelligence stack (23)

Posture: Full stack **built**; production **INERT** while use_bitnet false on active patch.
Shadow A/B (F-055) did not earn activation authority.

File	Purpose
__init__.py	Package: BitNet parameter / zone discovery engine
bitnet_inference.py	Inference for dual JSON model schemas
bitnet_runner.py	Schema enforcement + fail-closed dimension checks
bitnet_registry.py	Version registry - Exists != Selected != Enabled
model_contract.py	Canonical serialization contract (bitnet_v3)
model_bundle.py	CONTRACT-C model.bundle write/validate
composition.py	Encoder -> Backbone -> Heads -> Adapter composition
encoders.py / backbones.py / heads.py / adapters.py	Spec hierarchy components
layers.py	Pure numeric BitLinear primitives
defaults.py	Versioned numeric defaults (Spec v1.2.1)
label_contracts.py	CONTRACT-C label definitions
contract_c_trainer.py	CONTRACT-C trainer for BitLinear residual family
runtime_types.py	Shared inference value types
zone_cosine_searcher.py	Cosine match to zone vectors
zone_validator.py	Min quality criteria for candidate zones
forward_tester.py	OOS validation for zone registry
stability_checker.py	Zone stability across temporal splits

population_label_audit.py	Read-only population/label balance audit
r25_kill_test.py	R2.5 kill-test harness
_smoke_test.py	Module smoke test

Related: src/engines/zone_gate_engine.py (E1), F-004/F-050/F-055, BitNet lineage audit.

Do not: enable on active config without ΔG001 + promotion path.

2. src/utils/ - shared helpers (17)

Posture: LIVE support - imported widely; not decision authority.

File	Purpose
logging_config.py	Centralized per-flow logging
console_safe.py	Windows/cp1252-safe console output
jsonl_writer.py	Canonical append-only JSONL helper
trade_logger.py	Trade event logging
llm_logger.py	Structured LLM session logger (UAT/SIT)
log_identity.py / log_index_writer.py	Log identity + index writers
engine_telemetry.py	Structured engine observability wrapper
integrity_events.py	Integrity-event telemetry spine
sweep_trace_logger.py	CRT sweep decision traces (sweep_trace.jsonl)
episode_summarizer.py	Episode summarization helpers
pattern_hasher.py	Stable pattern IDs for replay/memory
run_manifest.py	Provenance manifest for research runs (ERP H1/H2/H3)
validation_contract.py	Mechanical anti-hallucination checks over manifests
config_dumper.py	Dump resolved config to timestamped JSON
registry_refresh.py	mtime hot-reload tracker for model/config files
zone_schema_migrator.py	BitNet zone schema migration utility

3. src/training/ - offline training (8)

Posture: RESEARCH / OFFLINE. TradeNet **built but unwired** in fusion (F-005).

File	Purpose
train_pipeline.py	Main train pipeline (incl. Phase-5 calibration gate before register)
trainer.py	Trainer core loop

<code>evaluator.py</code>	Offline evaluation
<code>phase5_calibration.py</code>	Phase-5 calibration (note: historical gaussian artifact builder path)
<code>stage1_dataset_builder.py</code>	Stage-1 truth dataset for TradingLLM-class training
<code>trade_net_v2.py</code>	TradeNet v2 model definition / train surface
<code>training_trigger.py</code>	Programmatic gate to invoke auto-train
<code>bar_semantic_tracker.py</code>	Append-only bar-level semantic journal for train/eval

Related: E2 RR clean labels; `models/*` artifacts; never confuses train success with production authority.

4. `src/portfolio/` - multi-instrument allocation (6)

Posture: BUILT / ORPHANED from live single-candle spine (F-013). Research/ops use only until wired.

File	Purpose
<code>allocator.py</code>	PortfolioAllocator - multi-instrument capital allocation
<code>capital_policy.py</code>	Capital policy rules
<code>correlation_engine.py</code>	Cross-instrument correlation
<code>exposure_tracker.py</code>	Exposure tracking
<code>replay.py</code>	Read-only shadow replay of allocator over historical ledger
<code>__init__.py</code>	Package marker

5. `src/replay/` - institutional replay memory (6)

Posture: SIDECAR (F-012 class - zero spine consumption by default).

File	Purpose
<code>replay_memory_engine.py</code>	Persistent institutional memory layer
<code>replay_similarity_index.py</code>	Similarity search over replay vectors
<code>replay_drift_governor.py</code>	Contamination / staleness / cluster drift monitor
<code>timing_advisor.py</code>	Read API for Pattern Timing Library (measure-only Phase 1)
<code>timing_reconstructor.py</code>	Forward-walk timing augmentation of outcomes
<code>__init__.py</code>	Package marker

6. `src/analytics/` - offline analytics (5)

Posture: OFFLINE research/ops metrics (not live decision).

File	Purpose
metrics_oracle.py	Independent backtest metric recompute (trust layer)
performance.py	Win rate, expectancy, drawdown breakdowns
clustering.py	Cluster losing trades by condition
sl_tp_comparator.py	Dual SL/TP method comparator
__init__.py	Package exports

7. src/expansion/ - controlled frequency scaling (6)

Posture: OPS/RESEARCH - bounded parameter/frequency expansion; not free optimizer.

File	Purpose
expansion_engine.py	Expansion engine core
config_mutator.py	Config mutation helpers for expansion trials
evaluator.py	Expansion trial evaluation
policy_schema.py	Expansion policy schema
llm_pattern_extractor.py	Optional LLM pattern extraction for expansion
__init__.py	Package: controlled trade frequency scaling

8. src/regime/ - regime surfaces (4)

Posture: RESEARCH-adjacent (complements E2 regime_conditioning; not Ultron).

File	Purpose
regime_classifier.py	Regime classification
market_state_cluster_engine.py	Cluster-native market state classification
config_router.py	Regime -> config routing helper
__init__.py	Package marker

9. src/msip/ - Market State Interpretation Platform shadow (6)

Posture: OBSERVE_ONLY - never mutates CRT; never gates execution (Ch.18 brief).

File	Purpose
market_state_vector.py	MSV value object (schema pin)
interpretation_config.py	msip_shadow HOW config; off when absent/disabled

shadow_emitter.py	Pure build + append-only emit
disagreement.py	Shadow vs CRT disagreement taxonomy (telemetry)
isolation.py	Import/mutation isolation policy
__init__.py	Shadow package marker

Related: run_h_msip_001/002 (E2); no production cutover without explicit gates.

10. src/multi_llm/ - multi-LLM ops package (5)

Posture: OPS tooling for hand-operated multi-model workflow; **isolated from trading spine**.
(Repo root multi_llm/ holds protocol docs, ledger, roles - Ch.23.)

File	Purpose
turn_ledger.py	Append-only multi-LLM turn record
context_pack.py	Bounded upload-ready context pack for one task
discussion.py	Reconstruct / rewind discussion (view-only)
tokens.py	Heuristic token estimates for pack bounds
__init__.py	Coordination layer package

11. src/retrieval/ - codebase RAG (9)

Posture: OPTIONAL tooling (ChromaDB + embeddings). Not on candle->order path.

File	Purpose
retriever.py	High-level retrieval + context assembly
vector_store.py	ChromaDB hybrid search store
embedding.py	sentence-transformers embedding pipeline
chunking.py	Structure-aware semantic chunking
corpus.py	Repo document discovery by domain
config.py	Retrieval tunables
monitor.py	RAG metrics
claude_integration.py	Claude Code RAG-grounded task helpers
__init__.py	Enterprise RAG package

12. Small surfaces

src/events/ (2)

File	Purpose
event_fabric.py	Canonical Event Fabric for runtime eventing
__init__.py	Package marker

src/search/ (1)

File	Purpose
regime_weight_searcher.py	Search/sweep helpers for regime weights (research/ops)

src/monitoring/ (2)

File	Purpose
health_checker.py	Lightweight stdlib HTTP health on port 8788
__init__.py	Package marker

Note: Control plane dashboard is 8787 (E1); health checker is a separate lightweight surface.

Cross-cutting findings (why these stay sidecar)

Finding / class	Implication for E4
F-004 / F-055	BitNet hard gate inert / shadow not activated
F-005	TradeNet fusion slot unwired
F-012	Replay / cognitive / HMF sidecar-only
F-013	Portfolio allocator + scan loop orphaned from live single-candle spine
F-060	Live Gaussian channel near-constant - separate from bitnet/ package
Ch.18 MSIP	Shadow only; no CRT mutation

Danger flags

Flag	Meaning
INERT_UNTIL_CONFIG	Present in tree; inactive unless config enables (BitNet)
UNWIRED	Built but no production caller (TradeNet neural slot, portfolio live)
OBSERVE_ONLY	MSIP shadow, many replay/timing measure paths
OPTIONAL_DEPS	retrieval (ChromaDB, sentence-transformers) may be absent
NO_FUSION_AUTHORITY	None of E4 grants \$6.5 production weight by existing

E4 exit criterion

Criterion	Status
Every Group B package listed in Ch.24 has a section	Met
File-level purpose for bitnet/utls/training/portfolio/replay/analytics/expansion/regime/msip/multi_llm/retrieval/events/search/monitoring	Met
Explicit non-authority vs spine	Met
Deep train/BitNet tutorials	Out of scope (lineage audits + topics)

Next

- **E5** - Group C dormant (strategies, scanner, journal, cognitive, feedback, llm_research, data_ingestion, uat, ui)
- **E6** - Remaining scripts outside research/governance/analysis catalogs

Related: [Ch.10](#) * [Ch.18 MSIP](#) * [Ch.23](#) * [E1](#) * [E3](#) * [Ch.24](#)

Encyclopedia E5 - Dormant / Archived Surfaces (Group C)

Phase: E5
Status: DONE (honest one-line + reopen rules - do not deep-read by default)
Date: 2026-08-07
Parent: [Chapter 24](#) * [Encyclopedia index](#)

Scope

Group C packages were **intentionally deprioritized** by the architecture book because they are not on the **active production candle->order story** - scaffold, multi-strategy experiments, sidecars with zero spine consumption, UAT harnesses, or **archived** parallel rails.

Package / tree	.py files	Default posture
src/strategies/	18	DORMANT scaffold (S01-S10)
src/scanner/	6	DORMANT / ORPHANED vs live single-candle spine
src/journal/	6	DORMANT identity/prove layers
src/cognitive/	2	DORMANT sidecar bus (F-012)
src/feedback/	2	DORMANT
src/llm_research/	5	DORMANT offline LLM research pipeline
src/data_ingestion/	6	MIXED - see §7 (not all dead)
src/uat/	4	DORMANT multi-strategy UAT harness

src/ui/	1	Empty / remnant package
archive/**	~19	ARCHIVED do not import from production
Total	~69	

Rule: DORMANT != delete. Means **do not reopen without explicit intent** and a reopen checklist.

How to use

1. **Default:** skip deep-reading these packages for spine work.
2. **If reopening:** read the package row + reopen conditions below; run construction protocol; do not silently wire into EngineRunner.
3. **Archive:** read only for forensics; never re-import as live path without a full design.

1. src/strategies/ - S01-S10 multi-strategy scaffold (18)

Posture: DORMANT - full strategy module framework exists; live production still runs the **single CRT spine**, not StrategyOrchestrator multi-strategy hot path (see F-013 class).

File	Purpose (one line)
base_strategy.py	ABC for S1-S10
strategy_result.py	Universal strategy output contract
strategy_intent.py	Typed hypothesis intent from strategies
intent_builder.py	StrategyResult -> intent adapter
strategy_orchestrator.py	Run all 10 strategies per candle + aggregate
strategy_package.py	Strategy package / registry architecture target
strategy_registry.py	Load/list StrategyPackage versions
s01_crt_wrapper.py	S1 CRT wrapper (closest to live spine)
s02_mean_reversion.py ... s10_trap_strategy.py	S2-S10 alternate strategies (MR, breakout, stat-arb, grid, scalp, news, ML ensemble, pattern, trap)
__init__.py	Package exports

Reopen if: multi-strategy production is an explicit goal + ValidationReport path + single-spine parity.

Do not: assume S08/S06 etc. are live because files exist.

Related: E3 multi_strategy_validator / strategy_backtest can exercise them offline.

2. `src/scanner/` - multi-symbol scan rail (6)

Posture: **DORMANT / ORPHANED** relative to live single-candle spine (scan->allocate->loop class, F-013).

File	Purpose
<code>scanner.py</code>	Multi-symbol scanner core
<code>universe.py</code>	Symbol universe definition
<code>ranker.py</code>	Rank scan hits
<code>signal_pool.py</code>	Pool of candidate signals
<code>spine_adapter.py</code>	Only designed path scanner -> <code>EngineRunner.run()</code>
<code>__init__.py</code>	Package marker

Reopen if: multi-symbol live allocation is authorized and wired end-to-end through Ultron.

Do not: confuse with `archive/inout_legacy/.../scanner.py` (different archived rail).

3. `src/journal/` - trade identity / provenance (6)

Posture: **DORMANT** as a production journal system; contracts exist for future correlation.

File	Purpose
<code>trade_identity_v1_0.py</code>	Minimal immutable trade identity (alert -> decision link)
<code>trade_provenance_v1_0.py</code>	Config/model/promotion lineage keyed by <code>trade_id</code>
<code>trade_execution_link_v1_0.py</code>	Broker/venue correlation layer (disposable Tier 0B)
<code>trade_logger.py</code> / <code>schema.py</code>	Journal logging + schema
<code>__init__.py</code>	Package marker

Note: Live logging often uses `src/utls/trade_logger.py` (E4) instead.

Archive twin: `archive/dead_code/journal/*` - older copies; ignore for new work.

4. `src/cognitive/` - cognitive bus (2)

Posture: **DORMANT sidecar** - async cognitive processing; **zero spine consumption** (F-012).

File	Purpose
<code>cognitive_bus.py</code>	Async cognitive processing bus
<code>__init__.py</code>	Package marker

Reopen if: explicit cognitive sidecar product goal; never block `EngineRunner` on bus health.

5. `src/feedback/` - AI feedback (2)

Posture: DORMANT.

File	Purpose
ai_feedback.py	AI feedback collection/loop (scaffold)
__init__.py	Package marker

Reopen if: closed-loop learning product is designed with measurement contract + no silent train-in-prod.

6. src/llm_research/ - offline LLM research pipeline (5)

Posture: DORMANT offline pipeline (pattern -> policy -> forward test). Not live LLM gate.

File	Purpose
pattern_extractor.py	Extract patterns via LLM offline
policy_builder.py	Build policies from extracted patterns
forward_tester.py	Forward-test generated policies
evaluator.py	Evaluate LLM research outputs
__init__.py	Pipeline architecture package

Distinct from: live llm_inference_client (E1, fail-open advisory) and multi-LLM ops (E4/Ch.23).

7. src/data_ingestion/ - MIXED (6)

Not pure dormant. Parts are legacy DB-oriented; parts are **LIVE integrity** used by research/backtest.

File	Posture	Purpose
dataset_integrity.py	LIVE-research	L3 whole-dataset sequence integrity preflight (F-039: limited call sites)
ohlcv_schema.py	LIVE-contract	Historical OHLCV column contract
session_autoderive.py	RESEARCH	Learn tradable session from data (vs hardcoded calendars)
xauusd_phase1_candidate.py	RESEARCH freeze	XAUUSD M15 Phase-1 candidate identity verification
historical_fetcher.py	LEGACY / convention tension	Multi-pair OHLCV into TimescaleDB (repo doctrine: no DB - prefer src/inout/* CSV fetchers)
__init__.py	-	Package marker

Prefer for new data I/O: src/inout/* fetchers (E1/Ch.15).

Prefer for integrity: dataset_integrity at backtest/research preflight when required.

8. src/uat/ - multi-strategy UAT harness (4)

Posture: DORMANT relative to single-spine production; useful only if multi-strategy returns.

File	Purpose
uat_runner.py	Orchestrates UAT areas for 10-strategy system
monte_carlo.py	Bootstrap resampling robustness checks
kill_switch.py	Daily/weekly loss kill-switch state machine
__init__.py	Package marker

Note: `kill_switch` ideas may overlap live risk concerns - production capital authority remains **UltronRiskGate** (E1/Ch.14).

9. `src/ui/` - remnant (1)

File	Purpose
__init__.py	Empty/remnant UI package

Archive: `archive/ui_legacy/dashboard_ARCHIVED_2026_05_02.py`, `archive/dead_code/ui/dashboard.py`.

Live UI surface: control-plane localhost dashboard (E1/Ch.22), not `src/ui/`.

10. `archive/**` - ARCHIVED (do not import)

Path	Why archived
<code>archive/inout_legacy/ARCHIVED_2026_05_02/</code>	Parallel execution rail (scanner/state_machine/controller/db) - bypassed spine; SQLite violated no-DB (Ch.15)
<code>archive/dead_code/**</code>	Extracted dead modules (old bitnet smoke, journal, ui, insight_reporter, bitnet_feature_builder)
<code>archive/ui_legacy/</code>	Old dashboard
<code>archive/scripts/</code>	Old historical data runner scripts (typo'd names preserved)
<code>archive/zips/</code>	Zip backups - not code authority

Replacement for archived INOUT rail: `HookedLiveEngine.process` (E1).

Replacement for data I/O: `src/inout/*` CSV fetchers.

Reopen checklist (any Group C package)

1. Explicit user/owner goal (not “file looks interesting”).
2. Classify against construction protocol / change contracts.
3. Prove callers (or add them) - no silent hot-path wire.
4. Measurement: if economic claim -> pre-registration + M4-class gate (E2).
5. If multi-strategy/scanner: address F-013 single-spine reality.
6. Never re-enable `archive/inout_legacy` as a second spine.
7. SESSION LOG + docs decision (Ch.17 drift protocol).

Danger flags

Flag	Meaning
DORMANT	Present; off active story; deprioritize
ORPHANED	Built path with no live caller (scanner/portfolio class)
ARCHIVED	Explicitly retired under archive/ - forensic only
MIXED	Package contains both live-adjacent and legacy files
CONVENTION_BREAK	e.g. TimescaleDB fetcher vs no-DB doctrine

E5 exit criterion

Criterion	Status
All Group C packages from Ch.24 have one-line map	Met
Archive trees listed with replacement path	Met
data_ingestion MIXED called out (not false dormant)	Met
Reopen checklist	Met
Full strategy design revival	Out of scope

Next

- **E6** - Remaining scripts outside research/governance/analysis catalogs (operator script matrix residual).

Related: [Ch.15 INOUT](#) * [E1 spine](#) * [E3 multi-strategy validators](#) * [Ch.24](#) * F-012 / F-013

Encyclopedia E6 - Remaining Operator Scripts (Group D residual)

Phase: E6
Status: DONE (category catalog for scripts not covered by E2/E3)
Date: 2026-08-07
Parent: [Chapter 24](#) * [Encyclopedia index](#)

Scope

Already catalogued	Where
scripts/research/ (126)	E2
scripts/governance/ (31)	E3
scripts/analysis/ (115)	E3
scripts/maintenance/ (9)	E3

E6 residual under scripts/	Count
data/	22
training/	13
root scripts/*.py	10
context/	5
misc/	5
evaluation/	3
export/	3
groq_bridge/	3
backtest/	2
control_plane/	1
metrics/	1
multi_llm/	1
portfolio/	1
E6 total	~70
All scripts/**/*.py	~351

Design rule: operator catalog by **job** (fetch data, train model, run backtest, multi-LLM context) - not 70 essays.
Prefer `src/` kernels; scripts stay thin.

Group: D (operational scripts).
Default danger: many **WRITE** data/, models/, results/, configs hashes.

How to use

Job	Directory
Fetch / convert / verify OHLCV	scripts/data/ -> src/inout/* (E1)
Train BitNet / RR / TradeNet / Gaussian	scripts/training/ -> src/training, src/bitnet (E4)
Portable Mind / multi-LLM context	scripts/context/ -> Ch.23
Control-plane server	scripts/control_plane/run_server.py -> Ch.22
Manual / debug backtest	scripts/backtest/ -> src/runtime/backtest_v2 (E1)
Groq bridge workflow	scripts/groq_bridge/
Model export / bootstrap	scripts/export/
One-off parity / zone / replay tools	scripts/misc/

Authoritative runnable catalog when regenerated: docs/reference/cli-matrix.md + docs/reference/script-matrix.md (E3 generators).

1. scripts/data/ - market data I/O (22)

Posture: LIVE ops for corpora. Writes under data/.

Fetch (thin wrappers over `src/inout`)

Script	Purpose
fetch_candles_mt5.py	MT5 -> CSV
fetch_candles_alphavantage.py	Alpha Vantage FX
fetch_candles_hummingbot.py	Hummingbot/exchange OHLCV
fetch_perp_funding.py	Binance perp funding/basis
fetch_crypto_ccxt.py	CCXT crypto fetch
fetch_forex_yfinance.py	Yahoo Finance forex
fetch_and_verify_binance.py / fetch_and_verify_mt5.py	Fetch + verify pipelines
check_availability.py / check_yfinance.py / test_yf_periods.py	Connectivity / period checks

Build / convert / verify

Script	Purpose
convert_binance_m1_to_m15.py / misc cousin resample	M1->M15 resample

convert_bnb_to_csv.py / csv_to_excel.py / split_bnb_excel.py	Format converters
build_m15_unified.py / unified_data_builder.py / prepare_data.py	Unified dataset builders
build_rr_dataset.py / build_tradenet_dataset.py	Train-set builders (label-sensitive)
generate_vectors.py	Feature vector generation helper
verify_output.py	Output verification

Prefer: production loaders + dataset_integrity (E5 MIXED) for sequence checks.

Danger: LABEL_SENSITIVE datasets (RR/TradeNet) - F-022 class.

2. scripts/training/ - model training CLIs (13)

Posture: OFFLINE. Artifacts under models/. No automatic production authority.

Script	Purpose
train_pipeline.py	Main train pipeline CLI
train_rr_model.py	RR pattern model train
train_bitnet.py / train_bitnet_contract_c.py	BitNet / CONTRACT-C train
train_trade_net_v2.py	TradeNet v2 train (unwired fusion - F-005)
train_gaussian_xauusd.py	Gaussian train path (artifact lineage caution - F-060)
phase5_calibration.py	Phase-5 calibration gate CLI
build_stage1_dataset.py	Stage-1 dataset for LLM/train
auto_tuner.py / auto_tuner_multi.py / auto_tuner_gemini_gate.py	Parameter tuner family
show_xauusd_gaussian_promotion.py	Promotion display helper
_write_hashes_closure_narrative.py	Hash/closure narrative helper

Related: E4 src/training/, E4 src/bitnet/, E3 promote path for any production registration.

3. Root scripts/*.py (10)

Script	Purpose	Danger
update_config_hash.py	Config hash helper (pair with maintenance _compute_hash)	HASH write
export_model_registry.py	Export model registry snapshot	read/write artifacts
auto_train_from_opportunities.py	Auto-train trigger from opportunities stream	TRAIN write
validate_integration.py	Integration validation harness	-

build_consolidated_docs.py	Docs consolidation builder	docs write
extract_folder_structure.py	Repo tree extract	-
rag_index.py	Build/update RAG index (src/retrieval)	optional deps
_gate5_compliance_pass.py	Gate-5 compliance counting helper	observe
tmp_cert_worktrees.py	Worktree certification temp helper	ops
__init__.py	Package marker	-

4. scripts/context/ - Portable Mind / multi-LLM (5)

Posture: OPS for multi-LLM workflow (Ch.23). Regenerates gitignored context/*.md.

Script	Purpose
build_context.py	Compile trigger - rebuild Portable Mind context/*.md
log_turn.py	Append multi-LLM turn to ledger
discussion.py	Discussion reconstruct / rewind
pack_story.py	Bounded story pack for handoff
seed_build_queue.py	Seed multi_llm/build_queue.jsonl

Related: E4 src/multi_llm/, root multi_llm/.

5. scripts/control_plane/ (1)

Script	Purpose
run_server.py	Start localhost control-plane HTTP server (:8787)

Related: E1 src/control_plane/, Ch.22. **No auth / localhost only.**

6. scripts/backtest/ (2)

Script	Purpose
manual_backtest.py	Manual backtest entry wrapper
backtest_debug_harness.py	Debug harness around backtest path

Related: E1 src/runtime/backtest_v2.py. Prefer official CLI entry points from cli-matrix when present.

7. scripts/export/ (3)

Script	Purpose
--------	---------

export_bitnet_model.py	Export BitNet model artifacts
generate_bootstrap_model.py	Bootstrap model generation
regen_bitnet_35.py	BitNet 3.5 regen helper

Danger: model artifact writes; no auto-promote.

8. scripts/evaluation/ (3)

Script	Purpose
run_benchmark.py	Benchmark runner
report.py	Evaluation report generation
__init__.py	Package marker

9. scripts/groq_bridge/ (3)

Script	Purpose
prepare_retrospective.py	Prepare retrospective pack for Groq/LLM
ingest_response.py	Ingest LLM response into workflow
apply_llm_suggestions.py	Apply suggestions (review carefully - may write)

Danger: treat apply step as human-gated.

10. scripts/misc/ (5)

Script	Purpose
run_parity.py	Parity run helper
trade_replay_validator.py	Trade replay validation
build_zone_registry_from_trades.py	Zone registry from trades (label-sensitive)
resample_m1_to_m15.py	Resample utility
bitnet_ternary_inference.py	BitNet ternary inference probe

11. Single-file operator CLIs

Script	Purpose
metrics/extract_metrics.py	Extract metrics from run artifacts
multi_llm/initiate_plan.py	Initiate multi-LLM plan cycle

portfolio/replay_allocator.py	Portfolio allocator shadow replay (F-013 class)
-------------------------------	---

Cross-links (do not duplicate)

Need	See instead of reinventing
Research qualify / phase drivers	E2
Promote / SITS / feature DAG	E3
Session log rotate / config hash	E3 maintenance
In-repo libraries behind scripts	E1 / E4

Danger flags (E6)

Flag	Meaning
WRITES_DATA	Touches data/** corpora
WRITES_MODELS	Touches models/**
WRITES_CONFIG_HASH	Production config hash paths
TRAIN_OFFLINE	No live trading authority
LOCALHOST_ONLY	Control plane / health
HUMAN_GATE	Groq apply / auto-train / promote-adjacent

E6 exit criterion

Criterion	Status
All scripts/ top-level dirs either in E2/E3 or listed here	Met
Residual ~70 scripts categorized by job	Met
Pointers to cli-matrix / script-matrix	Met
Per-script full API docs	Out of scope

Encyclopedia series status

Phase	Status
E0 Charter	DONE
E1 Spine Group A	DONE
E2 Research	DONE

E3 Governance	DONE
E4 Sidecars	DONE
E5 Dormant/Archive	DONE
E6 Remaining scripts	DONE

Optional later: E1b (`src/features/` registry depth), per-program research deep-dives, or machine-generated JSONL twin of all encyclopedia rows.

Related: [E2](#) * [E3](#) * [E4](#) * [Ch.22](#) * [Ch.23](#) * [Ch.24](#) * `docs/reference/cli-matrix.md`

Appendix A1 - Testing the System

Status of this chapter: Written

Why this chapter exists

Nearly every earlier chapter cited a test file as part of a concept's "Authoritative sources" - this appendix is the one place that explains how the test suite as a whole is organized, so those citations make sense in context.

What you need to already know

Any Part II-VIII chapter - this is reference material for all of them, not a narrative continuation.

The idea

Running it

`docs/reference/testing.md` documents running the entire suite, filtering by domain (illustrative examples given for the Agent layer, Engine orchestration, Governance gates, the Execution Planner, Risk gates, and the live-executor subpackage), running a single test, and running with coverage.

Layout

The suite mirrors `src/`'s structure. Some domains live as dedicated subdirectories - `tests/analytics/`, `tests/cognitive/`, `tests/config_layer/`, `tests/data_ingestion/`, `tests/engines/`, `tests/events/`, `tests/exec_telemetry/`, `tests/execution/`, `tests/features/`, `tests/governance/`, `tests/inout/`, `tests/interpreters/`, `tests/journal/`, `tests/mt5_analytics/`, `tests/portfolio/`, `tests/regime/`, `tests/replay/`, `tests/research/`, `tests/runtime/` - while others (Agent, most of Engine orchestration, Governance gates, Planner/Risk) live as flat `test_*.py` files directly under `tests/`, not in a dedicated subdirectory. `tests/fixtures/`, `tests/golden/`, `tests/harness/`, `tests/helpers/`, `tests/manual/`, `tests/production_configs/` hold shared test infrastructure rather than test cases themselves.

Representative patterns

The reference doc documents four representative test-writing patterns (assertion-based, invariant-based, parametrized, and registry-exhaustiveness testing), the conventions the suite enforces beyond just passing (see [Chapter 3](#)'s classification vocabulary - many of this book's "AUDITED" and "CLOSED" statuses trace back to a specific test file asserting the boundary they describe), and a guide for writing a new test.

A doc-drift this appendix surfaces

`docs/reference/testing.md`'s own header states a count - "116 files across 15 directories," recorded 2026-06-12. A direct directory count taken while researching this book found roughly **1,200+ files under tests/** - an order of magnitude higher. This is almost certainly the reference doc's count simply not having been refreshed as the suite grew, not a discrepancy in the suite itself; flagged here per the same `DOC_DRIFT` discipline as [Chapter 7](#)'s 38-vs-39-dim finding, and left unfixed by this book for the same reason - that's a separate, explicit doc-drift turn.

Classification

Concept	Status
The test suite itself	Production, actively maintained
docs/reference/testing.md's file-count header	Stale - DOC_DRIFT, count last refreshed 2026-06-12

Authoritative sources

- docs/reference/testing.md - the full reference (running, config, layout, coverage expectations, patterns, conventions, CI/regression).
- tests/conftest.py - shared fixtures and pytest configuration hooks.
- pyproject.toml - the pytest configuration itself.

Unresolved questions

- The exact current file/directory count for tests/ should be re-verified with a fresh count before testing.md's header is corrected - this appendix's "~1,200+" figure is itself an approximate count taken during this book's research, not a precise audit.

Previous: [Chapter 23 - Multi-LLM Coordination](#) * **Next:** [Appendix A2 - Unresolved Questions](#)
Related: [Chapter 03 - How This Book Fits](#)
Memory: none dedicated.

Appendix A2 - Unresolved Questions (Rollup)

Status of this chapter: Written (updated Grok review pass, 2026-08-07)

Why this chapter exists

Per this book's own charter: "if understanding is impossible, do not invent." Every chapter that hit a genuine gap in evidence recorded it locally rather than guessing. This appendix collects those in one place - and, after the Grok review pass, also records what was **closed in the book** without changing production code.

What you need to already know

This is a reference appendix, not a narrative chapter - each item links back to the chapter that owns the full context.

Resolved in the book (Grok review pass - no code changes)

Item	Resolution	Chapter
INOUT <code>src/inout/</code> vs <code>archive/inout_legacy/</code>	Name collision. Current <code>src/inout/</code> = data fetchers only; archived tree = parallel execution rail retired 2026-05-02	Ch.15
Program 9 missing from research chapter	Chaptered from pre-registration (<code>docs/research/preregistration-program-9.md</code>); run verdicts still artifact-bound	Ch.19
ATR / F-057 "flag only" passivity	Proposed remediation options + trade-offs added; still OPEN pending owner	Ch.13
Measurement Contract gap	5-step seal path proposed; still 0 sealed instances in-repo unless registry says otherwise	Ch.20
MSIP "no treatment"	Orientation brief in governance field guide	Ch.18
Density / meta overwhelm	Quick Start chapter added	Ch.00
22 un-chaptered packages	Orientation map added (not full chapters)	Ch.04

Still open - book-scope gaps

- **Repository Encyclopedia** - charter in [Chapter 24](#);
E0-E6 + E1b + JSONL twin DONE (`docs/book/encyclopedia/`, including `encyclopedia_rows.jsonl`). Optional later: relationship graph edges in JSONL; agent query UI.
- **Full-depth Part VI-VIII** - chapters 16-23 remain orientation-level relative to their source corpora (~250 governance files, ~475 research files). Deepening is expected future book work.
- **Optional MSIP dedicated chapter** - brief exists; full gate-by-gate narrative only if MSIP re-enters active execution.
- **Program 9 numeric verdict** - design is chaptered; Stage-1/Stage-2 results must still be read from artifacts / findings, not assumed.

- **Program 7 (Open Interest)** - named as out-of-scope for Program 6; status not independently verified as run.
- **Deeper Part VIII implementation walk** - agent/control-plane/multi-LLM chapters improved slightly but are not full replacements for docs/reference/agent-reference.md.

Still open - real repository issues (code/doc; not fixed by this book pass)

User instruction for this pass: **no code changes**. These remain tracked, not remediated.

- docs/reference/schemas.md §4 still documents the 38-dim v3.0 feature tuple; live code is 39-dim v4.0. Textbook DOC_DRIFT. See [Chapter 7](#). Recommended separate Doc Sync turn - not part of this PDF/book narrative pass.
- docs/reference/testing.md file-count header ("116 files across 15 directories," dated 2026-06-12) is stale vs ~1,200+ files under tests/. See [Appendix A1](#).
- ATR unit mismatch in compute_crt_levels - real geometry defect; options A/B/C in [Chapter 13](#); awaiting owner decision.
- F-057 crtconfig programmatic split-brain - OPEN; options 1/2/3 in Chapter 13; "backtest Phase 2."
- Research Measurement Contract - schema frozen, 0 sealed MC-* (verify registry); path in [Chapter 20](#).

Verification gaps (assumed from docs; not re-proven this pass)

- Whether Wyckoff / Market Profile / Order Flow interpreters have code beyond scaffolding ([Chapter 9](#)).
- Whether every tool in docs/reference/agent-reference.md is end-to-end reachable ([Chapter 21](#)).
- Full body of docs/reference/architecture.md beyond header/orientation ([Chapter 4](#)).

Out of scope for this pass (explicit)

- Fixing schemas.md / testing.md primary docs (Doc Sync turn).
- Implementing ATR or F-057 remediations (code + parity + promotion).
- Sealing MC-* instances or running Program 9 Stage drivers.
- Adding CLAUDE.md companion-row (already present from an earlier session).

Previous: [Appendix A1 - Testing the System](#) * **Next:** - (last chapter; return to [the Table of Contents](#))

Related: every chapter linked above.

Memory: none dedicated - this appendix is the book's own open-loop tracker.